

The Haira Programming Language

Language Specification

Version 1.0 Draft

Haira Team

February 21, 2026

© 2024-2026 Haira Team

This specification is licensed under CC BY 4.0.

Document Version: 1.0 Draft

Language Version: 1.0

Last Updated: February 21, 2026

Contents

Preface	15
I Core Language	17
1 Overview	19
1.1 Hello, Agent	19
1.2 What Haira Replaces	20
1.3 Language Characteristics	20
1.3.1 Four Agentic Primitives	20
1.3.2 Explicit Imports	20
1.3.3 Go-Style Error Handling	20
1.3.4 Agents and Tools	21
1.3.5 Workflows with Triggers	21
1.3.6 Streaming and Conversations	22
1.4 Type System	22
1.5 Control Flow	22
1.6 Functions	23
1.7 Concurrency	23
1.8 Pipe Operator	24
1.9 A Complete Example	24
1.10 Implementation Status	25
2 Lexical Structure	27
2.1 Source Encoding	27
2.2 Line Structure	27
2.2.1 Line Continuation	27
2.3 Comments	28
2.3.1 Single-Line Comments	28
2.3.2 Multi-Line Comments	28
2.3.3 Documentation Comments	28
2.4 Tokens	28
2.5 Keywords	28
2.6 Identifiers	29
2.6.1 Naming Conventions	29
2.7 Literals	29
2.7.1 Integer Literals	29
2.7.2 Floating-Point Literals	30
2.7.3 Boolean Literals	30
2.7.4 String Literals	31
2.7.5 String Interpolation	31

2.7.6	Raw Strings	31
2.7.7	Triple-Quoted Strings	32
2.7.8	Nil Literal	32
2.8	Operators	32
2.8.1	Arithmetic Operators	32
2.8.2	Comparison Operators	33
2.8.3	Logical Operators	33
2.8.4	Assignment Operators	33
2.8.5	Bitwise Operators	33
2.8.6	Bitwise Assignment Operators	33
2.8.7	Other Operators	33
2.9	Delimiters	33
2.10	Whitespace	33
2.11	Lexical Grammar Summary	33
3	Types	37
3.1	Type System Overview	37
3.2	Primitive Types	37
3.2.1	Integer Types	37
3.2.2	Floating-Point Types	38
3.2.3	Boolean Type	38
3.2.4	String Type	38
3.3	Composite Types	38
3.3.1	Arrays	38
3.3.2	Maps	39
3.3.3	Tuples	39
3.4	Option Types	40
3.4.1	Unwrapping Options	40
3.5	Struct Types	40
3.5.1	Struct Methods	41
3.5.2	Nested Structs	41
3.6	Enum Types	42
3.6.1	Enums with Associated Data	42
3.7	Type Aliases	43
3.8	Function Types	43
3.9	Generic Types	43
3.9.1	Type Constraints	44
3.10	Stream Types	44
3.10.1	Stream Type Syntax	44
3.10.2	Consuming Streams	45
3.10.3	Streams as Return Types	45
3.10.4	Stream Properties	45
3.11	The Error Type	45
3.12	Type Inference	46
3.13	Structural Typing	46
3.14	Type Grammar	47
4	Expressions	49
4.1	Primary Expressions	49
4.1.1	Literals	49
4.1.2	Identifiers	49
4.1.3	Parenthesized Expressions	49

4.2	Arithmetic Expressions	49
4.2.1	Integer Division	50
4.2.2	Modulo	50
4.3	Comparison Expressions	50
4.3.1	Equality	50
4.4	Logical Expressions	50
4.4.1	Short-Circuit Evaluation	50
4.5	Bitwise Expressions	51
4.5.1	Bitwise AND	51
4.5.2	Bitwise OR	51
4.5.3	Bitwise XOR	51
4.5.4	Bitwise NOT	51
4.5.5	Left Shift	52
4.5.6	Right Shift	52
4.5.7	Logical Right Shift	52
4.5.8	Common Bit Manipulation Patterns	52
4.5.9	Compound Bitwise Assignment	53
4.6	String Expressions	53
4.6.1	Concatenation	53
4.6.2	String Interpolation	53
4.7	Array and Map Expressions	54
4.7.1	Array Literals	54
4.7.2	Map Literals	54
4.7.3	Index Access	54
4.8	Field Access	54
4.9	Function Calls	54
4.9.1	Chained Calls	54
4.10	Pipe Expressions	55
4.10.1	Pipe with Arguments	55
4.10.2	Complex Pipelines	55
4.11	Range Expressions	55
4.12	Conditional Expressions	56
4.12.1	If Expression	56
4.12.2	Match Expression	56
4.13	Block Expressions	56
4.14	Struct Instantiation	56
4.15	Closure Expressions	57
4.16	Option Expressions	57
4.16.1	Nil Coalescing	57
4.16.2	Optional Chaining	57
4.17	Type Assertions	57
4.18	Operator Precedence	57
4.19	Expression Grammar	58
5	Statements	61
5.1	Variable Declarations	61
5.1.1	With Inference	61
5.1.2	With Explicit Type	61
5.1.3	Multiple Declarations	61
5.1.4	Uninitialized Variables	61
5.2	Assignment Statements	61
5.2.1	Multiple Assignment	62

5.2.2	Field Assignment	62
5.2.3	Index Assignment	62
5.3	Expression Statements	62
5.4	If Statements	62
5.4.1	Condition Binding	63
5.5	For Statements	63
5.5.1	Range-Based For	63
5.5.2	Collection Iteration	63
5.5.3	String Iteration	63
5.6	While Statements	64
5.7	Match Statements	64
5.7.1	Basic Matching	64
5.7.2	Multiple Patterns	64
5.7.3	Range Patterns	64
5.7.4	Guard Clauses	65
5.7.5	Destructuring Patterns	65
5.7.6	Match Exhaustiveness	65
5.8	Block Statements	66
5.9	Return Statements	66
5.9.1	Implicit Return	66
5.10	Break Statements	66
5.10.1	Labeled Break	67
5.11	Continue Statements	67
5.11.1	Labeled Continue	67
5.12	Import Statements	67
5.13	Defer Statements	68
5.13.1	Multiple Defers	68
5.14	Statement Grammar	68
6	Functions	71
6.1	Function Declarations	71
6.1.1	Basic Examples	71
6.1.2	No Return Type	71
6.2	Parameters	72
6.2.1	Required Parameters	72
6.2.2	Default Parameters	72
6.2.3	Named Arguments	72
6.2.4	Variadic Parameters	72
6.3	Return Values	73
6.3.1	Single Return	73
6.3.2	Multiple Returns	73
6.3.3	Implicit Return	73
6.3.4	Early Return	73
6.4	Function Types	73
6.5	Closures	74
6.5.1	Closure Syntax	74
6.5.2	Capture Semantics	74
6.6	Methods	75
6.6.1	Method Semantics	76
6.7	Generic Functions	76
6.7.1	Type Constraints	76
6.8	Higher-Order Functions	77

6.9	Recursion	77
6.9.1	Tail Call Optimization	77
6.10	Function Overloading	78
6.11	The main Function	78
6.12	Function Grammar	78
7	Error Handling	81
7.1	Philosophy	81
7.2	The Error Type	81
7.2.1	Creating Errors	81
7.3	Error Return Pattern	82
7.4	Handling Errors	82
7.4.1	Basic Error Checking	82
7.4.2	Propagating Errors	82
7.4.3	Wrapping Errors	82
7.5	Error Propagation Operator (?)	83
7.6	Try/Catch	84
7.6.1	Try/Catch in Functions	84
7.6.2	When to Use Each Pattern	84
7.7	Orelse Operator	84
7.8	Defer and Errdefer	85
7.8.1	Defer	85
7.8.2	Errdefer	85
7.9	Error Checking Patterns	85
7.9.1	Guard Pattern	85
7.10	Error Utilities	86
7.10.1	Checking Error Types	86
7.10.2	Error Chain Traversal	86
7.11	Option Types and Errors	86
7.12	Panic	87
7.13	Best Practices	87
7.14	Error Handling Grammar	87
II	Concurrency and Modules	89
8	Concurrency	91
8.1	Concurrency Model	91
8.2	Spawn	91
8.2.1	Spawning Functions	91
8.3	Channels	92
8.3.1	Sending and Receiving	92
8.3.2	Basic Channel Example	92
8.3.3	Buffered Channels	92
8.3.4	Closing Channels	93
8.3.5	Checking Channel State	93
8.4	Select	93
8.4.1	Select with Default	94
8.4.2	Select with Timeout	94
8.4.3	Select in Loop	94
8.5	Common Patterns	95
8.5.1	Worker Pool	95

8.5.2	Fan-Out, Fan-In	95
8.5.3	Pipeline	96
8.5.4	Semaphore	97
8.6	Channel Types	98
8.7	Synchronization	98
8.7.1	WaitGroup	98
8.7.2	Mutex	98
8.8	Concurrency Grammar	99
9	Modules and Imports	101
9.1	Module System Overview	101
9.2	Import Syntax	101
9.2.1	Basic Import	101
9.2.2	Aliased Import	101
9.2.3	Selective Import	102
9.2.4	Glob Import	102
9.3	Module Resolution	102
9.3.1	Standard Library	102
9.3.2	Project-Local Imports	103
9.3.3	External Packages	103
9.4	Module Definitions	103
9.4.1	File as Module	103
9.4.2	Visibility: The <code>pub</code> Keyword	104
9.4.3	Package Initialization	104
9.5	Package Structure	104
9.5.1	Single-File Package	104
9.5.2	Multi-File Package	105
9.5.3	The <code>mod.haira</code> File	105
9.6	Import Order	105
9.7	Circular Dependencies	105
9.8	The <code>haira.toml</code> File	106
9.9	Module Grammar	106
10	Standard Library	109
10.1	Core Modules	109
10.2	<code>io</code> Module	109
10.2.1	<code>io</code> Functions	109
10.3	<code>string</code> Module	110
10.3.1	<code>string</code> Functions	110
10.4	<code>array</code> Module	111
10.5	<code>map</code> Module	112
10.6	<code>math</code> Module	112
10.7	<code>conv</code> Module	113
10.8	Extended Modules	114
10.8.1	<code>fs</code> Module	114
10.8.2	<code>os</code> Module	114
10.8.3	<code>time</code> Module	115
10.8.4	<code>json</code> Module	115
10.8.5	<code>http</code> Module	116
10.8.6	<code>regex</code> Module	116
10.8.7	<code>encoding</code> Module	117
10.9	Testing	117

10.9.1	Test Blocks	117
10.9.2	Assertions	117
10.9.3	Running Tests	118
III	Agentic Orchestration	119
11	Providers	121
11.1	Provider Declaration	121
11.1.1	Cloud Providers	121
11.1.2	Local Providers	121
11.2	Provider Fields	122
11.3	Environment Variables	122
11.4	Referencing Providers	122
11.5	Multiple Providers	122
11.6	MCP Providers	123
11.6.1	Stdio Transport	123
11.6.2	SSE Transport	123
11.6.3	MCP Provider Fields	123
11.6.4	Using MCP in Agents	123
11.6.5	MCP Server	124
11.7	Provider Grammar	124
12	Tools	125
12.1	Tool Declaration	125
12.2	Basic Examples	125
12.2.1	HTTP Tool	125
12.2.2	Database Tool	126
12.2.3	External Service Tool	126
12.2.4	Agent-Delegating Tool	126
12.3	Triple-Quote Description	126
12.4	Auto-Generated JSON Schema	127
12.5	Tool Packs (Standard Library)	127
12.6	Error Handling in Tools	128
12.7	Tool Grammar	128
13	Agents	129
13.1	Agent Declaration	129
13.2	Agent Fields	129
13.2.1	Multiline System Prompts	129
13.3	Agent Methods	130
13.3.1	.ask() — Text In, Text Out	130
13.3.2	.run() — Structured In, Structured Out	130
13.3.3	.stream() — Text In, Streaming Out	131
13.4	Memory	131
13.4.1	No Memory (Default)	131
13.4.2	Conversation Memory	131
13.4.3	Summary Memory	131
13.4.4	Memory Types	132
13.5	Sessions	132
13.6	Handoffs	132
13.6.1	Declaring Handoffs	132

13.6.2	How Handoffs Work	133
13.6.3	.ask() — Automatic Handoffs	133
13.6.4	.run() — Manual Handoff Control	134
13.6.5	Handoff Chains	134
13.7	Tool-Calling Loop	134
13.8	Error Handling	135
13.9	Examples	135
13.9.1	Stateless Classifier	135
13.9.2	Research Agent with Tools	136
13.9.3	Conversational Assistant	136
13.10	Agent Grammar	137
14	Workflows	139
14.1	Workflow Declaration	139
14.2	Triggers	139
14.2.1	@webhook — HTTP Endpoint	139
14.2.2	@websocket — Bidirectional Streaming	140
14.2.3	@cron — Scheduled Execution	140
14.2.4	@event — Event-Driven	140
14.2.5	@manual — CLI Only	140
14.2.6	No Trigger (Sub-Workflow)	141
14.3	Sequential Steps	141
14.4	Named Steps	141
14.4.1	Syntax	141
14.4.2	Transparent Scope	142
14.4.3	Automatic Telemetry	142
14.4.4	Parallel Steps	143
14.4.5	Real-World Example	143
14.5	Parallel Execution	144
14.6	Branching	145
14.7	Loops	145
14.8	Sub-Workflows	146
14.9	Error Handling	146
14.10	Running Workflows	147
14.10.1	From main()	147
14.10.2	Server Mode	147
14.10.3	CLI	147
14.11	Orchestration Patterns	147
14.11.1	Sequential Chain	147
14.11.2	Parallel Fan-Out	148
14.11.3	Router / Dispatcher	148
14.11.4	Handoff	148
14.11.5	Iterative Refinement	148
14.11.6	Supervisor	149
14.11.7	Voting / Consensus	149
14.11.8	Human-in-the-Loop	149
14.11.9	Pattern Summary	150
14.12	Workflow Grammar	151

IV	Implementation	153
15	Complete Grammar	155
15.1	Notation	155
15.2	Lexical Grammar	155
15.2.1	Characters	155
15.2.2	Whitespace and Comments	155
15.2.3	Identifiers	156
15.2.4	Keywords	156
15.2.5	Literals	156
15.2.6	Operators	157
15.2.7	Delimiters	157
15.3	Syntactic Grammar	157
15.3.1	Program Structure	157
15.3.2	Imports	158
15.3.3	Core Declarations	158
15.3.4	Attributes	159
15.3.5	Decorators	159
15.3.6	Provider Declarations	159
15.3.7	Tool Declarations	159
15.3.8	Agent Declarations	160
15.3.9	Workflow Declarations	160
15.3.10	Types	160
15.3.11	Statements	161
15.3.12	Expressions	163
15.3.13	Patterns	165
15.4	Grammar Summary	165
15.4.1	Entry Points	165
15.4.2	Precedence (High to Low)	165
15.4.3	Associativity	166
16	Compiler Architecture	167
16.1	Overview	167
16.2	Lexer	168
16.2.1	Token Types	168
16.2.2	Lexer Features	168
16.3	Parser	169
16.3.1	AST Nodes	169
16.3.2	Operator Precedence	170
16.4	Resolver	170
16.5	Checker	170
16.5.1	Type System	171
16.5.2	Agentic Validation	171
16.6	Code Generation	171
16.6.1	Emission Strategy	172
16.6.2	Agent Initialization Order	172
16.6.3	Workflow Code Generation	172
16.6.4	Tool Schema Generation	172
16.6.5	Import Detection	173
16.6.6	Go Compilation	173
16.7	Runtime Library	173
16.7.1	Runtime Structure	173

16.7.2	Provider Management	174
16.7.3	Agent Execution Loop	174
16.7.4	Workflow Orchestration	174
16.7.5	Session Management	174
16.8	Project Structure	175
16.9	CLI Commands	175
16.10	Error Messages	176
16.10.1	Missing Tool Description	176
16.10.2	Agent Referencing Undefined Provider	176
16.10.3	Undefined Variable	177
16.10.4	Workflow with Invalid Trigger	177
16.11	Runtime Bundling	177
A	Chatbot Patterns	179
A.1	Simple REST Chatbot	179
A.2	Streaming WebSocket Chatbot	180
A.3	Dual-Mode Chatbot	180
A.4	RAG Chatbot	181
A.5	Multi-Agent Routing	182
A.6	Agent Handoffs	183
A.7	Chatbot with Structured Actions	184
A.8	Best Practices	184
B	Generative UI	187
B.1	Motivation	187
B.2	The <code>@render</code> Decorator	187
B.3	Built-in Component Catalog	188
B.3.1	table — Tabular Data	188
B.3.2	status-card — Result Summary	189
B.3.3	code-block — Source Code or SQL	189
B.3.4	diff — Before/After Comparison	190
B.3.5	key-value — Property List	190
B.3.6	progress — Multi-Step Progress	191
B.3.7	form — Interactive Input	191
B.3.8	Component Summary	192
B.4	Streaming Protocol	192
B.4.1	Current Protocol	193
B.4.2	Extended Protocol	193
B.4.3	Dual Consumption	194
B.5	Agent Behavior with Generative UI	194
B.6	Fallback Behavior	194
B.6.1	Non-Chat Contexts	194
B.6.2	API Consumers	194
B.7	Complete Example	195
B.8	Custom Components	196
B.8.1	Composite Components	197
B.8.2	Custom Web Components	197
B.8.3	Component Resolution	200
B.8.4	Build Process	201
B.8.5	Referencing Components from Tools	202
B.8.6	Theme Integration	202
B.8.7	Interactive Components	203

B.8.8 Example: Custom Schema Diagram	204
B.9 Design Principles	206
B.10 External Frontend API	207
B.10.1 Discovery: GET <code>/_api/</code>	207
B.10.2 Workflow Metadata: GET <code>/_api/{path}</code>	208
B.10.3 Streaming: POST <code>/{path}</code> with SSE	209
B.10.4 Form Submission: POST <code>/{path}</code> with JSON	209
B.10.5 CORS Support	210
B.10.6 External Frontend Example	210
B.10.7 API Parity Guarantee	212
B.11 Grammar Extension	212
C Reserved Keywords	213
D Operator Precedence	215
E Standard Library Reference	217

Preface

HAIRA is a statically-typed, compiled programming language for building AI agents and workflows. It combines the simplicity of Go, the safety of Rust, and the expressiveness of modern functional languages, while making agents, tools, and workflows first-class language constructs—not library imports.

Design Philosophy

1. **Agents and workflows are primitives** – `provider`, `tool`, `agent`, and `workflow` are language keywords, not framework abstractions.
2. **Type-safe orchestration** – Agent inputs, outputs, and tool schemas are verified at compile time.
3. **Go-style simplicity** – Familiar error handling, explicit imports, and straightforward syntax.
4. **Zero boilerplate** – No framework setup, no dependency injection, no configuration files.
5. **Native performance** – Compiles to Go, then to standalone native binaries via `go build`.

Target Domains

HAIRA replaces the entire agentic stack:

- **Python + LangChain/LangGraph** – for code-first agent builders
- **n8n / Make / Zapier** – for workflow automation
- **CrewAI / AutoGen** – for multi-agent systems
- **Custom chatbot backends** – REST and WebSocket with streaming
- **General-purpose programming** – CLI tools, web services, data pipelines

Implementation Roadmap

This specification is organized so that each chapter can be implemented incrementally:

Release	Chapters	Features
v0.1	1–3	Lexer, parser, basic types
v0.2	4–5	Expressions, statements

Release	Chapters	Features
v0.3	6–7	Functions, error handling
v0.4	8	Concurrency primitives (spawn, chan, select)
v0.5	9–10	Modules, standard library
v0.6	11–12	Providers, tools
v0.7	13–14	Agents, workflows
v0.8	15	Chatbot patterns, streaming
v1.0	16–17	Complete grammar, compiler completion

Part I

Core Language

Chapter 1

Overview

HAIRA is a statically-typed, compiled programming language for building AI agents and workflows. It combines the simplicity of Go, the safety of Rust, and purpose-built primitives for agentic orchestration — agents, tools, workflows, and providers are first-class language constructs, not library imports.

HAIRA compiles to native binaries via Cranelift, with no runtime dependencies beyond the standard library.

1.1 Hello, Agent

```
1 import "io"
2
3 provider local {
4     backend: "ollama"
5     host: "localhost:11434"
6     model: "llama3:8b"
7 }
8
9 agent Greeter {
10     provider: local
11     system: "You are a friendly greeter. Keep it short."
12 }
13
14 fn main() {
15     reply, err = Greeter.ask("Say hello to the world!")
16     if err != nil {
17         io.println("Error: " + err.message)
18         return
19     }
20     io.println(reply)
21 }
```

To compile and run:

```
$ haira build hello.haira
$ ./hello
Hello, world! Great to meet you!
```

1.2 What Haira Replaces

HAIRA is designed to replace the fragmented toolchain currently used to build agentic systems:

Current Stack	Haira Equivalent
Python + LangChain/LangGraph	agent + tool + workflow
n8n / Make / Zapier	workflow with @webhook, @cron triggers
CrewAI / AutoGen	Multi-agent workflow composition
FastAPI / Flask	<code>http.Server()</code> with typed endpoints
<code>.env</code> files + API configs	provider blocks

Table 1.1: Haira replaces the fragmented agentic stack

1.3 Language Characteristics

1.3.1 Four Agentic Primitives

HAIRA adds four keywords to a general-purpose language:

- **provider** — Configure LLM backends (Anthropic, OpenAI, Ollama, local models)
- **tool** — Typed functions with LLM-visible descriptions; the compiler auto-generates JSON schemas
- **agent** — LLM-powered entities with a model, system prompt, tools, and optional memory
- **workflow** — Composable pipelines with triggers, typed I/O, and native concurrency

1.3.2 Explicit Imports

All dependencies are explicitly declared at the top of each file:

```

1 import "io"
2 import "json"
3 import "tools/http"
4 import "tools/postgres"

```

1.3.3 Go-Style Error Handling

Functions that can fail return a tuple of result and error:

```

1 import "io"
2 import "fs"
3
4 fn main() {
5     content, err = fs.read_file("config.json")
6     if err != nil {
7         io.println("Error: " + err.message)
8         return
9     }
10    io.println(content)
11 }

```

1.3.4 Agents and Tools

Agents are declared with a model, system prompt, and tools. Tools are typed functions with descriptions:

```
1 import "io"
2 import "tools/http"
3
4 provider anthropic {
5     api_key: env("ANTHROPIC_API_KEY")
6     model: "claude-sonnet-4-20250514"
7 }
8
9 tool search(query: string) -> [Result] {
10     ""Search the web for information""
11
12     resp, err = http.get("https://api.search.com?q={query}")
13     if err != nil { return [], err }
14     return json.decode(resp.body, [Result])
15 }
16
17 agent Researcher {
18     provider: anthropic
19     system: "You are a thorough researcher. Always cite
20             sources."
21     tools: [search]
22     temperature: 0.2
23 }
24
25 fn main() {
26     answer, err = Researcher.ask("What is quantum computing?")
27     if err != nil {
28         io.println("Error: " + err.message)
29         return
30     }
31     io.println(answer)
32 }
```

1.3.5 Workflows with Triggers

Workflows are functions with triggers. They can be exposed as webhooks, scheduled with cron, or triggered by events:

```
1 @webhook("/summarize")
2 workflow Summarizer(url: string) -> Summary {
3     content = fetch_page(url)
4     summary = Researcher.ask("Summarize this: {content}")
5     return Summary{ text: summary, word_count: summary.len() }
6 }
7
8 fn main() {
9     server = http.Server([Summarizer])
10    io.println("Running on :8080")
11    server.listen(8080)
12 }
```

1.3.6 Streaming and Conversations

Agents support streaming responses and session-based memory for chatbot use cases:

```

1  agent Assistant {
2      provider: anthropic
3      system: "You are a helpful assistant."
4      memory: conversation(max_turns: 50)
5  }
6
7  @websocket("/chat")
8  workflow Chat(message: string, session_id: string) ->
9      stream<string> {
10     return Assistant.stream(message, session: session_id)
11 }

```

1.4 Type System

HAIRA uses structural typing with type inference for local variables:

```

1  // Type inferred as int
2  count = 42
3
4  // Explicit type annotation
5  name: string = "Haira"
6
7  // Struct definition
8  struct Point {
9      x: float
10     y: float
11 }
12
13 // Option type (nullable)
14 maybe_value: int? = nil
15
16 // Arrays and maps
17 numbers: []int = [1, 2, 3]
18 scores: [string:int] = ["alice": 100, "bob": 95]

```

1.5 Control Flow

```

1  import "io"
2
3  fn main() {
4      x = 10
5      if x > 5 {
6          io.println("Large")
7      } else {
8          io.println("Small")
9      }
10
11     for i in 0..5 {
12         io.println(i)
13     }
14 }

```

```

13     }
14
15     status = 200
16     match status {
17         200 => io.println("OK")
18         404 => io.println("Not Found")
19         _  => io.println("Unknown")
20     }
21 }

```

1.6 Functions

Functions are declared with **fn**:

```

1  fn add(a: int, b: int) -> int {
2      return a + b
3  }
4
5  fn divide(a: int, b: int) -> (int, Error?) {
6      if b == 0 {
7          return 0, Error{message: "division by zero"}
8      }
9      return a / b, nil
10 }

```

1.7 Concurrency

HAIRA provides Go-style concurrency with **spawn** and channels:

```

1  import "io"
2
3  fn main() {
4      ch = chan<int>(10)
5
6      spawn {
7          for i in 0..5 {
8              ch <- i
9          }
10         close(ch)
11     }
12
13     for value in ch {
14         io.println(value)
15     }
16 }

```

Inside workflows, **spawn** blocks run steps in parallel:

```

1  @manual
2  workflow ParallelReview(article: Article) -> [Review] {
3      reviews = spawn {
4          Reviewer.run(ReviewRequest{ article: article, focus:
5              "accuracy" })
6      }
7  }

```

```

5         Reviewer.run(ReviewRequest{ article: article, focus:
           "clarity" })
6         Reviewer.run(ReviewRequest{ article: article, focus:
           "style" })
7     }
8     return reviews
9 }

```

1.8 Pipe Operator

The pipe operator `|>` enables functional composition:

```

1  import "io"
2  import "string"
3  import "array"
4
5  fn main() {
6      result = "  hello, world  "
7          |> string.trim
8          |> string.to_upper
9          |> string.split(", ")
10         |> array.first
11
12      io.println(result) // "HELLO"
13 }

```

1.9 A Complete Example

A multi-agent workflow that researches a topic, writes an article, and reviews it:

```

1  import "io"
2
3  provider anthropic {
4      api_key: env("ANTHROPIC_API_KEY")
5      model: "claude-sonnet-4-20250514"
6  }
7
8  struct Article { title: string, body: string, sources: [string]
9  }
10 struct Review { aspect: string, score: float, notes: string }
11
12 tool search(query: string) -> [string] {
13     """Search the web and return relevant URLs"""
14
15     resp, err = http.get("https://api.search.com?q={query}")
16     if err != nil { return [], err }
17     return json.decode(resp.body, [string])
18 }
19
20 agent Researcher {
21     provider: anthropic
22     system: "You are a thorough researcher."
23     tools: [search]
24     temperature: 0.2
25 }

```



```

24 }
25
26 agent Writer {
27     provider: anthropic
28     system: "You are a clear technical writer."
29     temperature: 0.7
30 }
31
32 agent Reviewer {
33     provider: anthropic
34     system: "You are a critical but constructive editor."
35     temperature: 0.1
36 }
37
38 @manual
39 workflow WriteArticle(topic: string) -> Article {
40     research = Researcher.ask("Research this topic thoroughly:
41                               {topic}")
42
43     draft: Article, err = Writer.run(
44         { research: research, topic: topic }
45     ) -> Article
46     if err != nil { return Article{}, err }
47
48     reviews = spawn {
49         Reviewer.ask("Review for accuracy: {draft.body}")
50         Reviewer.ask("Review for clarity: {draft.body}")
51     }
52
53     io.println("Reviews complete: {reviews.len()}")
54     return draft
55 }
56
57 fn main() {
58     article, err = WriteArticle("The future of agentic
59                               programming")
60     if err != nil {
61         io.println("Failed: " + err.message)
62         return
63     }
64     io.println(article.title)
65     io.println(article.body)
66 }

```

1.10 Implementation Status

This specification documents the complete HAIRA language. Implementation proceeds incrementally:

Feature	Chapter	Target Release
Lexer & Parser	2–3	v0.1
Expressions & Statements	4–5	v0.2
Functions & Error Handling	6–7	v0.3
Concurrency	8	v0.4
Modules & Stdlib	9–10	v0.5
Providers & Tools	11–12	v0.6
Agents	13	v0.7
Workflows & Chatbot Patterns	14–15	v0.8
Full Grammar & Compiler	16–17	v1.0

Table 1.2: Implementation roadmap by chapter

Chapter 2

Lexical Structure

This chapter defines the lexical grammar of HAIRA: the rules for converting source text into tokens.

2.1 Source Encoding

HAIRA source files are encoded in UTF-8. The file extension is `.haira`.

```
hello.haira
utils/string_helpers.haira
```

2.2 Line Structure

HAIRA is a line-oriented language. Statements are terminated by newlines, not semicolons.

```
1 x = 1
2 y = 2
3 z = x + y
```

2.2.1 Line Continuation

Lines can be continued with a trailing backslash or when the line ends with an operator or open bracket:

```
1 // Explicit continuation
2 long_result = very_long_function_name(arg1, arg2) \
3     + another_function(arg3)
4
5 // Implicit continuation (ends with operator)
6 total = first_value +
7     second_value +
8     third_value
9
10 // Implicit continuation (open bracket)
11 users = [
12     "alice",
13     "bob",
14     "charlie"
15 ]
```

2.3 Comments

2.3.1 Single-Line Comments

```
1 // This is a single-line comment
2 x = 42 // Inline comment
```

2.3.2 Multi-Line Comments

```
1 /*
2     This is a multi-line comment.
3     It can span multiple lines.
4 */
```

Multi-line comments can be nested:

```
1 /*
2     Outer comment
3     /* Nested comment */
4     Still in outer comment
5 */
```

2.3.3 Documentation Comments

```
1 /// This is a documentation comment for the following item.
2 /// It can span multiple lines.
3 fn calculate_area(width: float, height: float) -> float {
4     return width * height
5 }
```

2.4 Tokens

The lexer produces the following token types:

1. Keywords
2. Identifiers
3. Literals (numbers, strings, booleans)
4. Operators
5. Delimiters

2.5 Keywords

The following identifiers are reserved as keywords:

<i>General-purpose keywords</i>				
and	as	break	catch	chan
continue	defer	else	enum	export
false	fn	for	from	if
impl	import	in	match	nil
not	or	pub	return	select
spawn	struct	trait	true	try
type	while			
<i>Agentic keywords</i>				
agent	provider	tool	workflow	

Table 2.1: Reserved keywords

Note

The four agentic keywords (**agent**, **provider**, **tool**, **workflow**) are first-class language constructs for building AI-powered systems. See Chapters [11–14](#).

2.6 Identifiers

Identifiers name variables, functions, types, and modules.

```

1 identifier    = letter { letter | digit | "_" } ;
2 letter       = "a".."z" | "A".."Z" | "_" ;
3 digit        = "0".."9" ;
```

Valid identifiers:

```

x
name
userName
user_name
_private
Point3D
MAX_SIZE
```

Invalid identifiers:

```

3d_point    // Cannot start with digit
my-name     // Hyphens not allowed
class       // Reserved keyword (if added)
```

2.6.1 Naming Conventions

2.7 Literals

2.7.1 Integer Literals

```

1 integer_literal = decimal_literal
2                 | hex_literal
```

Entity	Convention
Variables	snake_case
Functions	snake_case
Types/Structs	PascalCase
Constants	SCREAMING_SNAKE_CASE
Modules	snake_case

Table 2.2: Naming conventions

```
3 | octal_literal
4 | binary_literal ;
5
6 decimal_literal = digit { digit | "_" } ;
7 hex_literal    = "0" ("x"|"X") hex_digit { hex_digit | "_" } ;
8 octal_literal  = "0" ("o"|"O") octal_digit { octal_digit | "_"
9   } ;
10 binary_literal = "0" ("b"|"B") binary_digit { binary_digit |
11   "_" } ;
12 hex_digit      = digit | "a".."f" | "A".."F" ;
13 octal_digit    = "0".."7" ;
14 binary_digit   = "0" | "1" ;
```

Examples:

```
1 decimal = 42
2 with_underscores = 1_000_000
3 hex = 0xFF
4 octal = 0o755
5 binary = 0b1010_1100
```

Note

All integer literals are of type `int` (64-bit signed).

2.7.2 Floating-Point Literals

```
1 float_literal = decimal_literal "." decimal_literal [ exponent ]
2               | decimal_literal exponent ;
3
4 exponent = ("e"|"E") ["+"|"-"] decimal_literal ;
```

Examples:

```
1 pi = 3.14159
2 scientific = 6.022e23
3 negative_exp = 1.6e-19
```

2.7.3 Boolean Literals



```

1 is_valid = true
2 is_empty = false

```

2.7.4 String Literals

```

1 string_literal = ''' { string_char } ''' ;
2 string_char    = any_char - (''' | '\'' | newline)
3                | escape_sequence ;

```

Escape sequences:

Sequence	Meaning
<code>\n</code>	Newline (LF)
<code>\r</code>	Carriage return (CR)
<code>\t</code>	Horizontal tab
<code>\\</code>	Backslash
<code>\"</code>	Double quote
<code>\0</code>	Null character
<code>\xNN</code>	Hex byte (00-FF)
<code>\u{NNNN}</code>	Unicode code point

Table 2.3: String escape sequences

Examples:

```

1 simple = "Hello, World!"
2 with_escapes = "Line 1\nLine 2"
3 unicode = "Hello \u{1F44B}" // Wave emoji

```

2.7.5 String Interpolation

Strings can contain interpolated expressions using `${...}`:

```

1 name = "Alice"
2 age = 30
3 greeting = "Hello, ${name}! You are ${age} years old."

```

Implementation Note

String interpolation is desugared to concatenation during parsing:

```

1 greeting = "Hello, " + name + "! You are " +
  to_string(age) + " years old."

```

2.7.6 Raw Strings

Raw strings preserve backslashes literally:

```

1 regex_pattern = r"^\d{3}-\d{4}$"

```

```
2 windows_path = r"C:\Users\Alice\Documents"
```

2.7.7 Triple-Quoted Strings

Triple-quoted strings use `"""..."""` for multi-line content:

```
1 query = """
2     SELECT *
3     FROM users
4     WHERE active = true
5     """
```

The leading whitespace common to all lines is stripped.

Triple-quoted strings are also used for:

- Tool descriptions (mandatory, see Chapter 12)
- Agent system prompts (optional, for long prompts, see Chapter 13)

```
1 tool search(query: string) -> [Result] {
2     """Search the web for information"""
3     // ...
4 }
5
6 agent Assistant {
7     provider: anthropic
8     system: """
9         You are a helpful assistant.
10        Always be polite and concise.
11    """
12 }
```

2.7.8 Nil Literal

```
1 empty: User? = nil
```

2.8 Operators

2.8.1 Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction (or unary negation)
*	Multiplication
/	Division
%	Modulo (remainder)

Operator	Description
==	Equal
!=	Not equal
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

2.8.2 Comparison Operators

2.8.3 Logical Operators

Operator	Description
and, &&	Logical AND
or,	Logical OR
not, !	Logical NOT

2.8.4 Assignment Operators

Operator	Description
=	Assignment
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign

2.8.5 Bitwise Operators

2.8.6 Bitwise Assignment Operators

2.8.7 Other Operators

2.9 Delimiters

2.10 Whitespace

Whitespace (spaces, tabs) is used to separate tokens but is otherwise insignificant, except for indentation in multi-line strings.

```

1 x=1+2           // Valid
2 x = 1 + 2       // Also valid, preferred for readability

```

2.11 Lexical Grammar Summary

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT (complement)
<<	Left shift
>>	Right shift (arithmetic for signed, logical for unsigned)
>>>	Logical right shift (always fills with zeros)

Operator	Description
&=	Bitwise AND and assign
=	Bitwise OR and assign
^=	Bitwise XOR and assign
<<=	Left shift and assign
>>=	Right shift and assign
>>>=	Logical right shift and assign

```

1 token = keyword
2     | identifier
3     | integer_literal
4     | float_literal
5     | string_literal
6     | operator
7     | delimiter ;
8
9 keyword = "agent" | "and" | "as" | "break" | "catch" | "chan"
10         | "continue" | "defer" | "else" | "enum" | "false"
11         | "fn" | "for" | "from" | "if" | "import" | "in"
12         | "match" | "nil" | "not" | "or" | "provider" | "return"
13         | "select" | "spawn" | "struct" | "tool" | "true" |
14         | "try"
15         | "type" | "while" | "workflow" ;
16
17 identifier = letter { letter | digit | "_" } ;
18
19 operator = "+" | "-" | "*" | "/" | "%"
20         | "==" | "!=" | "<" | ">" | "<=" | ">="
21         | "and" | "or" | "not" | "&&" | "||" | "!"
22         | "&" | "|" | "^" | "~" | "<<" | ">>" | ">>>"
23         | "=" | "+=" | "-=" | "*=" | "/="
24         | "&=" | "|=" | "^=" | "<<=" | ">>=" | ">>>="
25         | ">" | ".." | "..=" | "<-" | "->" | "=>"
26         | "@" ;
27
28 delimiter = "(" | ")" | "[" | "]" | "{" | "}"
29         | "," | ":" | "." | "?" ;

```

Operator	Description
>	Pipe (function composition)
..	Range (exclusive end)
..=	Range (inclusive end)
<-	Channel send
->	Function return type
=>	Match arm
@	Decorator (workflow triggers)

Delimiter	Usage
()	Grouping, function calls, tuples
[]	Array literals, indexing
{ }	Blocks, struct literals, maps
,	Separator
:	Type annotations, map entries
.	Member access
?	Optional type marker

Chapter 3

Types

HAIRA is a statically-typed language with structural typing and type inference for local variables. All types are known at compile time.

3.1 Type System Overview

- **Static typing** – All types resolved at compile time
- **Structural typing** – Type compatibility based on structure, not name
- **Type inference** – Local variables infer types from initializers
- **Explicit annotations** – Required for function signatures
- **No null** – Option types (T?) replace null pointers

3.2 Primitive Types

3.2.1 Integer Types

Type	Size	Range
int	64 bits	-2^{63} to $2^{63} - 1$
i8	8 bits	-128 to 127
i16	16 bits	-32768 to 32767
i32	32 bits	-2^{31} to $2^{31} - 1$
i64	64 bits	-2^{63} to $2^{63} - 1$
u8	8 bits	0 to 255
u16	16 bits	0 to 65535
u32	32 bits	0 to $2^{32} - 1$
u64	64 bits	0 to $2^{64} - 1$

Table 3.1: Integer types

```
1 count: int = 42
2 byte: u8 = 255
3 offset: i32 = -100
```

Note

The default `int` is an alias for `i64`. Integer literals without explicit type default to `int`.

3.2.2 Floating-Point Types

Type	Size	Precision
<code>float</code>	64 bits	IEEE 754 double
<code>f32</code>	32 bits	IEEE 754 single
<code>f64</code>	64 bits	IEEE 754 double

Table 3.2: Floating-point types

```
1 pi: float = 3.14159
2 temperature: f32 = 98.6
```

3.2.3 Boolean Type

```
1 is_valid: bool = true
2 is_empty: bool = false
```

Boolean values are used in conditionals and logical expressions. Only `true` and `false` are valid boolean values—there is no implicit conversion from other types.

3.2.4 String Type

Strings are immutable sequences of UTF-8 encoded bytes:

```
1 name: string = "Haira"
2 greeting: string = "Hello, " + name
```

String operations:

```
1 import "string"
2
3 s = "Hello, World!"
4 length = string.len(s)           // 13
5 upper = string.to_upper(s)       // "HELLO, WORLD!"
6 parts = string.split(s, ", ")    // ["Hello", "World!"]
```

3.3 Composite Types**3.3.1 Arrays**

Arrays are ordered, homogeneous, dynamically-sized collections:

```
1 // Array literal
2 numbers: []int = [1, 2, 3, 4, 5]
```

```

3
4 // Empty array
5 empty: []string = []
6
7 // Array operations
8 import "array"
9
10 first = numbers[0]           // 1
11 length = array.len(numbers)  // 5
12 numbers = array.push(numbers, 6)

```

```

1 array_type = "[" "]" type ;

```

3.3.2 Maps

Maps are unordered key-value collections:

```

1 // Map literal
2 scores: [string:int] = [
3     "alice": 100,
4     "bob": 95,
5     "charlie": 87
6 ]
7
8 // Empty map
9 empty: [string:User] = [:]
10
11 // Map operations
12 alice_score = scores["alice"] // 100
13 scores["david"] = 92          // Insert

```

```

1 map_type = "[" type ":" type "]" ;

```

Note

Map keys must be comparable types: primitives, strings, or structs containing only comparable fields.

3.3.3 Tuples

Tuples are fixed-size, heterogeneous collections:

```

1 // Tuple type
2 point: (int, int) = (10, 20)
3
4 // Accessing elements
5 x = point.0 // 10
6 y = point.1 // 20
7
8 // Multiple return values use tuples
9 fn divide(a: int, b: int) -> (int, int) {
10     return a / b, a % b

```

```

11 }
12
13 quotient, remainder = divide(17, 5)

```

```

1 tuple_type = "(" type { "," type } ")" ;

```

3.4 Option Types

Option types represent values that may or may not exist, replacing null pointers:

```

1 // Optional value
2 maybe_name: string? = nil
3 maybe_age: int? = 25
4
5 // Checking for nil
6 if maybe_name != nil {
7     io.println(maybe_name)
8 }
9
10 // Default value with 'or'
11 name = maybe_name or "Unknown"

```

```

1 option_type = type "?" ;

```

3.4.1 Unwrapping Options

```

1 value: int? = get_value()
2
3 // Safe unwrap with default
4 result = value or 0
5
6 // Conditional unwrap
7 if value != nil {
8     // value is automatically unwrapped here
9     io.println(value)
10 }
11
12 // Match on option
13 match value {
14     nil => io.println("No value")
15     v => io.println("Got: " + to_string(v))
16 }

```

3.5 Struct Types

Structs are named product types with named fields:

```

1 struct User {
2     id: int

```



```
3     name: string
4     email: string
5     active: bool
6 }
7
8 // Creating a struct
9 user = User{
10     id: 1,
11     name: "Alice",
12     email: "alice@example.com",
13     active: true
14 }
15
16 // Accessing fields
17 io.println(user.name)
18
19 // Updating fields (structs are mutable by default)
20 user.active = false
```

3.5.1 Struct Methods

Methods can be defined on structs:

```
1 struct Rectangle {
2     width: float
3     height: float
4 }
5
6 Rectangle.area() -> float {
7     return self.width * self.height
8 }
9
10 Rectangle.perimeter() -> float {
11     return 2 * (self.width + self.height)
12 }
13
14 // Usage
15 rect = Rectangle{width = 10.0, height = 5.0}
16 io.println(rect.area())           // 50.0
17 io.println(rect.perimeter())      // 30.0
```

3.5.2 Nested Structs

```
1 struct Address {
2     street: string
3     city: string
4     country: string
5 }
6
7 struct Person {
8     name: string
9     address: Address
10 }
11
```

```
12 person = Person{
13     name: "Bob",
14     address: Address{
15         street: "123 Main St",
16         city: "New York",
17         country: "USA"
18     }
19 }
20
21 io.println(person.address.city) // "New York"
```

3.6 Enum Types

Enums define a type with a fixed set of named values:

```
1 enum Color {
2     Red,
3     Green,
4     Blue
5 }
6
7 enum Status {
8     Pending,
9     Active,
10    Completed,
11    Failed
12 }
13
14 // Usage
15 color: Color = Color.Red
16 status: Status = Status.Active
17
18 match color {
19     Color.Red => io.println("Stop")
20     Color.Green => io.println("Go")
21     Color.Blue => io.println("Info")
22 }
```

3.6.1 Enums with Associated Data

Enums can carry associated data (tagged unions):

```
1 enum Result<T> {
2     Ok(T),
3     Err(Error)
4 }
5
6 enum Message {
7     Text(string),
8     Number(int),
9     Pair(int, int)
10 }
11
12 // Usage
13 msg: Message = Message.Text("Hello")
```

```

14
15 match msg {
16     Message.Text(s) => io.println("Text: " + s)
17     Message.Number(n) => io.println("Number: " + to_string(n))
18     Message.Pair(a, b) => io.println("Pair: " + to_string(a) +
19         ", " + to_string(b))

```

3.7 Type Aliases

Type aliases create alternative names for existing types:

```

1 type UserId = int
2 type UserMap = [string:User]
3 type Handler = fn(Request) -> Response
4
5 // Usage
6 id: UserId = 42
7 users: UserMap = [:]

```

3.8 Function Types

Functions are first-class values with types:

```

1 // Function type syntax
2 type Comparator = fn(int, int) -> bool
3 type Transformer = fn(string) -> string
4 type Callback = fn()
5
6 // Higher-order functions
7 fn apply(f: fn(int) -> int, x: int) -> int {
8     return f(x)
9 }
10
11 fn double(n: int) -> int {
12     return n * 2
13 }
14
15 result = apply(double, 21) // 42

```

3.9 Generic Types

Generic types parameterize over other types:

```

1 // Generic struct
2 struct Pair<T, U> {
3     first: T
4     second: U
5 }
6
7 // Usage
8 pair: Pair<int, string> = Pair{first: 1, second: "one"}

```

```

9
10 // Generic function
11 fn swap<T, U>(p: Pair<T, U>) -> Pair<U, T> {
12     return Pair{first: p.second, second: p.first}
13 }
14
15 swapped = swap(pair) // Pair<string, int>

```

3.9.1 Type Constraints

Generics can be constrained to types implementing certain behaviors:

```

1 // Constraint definition
2 constraint Comparable {
3     fn compare(other: Self) -> int
4 }
5
6 // Constrained generic
7 fn max<T: Comparable>(a: T, b: T) -> T {
8     if a.compare(b) > 0 {
9         return a
10    }
11    return b
12 }
13
14 // Multiple constraints
15 fn process<T: Comparable + Printable>(value: T) {
16     io.println(value.to_string())
17 }

```

3.10 Stream Types

The `stream<T>` type represents a lazy, asynchronous sequence of values produced over time. Streams are the primary mechanism for handling token-by-token output from agents and for streaming responses over WebSocket connections.

```

1 // Stream of string tokens from an agent
2 for chunk in Assistant.stream("Tell me a story") {
3     io.print(chunk)
4 }

```

3.10.1 Stream Type Syntax

```

1 stream_type = "stream" "<" type ">" ;

```

A stream is parameterized by the element type it yields:

```

1 // Common stream types
2 stream<string> // Stream of text tokens (agent output)
3 stream<byte>   // Stream of raw bytes
4 stream<Event>  // Stream of typed events

```

3.10.2 Consuming Streams

Streams are consumed with **for** loops. Each iteration yields the next value as it becomes available:

```
1 // Iterate over a stream
2 tokens: stream<string> = Researcher.stream("Explain quantum
3   computing")
4 for token in tokens {
5     io.print(token) // Print each token as it arrives
6 }
```

3.10.3 Streams as Return Types

Functions and workflows can return streams, enabling streaming pipelines:

```
1 // Workflow returning a stream (e.g., for WebSocket)
2 @websocket("/chat")
3 workflow StreamingChat(message: string, session_id: string) ->
4   stream<string> {
5       return Assistant.stream(message, session: session_id)
6   }
```

When a workflow returns `stream<T>`, the runtime streams each element to the client as it is produced—no buffering.

3.10.4 Stream Properties

- **Lazy** – Values are produced on demand, not computed upfront
- **Single-use** – A stream can only be iterated once
- **Asynchronous** – The **for** loop suspends until the next value is available
- **Finite** – Streams eventually terminate (the loop exits when the source is exhausted)

Note

Streams are distinct from channels (`chan<T>`). A channel is a bidirectional communication primitive between concurrent tasks. A stream is a unidirectional, read-only sequence—typically produced by an agent or an external data source.

3.11 The Error Type

The built-in `Error` type represents errors:

```
1 struct Error {
2     message: string
3     code: int?
4     source: Error?
5 }
6
7 // Creating errors
8 err = Error{message: "file not found"}
```

```

9  err_with_code = Error{message: "permission denied", code: 403}
10
11  // Error chaining
12  wrapped = Error{
13      message: "failed to read config",
14      source: err
15  }

```

3.12 Type Inference

HAIRA infers types for local variables from their initializers:

```

1  // Types inferred
2  x = 42                // int
3  pi = 3.14            // float
4  name = "Haira"       // string
5  active = true        // bool
6  numbers = [1, 2, 3]  // []int
7
8  // Explicit annotation when needed
9  count: i32 = 100     // Specific integer type

```

Warning

Type annotations are **required** for:

- Function parameters
- Function return types
- Struct fields
- Uninitialized variables

3.13 Structural Typing

Types are compatible based on structure, not name:

```

1  struct Point2D {
2      x: float
3      y: float
4  }
5
6  struct Vector2D {
7      x: float
8      y: float
9  }
10
11 fn magnitude(p: {x: float, y: float}) -> float {
12     return math.sqrt(p.x * p.x + p.y * p.y)
13 }
14
15 // Both work because they have the same structure
16 point = Point2D{x: 3.0, y: 4.0}

```

```
17 vector = Vector2D{x: 3.0, y: 4.0}
18
19 io.println(magnitude(point))    // 5.0
20 io.println(magnitude(vector))  // 5.0
```

3.14 Type Grammar

```
1  type = primitive_type
2      | array_type
3      | map_type
4      | tuple_type
5      | option_type
6      | function_type
7      | generic_type
8      | stream_type
9      | struct_type
10     | identifier ;
11
12 primitive_type = "int" | "i8" | "i16" | "i32" | "i64"
13                | "u8" | "u16" | "u32" | "u64"
14                | "float" | "f32" | "f64"
15                | "bool" | "string" ;
16
17 array_type = "[" " " type ;
18
19 map_type = "[" type ":" type "]" ;
20
21 tuple_type = "(" type { "," type } ")" ;
22
23 option_type = type "?" ;
24
25 function_type = "fn" "(" [ type_list ] ")" [ "->" type ] ;
26
27 generic_type = identifier "<" type_list ">" ;
28
29 stream_type = "stream" "<" type ">" ;
30
31 type_list = type { "," type } ;
```


Chapter 4

Expressions

Expressions produce values. Every expression in HAIRA has a type determined at compile time.

4.1 Primary Expressions

4.1.1 Literals

```
1 42           // int literal
2 3.14         // float literal
3 "hello"      // string literal
4 true        // bool literal
5 nil         // nil literal
```

4.1.2 Identifiers

```
1 x           // Variable reference
2 user.name   // Field access
3 array[0]    // Index access
```

4.1.3 Parenthesized Expressions

```
1 (1 + 2) * 3 // Grouping for precedence
```

4.2 Arithmetic Expressions

```
1 a + b       // Addition
2 a - b       // Subtraction
3 a * b       // Multiplication
4 a / b       // Division
5 a % b       // Modulo (remainder)
6 -a         // Unary negation
```

4.2.1 Integer Division

Integer division truncates toward zero:

```
1 7 / 3          // 2
2 -7 / 3         // -2
3 7 / -3         // -2
```

4.2.2 Modulo

The modulo operator returns the remainder:

```
1 7 % 3          // 1
2 -7 % 3         // -1
3 7 % -3         // 1
```

4.3 Comparison Expressions

```
1 a == b          // Equal
2 a != b          // Not equal
3 a < b           // Less than
4 a > b           // Greater than
5 a <= b          // Less or equal
6 a >= b          // Greater or equal
```

All comparison expressions produce `bool` values.

4.3.1 Equality

Equality (`==`) compares by value for primitives and by structural equality for composite types:

```
1 1 == 1          // true
2 "hello" == "hello" // true
3 [1, 2] == [1, 2] // true
4 User{id: 1} == User{id: 1} // true (structural)
```

4.4 Logical Expressions

```
1 a and b          // Logical AND (also &&)
2 a or b           // Logical OR (also ||)
3 not a            // Logical NOT (also !)
```

4.4.1 Short-Circuit Evaluation

Logical operators use short-circuit evaluation:

```
1 // 'b' is not evaluated if 'a' is false
2 a and b
```

```

3
4 // 'b' is not evaluated if 'a' is true
5 a or b

```

4.5 Bitwise Expressions

Bitwise operators perform bit-level operations on integer types. All bitwise operators work on any integer type (int, i8–i64, u8–u64).

```

1 a & b           // Bitwise AND
2 a | b           // Bitwise OR
3 a ^ b           // Bitwise XOR
4 ~a              // Bitwise NOT (one's complement)
5 a << n          // Left shift by n bits
6 a >> n          // Right shift by n bits
7 a >>> n         // Logical right shift by n bits

```

4.5.1 Bitwise AND

Sets each bit to 1 only if both corresponding bits are 1:

```

1 0b1100 & 0b1010 // 0b1000 (8)
2 flags & MASK     // Extract bits using mask
3 value & 0xFF      // Keep only lowest 8 bits

```

4.5.2 Bitwise OR

Sets each bit to 1 if at least one corresponding bit is 1:

```

1 0b1100 | 0b1010 // 0b1110 (14)
2 flags | FLAG_ENABLED // Set a flag bit

```

4.5.3 Bitwise XOR

Sets each bit to 1 if exactly one corresponding bit is 1:

```

1 0b1100 ^ 0b1010 // 0b0110 (6)
2 value ^ mask     // Toggle bits
3 a ^ b ^ b        // Returns a (XOR is self-inverse)

```

4.5.4 Bitwise NOT

Inverts all bits (one's complement):

```

1 ~0b00001111 // 0b11110000 (for 8-bit)
2 ~0u8         // 255 (all bits set)
3 ~0           // -1 (for signed int)

```

Warning

The result of `~` depends on the bit width of the operand type. For `u8`, `~0` is 255; for `int`, `~0` is `-1`.

4.5.5 Left Shift

Shifts bits to the left, filling vacated positions with zeros:

```
1 1 << 4           // 16 (0b10000)
2 value << n       // Equivalent to value * 2^n
3 0b0001 << 3     // 0b1000 (8)
```

Warning

Shifting by a negative amount or by more than the bit width of the type is undefined behavior.

4.5.6 Right Shift

The behavior of `>>` depends on the signedness of the operand:

- **Unsigned types:** Logical shift (fills with zeros)
- **Signed types:** Arithmetic shift (fills with sign bit)

```
1 // Unsigned: logical shift (fills with 0)
2 x: u8 = 0b11110000
3 x >> 2           // 0b00111100 (60)
4
5 // Signed: arithmetic shift (fills with sign bit)
6 y: i8 = -16      // 0b11110000 in two's complement
7 y >> 2           // -4 (0b11111100, sign preserved)
```

4.5.7 Logical Right Shift

The `>>>` operator always performs a logical right shift, filling with zeros regardless of signedness:

```
1 x: i8 = -16      // 0b11110000
2 x >>> 2          // 60 (0b00111100, zeros filled)
3
4 // Useful for treating signed integers as bit patterns
5 hash: int = -1
6 hash >>> 1       // Large positive number
```

4.5.8 Common Bit Manipulation Patterns

```
1 // Check if bit n is set
2 is_set = (value >> n) & 1 == 1
3
```

```

4 // Set bit n
5 value = value | (1 << n)
6
7 // Clear bit n
8 value = value & ~(1 << n)
9
10 // Toggle bit n
11 value = value ^ (1 << n)
12
13 // Check if power of two (positive integers)
14 is_power_of_two = value > 0 and (value & (value - 1)) == 0
15
16 // Extract n bits starting at position p
17 bits = (value >> p) & ((1 << n) - 1)
18
19 // Count trailing zeros (manual approach)
20 fn count_trailing_zeros(x: u32) -> int {
21     if x == 0 { return 32 }
22     count = 0
23     while (x & 1) == 0 {
24         count += 1
25         x = x >> 1
26     }
27     return count
28 }

```

4.5.9 Compound Bitwise Assignment

```

1 flags &= MASK           // flags = flags & MASK
2 flags |= FLAG           // flags = flags | FLAG
3 flags ^= TOGGLE         // flags = flags ^ TOGGLE
4 value <<= 2              // value = value << 2
5 value >>= 1              // value = value >> 1
6 value >>>= 1             // value = value >>> 1

```

4.6 String Expressions

4.6.1 Concatenation

```

1 "Hello, " + "World!"    // "Hello, World!"

```

4.6.2 String Interpolation

```

1 name = "Alice"
2 age = 30
3 "Hello, ${name}! You are ${age} years old."

```

Expressions inside `${...}` are evaluated and converted to strings.

4.7 Array and Map Expressions

4.7.1 Array Literals

```
1 [] // Empty array (type from context)
2 [1, 2, 3] // []int
3 ["a", "b", "c"] // []string
```

4.7.2 Map Literals

```
1 [:] // Empty map (type from context)
2 ["a": 1, "b": 2] // [string:int]
3 [1: "one", 2: "two"] // [int:string]
```

4.7.3 Index Access

```
1 array[0] // First element
2 array[array.len - 1] // Last element
3 map["key"] // Map lookup (returns T?)
```

Note

Array indexing with an out-of-bounds index causes a runtime panic. Map indexing returns an option type.

4.8 Field Access

```
1 user.name // Struct field
2 point.x // Struct field
3 tuple.0 // Tuple element
```

4.9 Function Calls

```
1 func() // No arguments
2 func(a, b, c) // With arguments
3 obj.method() // Method call
4 obj.method(a, b) // Method with arguments
```

4.9.1 Chained Calls

```
1 text.trim().to_upper().split(" ")
```

4.10 Pipe Expressions

The pipe operator `|>` passes the left operand as the first argument to the right function:

```
1 // These are equivalent:
2 result = f(g(h(x)))
3 result = x |> h |> g |> f
```

4.10.1 Pipe with Arguments

When the right side has arguments, the left value becomes the first argument:

```
1 // These are equivalent:
2 result = string.split(text, ",")
3 result = text |> string.split(",")
```

4.10.2 Complex Pipelines

```
1 import "io"
2 import "string"
3 import "array"
4
5 fn main() {
6     data = "  apple, banana, cherry  "
7
8     result = data
9         |> string.trim
10        |> string.split(", ")
11        |> array.map(string.to_upper)
12        |> array.filter(fn(s) { string.len(s) > 5 })
13        |> string.join(" - ")
14
15     io.println(result) // "BANANA - CHERRY"
16 }
```

4.11 Range Expressions

```
1 0..10 // 0 to 9 (exclusive end)
2 0..=10 // 0 to 10 (inclusive end)
```

Ranges are used primarily in **for** loops:

```
1 for i in 0..5 {
2     io.println(i) // 0, 1, 2, 3, 4
3 }
4
5 for i in 0..=5 {
6     io.println(i) // 0, 1, 2, 3, 4, 5
7 }
```

4.12 Conditional Expressions

4.12.1 If Expression

if can be used as an expression:

```
1 max = if a > b { a } else { b }
2
3 status = if score >= 90 {
4     "A"
5 } else if score >= 80 {
6     "B"
7 } else {
8     "C"
9 }
```

Warning

When used as an expression, all branches must have the same type and **else** is required.

4.12.2 Match Expression

match expressions provide pattern matching:

```
1 result = match value {
2     0 => "zero"
3     1 => "one"
4     n if n < 0 => "negative"
5     _ => "other"
6 }
```

See Chapter 5 for complete pattern matching semantics.

4.13 Block Expressions

Blocks are expressions that evaluate to their last expression:

```
1 result = {
2     x = compute_x()
3     y = compute_y()
4     x + y // This is the value of the block
5 }
```

4.14 Struct Instantiation

```
1 user = User{
2     id: 1,
3     name: "Alice",
4     email: "alice@example.com"
5 }
6
```



```
7 // With shorthand (field name matches variable)
8 name = "Bob"
9 email = "bob@example.com"
10 user = User{id: 2, name, email}
```

4.15 Closure Expressions

Anonymous functions (closures):

```
1 // Full syntax
2 add = fn(a: int, b: int) -> int {
3     return a + b
4 }
5
6 // Short syntax for single expression
7 double = fn(x: int) -> int { x * 2 }
8
9 // Type inference for closures
10 numbers = [1, 2, 3]
11 doubled = array.map(numbers, fn(n) { n * 2 })
```

4.16 Option Expressions

4.16.1 Nil Coalescing

The `or` operator provides a default for optional values:

```
1 name = maybe_name or "Unknown"
2 value = get_value() or default_value
```

4.16.2 Optional Chaining

```
1 // Returns nil if any part is nil
2 street = user?.address?.street
```

4.17 Type Assertions

```
1 // Assert that value is a specific type
2 x = value as int
3
4 // Safe assertion (returns option)
5 x = value as? int
```

4.18 Operator Precedence

From highest to lowest:

1. Postfix: () [] . ?.
2. Unary: ! - ~ not
3. Multiplicative: * / %
4. Additive: + -
5. Shift: << >> >>>
6. Bitwise AND: &
7. Bitwise XOR: ^
8. Bitwise OR: |
9. Range:=
10. Pipe: |>
11. Comparison: < > <= >=
12. Equality: == !=
13. Logical AND: and &&
14. Logical OR: or ||
15. Assignment: = += -= *= /= &= |= ^= <= >= >>=

4.19 Expression Grammar

```

1 expression = assignment ;
2
3 assignment = or_expr [ ("=" | "+=" | "-=" | "*=" | "/"=
4               | "&=" | "|=" | "^=" | "<=" | ">=" | ">>=")
5               assignment ] ;
6
7 or_expr = and_expr { ("or" | "||") and_expr } ;
8
9 and_expr = equality { ("and" | "&&") equality } ;
10
11 equality = comparison { ("==" | "!=") comparison } ;
12
13 comparison = pipe { ("<" | ">" | "<=" | ">=") pipe } ;
14
15 pipe = range { "|>" range } ;
16
17 range = bitwise_or [ (".." | "..=") bitwise_or ] ;
18
19 bitwise_or = bitwise_xor { "|" bitwise_xor } ;
20
21 bitwise_xor = bitwise_and { "^" bitwise_and } ;
22
23 bitwise_and = shift { "&" shift } ;
24
25 shift = additive { ("<<" | ">>" | ">>>") additive } ;

```

```
26 additive = multiplicative { ("+" | "-") multiplicative } ;
27
28 multiplicative = unary { ("*" | "/" | "%") unary } ;
29
30 unary = ("!" | "-" | "~" | "not") unary | postfix ;
31
32 postfix = primary { call | index | field } ;
33
34 call = "(" [ arguments ] ")" ;
35
36 index = "[" expression "]" ;
37
38 field = "." identifier ;
39
40 primary = identifier
41         | literal
42         | "(" expression ")"
43         | array_literal
44         | map_literal
45         | struct_literal
46         | closure
47         | if_expr
48         | match_expr
49         | block ;
50
51 arguments = expression { "," expression } ;
```


Chapter 5

Statements

Statements perform actions. Unlike expressions, statements do not produce values.

5.1 Variable Declarations

5.1.1 With Inference

```
1 x = 42                // Type inferred as int
2 name = "Haira"        // Type inferred as string
3 items = [1, 2, 3]     // Type inferred as []int
```

5.1.2 With Explicit Type

```
1 x: int = 42
2 name: string = "Haira"
3 count: int = 100
```

5.1.3 Multiple Declarations

```
1 a, b = 1, 2
2 x, y, z = compute()    // Destructuring tuple
```

5.1.4 Uninitialized Variables

```
1 x: int                // Must be assigned before use
```

Warning

Using an uninitialized variable is a compile-time error.

5.2 Assignment Statements

```
1 x = 10           // Simple assignment
2 x += 5           // Add and assign (x = x + 5)
3 x -= 3           // Subtract and assign
4 x *= 2           // Multiply and assign
5 x /= 4           // Divide and assign
```

5.2.1 Multiple Assignment

```
1 a, b = b, a      // Swap values
2 x, y = get_coordinates()
3 quotient, remainder = divide(17, 5)
```

5.2.2 Field Assignment

```
1 user.name = "Bob"
2 point.x = 10.0
```

5.2.3 Index Assignment

```
1 array[0] = 100
2 map["key"] = "value"
```

5.3 Expression Statements

Any expression can be used as a statement:

```
1 io.println("Hello") // Function call
2 process(data)        // Function call, ignoring return
3 x + y                // Valid but pointless
```

Note

The compiler may warn about expression statements that have no side effects.

5.4 If Statements

```
1 if condition {
2     // executed if condition is true
3 }
4
5 if condition {
6     // true branch
7 } else {
8     // false branch
9 }
```

```
10
11 if condition1 {
12     // first
13 } else if condition2 {
14     // second
15 } else {
16     // default
17 }
```

5.4.1 Condition Binding

```
1 // Bind and check in one statement
2 if value = get_optional_value(); value != nil {
3     io.println(value)
4 }
```

5.5 For Statements

5.5.1 Range-Based For

```
1 // Exclusive range
2 for i in 0..10 {
3     io.println(i) // 0 through 9
4 }
5
6 // Inclusive range
7 for i in 0..=10 {
8     io.println(i) // 0 through 10
9 }
```

5.5.2 Collection Iteration

```
1 // Array iteration
2 for item in items {
3     io.println(item)
4 }
5
6 // With index
7 for i, item in items {
8     io.println("${i}: ${item}")
9 }
10
11 // Map iteration
12 for key, value in scores {
13     io.println("${key}: ${value}")
14 }
```

5.5.3 String Iteration

```
1 for char in "hello" {  
2     io.println(char) // Each UTF-8 character  
3 }  
4  
5 for i, char in "hello" {  
6     io.println("${i}: ${char}")  
7 }
```

5.6 While Statements

```
1 while condition {  
2     // body  
3 }  
4  
5 // Infinite loop  
6 while true {  
7     if should_stop() {  
8         break  
9     }  
10 }
```

5.7 Match Statements

5.7.1 Basic Matching

```
1 match value {  
2     1 => io.println("one")  
3     2 => io.println("two")  
4     3 => io.println("three")  
5     _ => io.println("other")  
6 }
```

5.7.2 Multiple Patterns

```
1 match value {  
2     1 | 2 | 3 => io.println("small")  
3     4 | 5 | 6 => io.println("medium")  
4     _ => io.println("large")  
5 }
```

5.7.3 Range Patterns

```
1 match score {  
2     90..100 => io.println("A")  
3     80..90  => io.println("B")  
4     70..80  => io.println("C")  
5     60..70  => io.println("D")  
6 }
```



```
6   _ => io.println("F")
7 }
```

5.7.4 Guard Clauses

```
1 match value {
2   n if n < 0 => io.println("negative")
3   n if n == 0 => io.println("zero")
4   n if n > 0 => io.println("positive")
5 }
```

5.7.5 Destructuring Patterns

```
1 // Struct destructuring
2 match user {
3   User{name: "admin", active: true} => io.println("Active
4     admin")
5   User{name: n, active: false} => io.println("Inactive: " + n)
6   _ => io.println("Regular user")
7 }
8 // Tuple destructuring
9 match point {
10  (0, 0) => io.println("origin")
11  (x, 0) => io.println("on x-axis at " + to_string(x))
12  (0, y) => io.println("on y-axis at " + to_string(y))
13  (x, y) => io.println("at (" + to_string(x) + ", " +
14    to_string(y) + ")")
15 }
16 // Enum destructuring
17 match result {
18   Result.Ok(value) => io.println("Got: " + to_string(value))
19   Result.Err(err) => io.println("Error: " + err.message)
20 }
```

5.7.6 Match Exhaustiveness

```
1 enum Direction { North, South, East, West }
2
3 match direction {
4   Direction.North => go_north()
5   Direction.South => go_south()
6   Direction.East  => go_east()
7   Direction.West  => go_west()
8   // No _ needed: all cases covered
9 }
```

Note

The compiler verifies that match expressions are exhaustive. If not all cases are covered and there is no wildcard (`_`), the compiler produces an error.

5.8 Block Statements

```
1 {  
2     x = 10  
3     y = 20  
4     io.println(x + y)  
5 }
```

Blocks create a new scope. Variables declared inside are not visible outside.

5.9 Return Statements

```
1 fn add(a: int, b: int) -> int {  
2     return a + b  
3 }  
4  
5 fn early_return(x: int) -> int {  
6     if x < 0 {  
7         return 0  
8     }  
9     return x * 2  
10 }  
11  
12 // Multiple returns  
13 fn divide(a: int, b: int) -> (int, Error?) {  
14     if b == 0 {  
15         return 0, Error{message: "division by zero"}  
16     }  
17     return a / b, nil  
18 }
```

5.9.1 Implicit Return

The last expression in a function is implicitly returned:

```
1 fn add(a: int, b: int) -> int {  
2     a + b // No return keyword needed  
3 }
```

5.10 Break Statements

```
1 for i in 0..100 {  
2     if i > 10 {  
3         break
```

```
4     }
5     io.println(i)
6 }
7
8 while true {
9     if done() {
10         break
11     }
12 }
```

5.10.1 Labeled Break

```
1 outer: for i in 0..10 {
2     for j in 0..10 {
3         if i * j > 25 {
4             break outer
5         }
6     }
7 }
```

5.11 Continue Statements

```
1 for i in 0..10 {
2     if i % 2 == 0 {
3         continue // Skip even numbers
4     }
5     io.println(i)
6 }
```

5.11.1 Labeled Continue

```
1 outer: for i in 0..5 {
2     for j in 0..5 {
3         if j == 2 {
4             continue outer
5         }
6         io.println("${i}, ${j}")
7     }
8 }
```

5.12 Import Statements

Import statements must appear at the top of the file:

```
1 import "io"
2 import "json"
3 import db from "haira/postgres"
4 import { User, Post } from "models"
```

See Chapter 9 for complete import semantics.

5.13 Defer Statements

defer schedules a statement to run when the current scope exits:

```

1 fn read_file(path: string) -> (string, Error?) {
2     file, err = fs.open(path)
3     if err != nil {
4         return "", err
5     }
6     defer fs.close(file)
7
8     content, err = fs.read_all(file)
9     if err != nil {
10        return "", err // file is closed here
11    }
12
13    return content, nil // file is closed here too
14 }
```

5.13.1 Multiple Defers

Multiple **defer** statements execute in reverse order (LIFO):

```

1 fn example() {
2     defer io.println("first") // Runs third
3     defer io.println("second") // Runs second
4     defer io.println("third") // Runs first
5 }
6 // Output: third, second, first
```

5.14 Statement Grammar

```

1 statement = variable_decl
2             | assignment
3             | expression_stmt
4             | if_stmt
5             | for_stmt
6             | while_stmt
7             | match_stmt
8             | return_stmt
9             | break_stmt
10            | continue_stmt
11            | defer_stmt
12            | block ;
13
14 variable_decl = identifier [ ":" type ] "=" expression
15                | identifier_list "=" expression_list ;
16
17 assignment = lvalue assign_op expression ;
18
19 assign_op = "=" | "+=" | "-=" | "*=" | "/=" ;
```

```
20
21 lvalue = identifier | field_access | index_access ;
22
23 expression_stmt = expression ;
24
25 if_stmt = "if" [ simple_stmt ";" ] expression block
26           [ "else" ( if_stmt | block ) ] ;
27
28 for_stmt = "for" [ identifier "," ] identifier "in" expression
29           block ;
30
31 while_stmt = "while" expression block ;
32
33 match_stmt = "match" expression "{ " { match_arm } "}" ;
34
35 match_arm = pattern [ "if" expression ] "=>" ( expression |
36           block ) ;
37
38 return_stmt = "return" [ expression_list ] ;
39
40 break_stmt = "break" [ identifier ] ;
41
42 continue_stmt = "continue" [ identifier ] ;
43
44 defer_stmt = "defer" statement ;
45
46 block = "{ " { statement } "}" ;
```


Chapter 6

Functions

Functions are the primary unit of code organization in HAIRA. They are first-class values that can be passed as arguments, returned from other functions, and stored in variables.

6.1 Function Declarations

```
1 fn function_name(param1: Type1, param2: Type2) -> ReturnType {  
2     // body  
3 }
```

6.1.1 Basic Examples

```
1 fn greet() {  
2     io.println("Hello!")  
3 }  
4  
5 fn add(a: int, b: int) -> int {  
6     return a + b  
7 }  
8  
9 fn is_positive(n: int) -> bool {  
10    return n > 0  
11 }
```

6.1.2 No Return Type

Functions without a return type implicitly return ():

```
1 fn log_message(msg: string) {  
2     io.println("[LOG] " + msg)  
3 }
```

6.2 Parameters

6.2.1 Required Parameters

All parameters must have explicit type annotations:

```
1 fn calculate(x: int, y: int, z: float) -> float {  
2     return (x + y) as float * z  
3 }
```

6.2.2 Default Parameters

Parameters can have default values:

```
1 fn greet(name: string, greeting: string = "Hello") {  
2     io.println(greeting + ", " + name + "!")  
3 }  
4  
5 greet("Alice")           // "Hello, Alice!"  
6 greet("Bob", "Hi")       // "Hi, Bob!"
```

Note

Parameters with defaults must come after parameters without defaults.

6.2.3 Named Arguments

Functions can be called with named arguments:

```
1 fn create_user(name: string, email: string, age: int = 0) ->  
   User {  
2     return User{name: name, email: email, age: age}  
3 }  
4  
5 // Positional  
6 create_user("Alice", "alice@example.com", 30)  
7  
8 // Named  
9 create_user(name: "Bob", email: "bob@example.com")  
10  
11 // Mixed (positional first, then named)  
12 create_user("Charlie", email: "charlie@example.com", age: 25)
```

6.2.4 Variadic Parameters

Functions can accept variable numbers of arguments:

```
1 fn sum(numbers: ...int) -> int {  
2     total = 0  
3     for n in numbers {  
4         total += n  
5     }  
6     return total
```



```
7 }  
8  
9 sum(1, 2, 3)           // 6  
10 sum(1, 2, 3, 4, 5)    // 15
```

6.3 Return Values

6.3.1 Single Return

```
1 fn double(n: int) -> int {  
2     return n * 2  
3 }
```

6.3.2 Multiple Returns

Functions can return multiple values using tuples:

```
1 fn divide(a: int, b: int) -> (int, int) {  
2     return a / b, a % b  
3 }  
4  
5 quotient, remainder = divide(17, 5) // 3, 2
```

6.3.3 Implicit Return

The last expression in a function body is implicitly returned:

```
1 fn add(a: int, b: int) -> int {  
2     a + b // Implicit return  
3 }  
4  
5 fn max(a: int, b: int) -> int {  
6     if a > b { a } else { b } // Implicit return  
7 }
```

6.3.4 Early Return

Use **return** for early exit:

```
1 fn factorial(n: int) -> int {  
2     if n <= 1 {  
3         return 1  
4     }  
5     return n * factorial(n - 1)  
6 }
```

6.4 Function Types

Functions have types that can be used in type annotations:

```

1 // Function type: fn(int, int) -> int
2 type BinaryOp = fn(int, int) -> int
3
4 fn apply(op: BinaryOp, a: int, b: int) -> int {
5     return op(a, b)
6 }
7
8 fn add(a: int, b: int) -> int { a + b }
9 fn mul(a: int, b: int) -> int { a * b }
10
11 apply(add, 2, 3) // 5
12 apply(mul, 2, 3) // 6

```

6.5 Closures

Closures are anonymous functions that can capture values from their environment:

```

1 fn make_counter() -> fn() -> int {
2     count = 0
3     return fn() -> int {
4         count += 1
5         return count
6     }
7 }
8
9 counter = make_counter()
10 counter() // 1
11 counter() // 2
12 counter() // 3

```

6.5.1 Closure Syntax

```

1 // Full syntax
2 fn(a: int, b: int) -> int {
3     return a + b
4 }
5
6 // Short syntax (single expression)
7 fn(a: int, b: int) -> int { a + b }
8
9 // With type inference (in context)
10 numbers = [1, 2, 3]
11 doubled = array.map(numbers, fn(n) { n * 2 })

```

6.5.2 Capture Semantics

Closures capture variables by reference:

```

1 fn example() {
2     values = []
3 }

```

```

4     for i in 0..3 {
5         // Captures 'i' by reference
6         values = array.push(values, fn() { i })
7     }
8
9     // All closures see final value of i
10    values[0]() // 3
11    values[1]() // 3
12    values[2]() // 3
13 }

```

To capture by value, use explicit binding:

```

1 fn example() {
2     values = []
3
4     for i in 0..3 {
5         captured = i // Capture current value
6         values = array.push(values, fn() { captured })
7     }
8
9     values[0]() // 0
10    values[1]() // 1
11    values[2]() // 2
12 }

```

6.6 Methods

Methods are functions associated with a type:

```

1 struct Rectangle {
2     width: float
3     height: float
4 }
5
6 Rectangle.area() -> float {
7     return self.width * self.height
8 }
9
10 Rectangle.perimeter() -> float {
11     return 2.0 * (self.width + self.height)
12 }
13
14 Rectangle.scale(factor: float) -> Rectangle {
15     return Rectangle{
16         width = self.width * factor,
17         height = self.height * factor
18     }
19 }
20
21 // Usage
22 rect = Rectangle{width = 10.0, height = 5.0}
23 rect.area() // 50.0
24 rect.perimeter() // 30.0
25 rect.scale(2.0) // Rectangle{width: 20.0, height: 10.0}

```

6.6.1 Method Semantics

Methods use `self` to access the receiver. All methods can mutate the receiver (pointer semantics). Method visibility is inherited from the type: if the type is **pub**, all its methods are accessible to importers. Methods cannot be defined on imported types.

```
1 struct Point {
2     x: float
3     y: float
4 }
5
6 Point.distance_to(other: Point) -> float {
7     dx = self.x - other.x
8     dy = self.y - other.y
9     return math.sqrt(dx*dx + dy*dy)
10 }
11
12 Point.translate(dx: float, dy: float) {
13     self.x += dx
14     self.y += dy
15 }
```

6.7 Generic Functions

Functions can be parameterized over types:

```
1 fn identity<T>(value: T) -> T {
2     return value
3 }
4
5 fn swap<T, U>(pair: (T, U)) -> (U, T) {
6     return (pair.1, pair.0)
7 }
8
9 fn first<T>(items: []T) -> T? {
10     if array.len(items) == 0 {
11         return nil
12     }
13     return items[0]
14 }
```

6.7.1 Type Constraints

Generic types can be constrained:

```
1 constraint Comparable {
2     fn compare(other: Self) -> int
3 }
4
5 fn max<T: Comparable>(a: T, b: T) -> T {
6     if a.compare(b) > 0 {
7         return a
8     }
9     return b
10 }
```

```

11
12 fn sort<T: Comparable>(items: []T) -> []T {
13     // Sorting implementation
14 }

```

6.8 Higher-Order Functions

Functions that take or return other functions:

```

1 // Function as parameter
2 fn apply_twice(f: fn(int) -> int, x: int) -> int {
3     return f(f(x))
4 }
5
6 fn double(n: int) -> int { n * 2 }
7
8 apply_twice(double, 5) // 20
9
10 // Function as return value
11 fn make_multiplier(factor: int) -> fn(int) -> int {
12     return fn(n: int) -> int {
13         return n * factor
14     }
15 }
16
17 triple = make_multiplier(3)
18 triple(7) // 21

```

6.9 Recursion

Functions can call themselves:

```

1 fn factorial(n: int) -> int {
2     if n <= 1 {
3         return 1
4     }
5     return n * factorial(n - 1)
6 }
7
8 fn fibonacci(n: int) -> int {
9     match n {
10         0 => 0
11         1 => 1
12         _ => fibonacci(n - 1) + fibonacci(n - 2)
13     }
14 }

```

6.9.1 Tail Call Optimization

HAIRA optimizes tail-recursive functions:

```

1 fn factorial_tail(n: int, acc: int = 1) -> int {

```

```

2     if n <= 1 {
3         return acc
4     }
5     return factorial_tail(n - 1, n * acc) // Tail call
6 }

```

6.10 Function Overloading

HAIRA does not support function overloading. Each function name must be unique within its scope. Use different names or generic functions instead:

```

1 // Instead of overloading, use descriptive names
2 fn add_ints(a: int, b: int) -> int { a + b }
3 fn add_floats(a: float, b: float) -> float { a + b }
4
5 // Or use generics
6 fn add<T: Addable>(a: T, b: T) -> T { a + b }

```

6.11 The main Function

Every executable HAIRA program must have a `main` function:

```

1 fn main() {
2     io.println("Hello, World!")
3 }
4
5 // With exit code
6 fn main() -> int {
7     io.println("Hello, World!")
8     return 0
9 }

```

6.12 Function Grammar

```

1 function_decl = "fn" [ receiver ] identifier [ type_params ]
2               "(" [ params ] ")" [ "->" type ] block ;
3
4 receiver = "(" identifier ":" [ "*" ] type ")" ;
5
6 type_params = "<" type_param { "," type_param } ">" ;
7
8 type_param = identifier [ ":" constraint_list ] ;
9
10 constraint_list = identifier { "+" identifier } ;
11
12 params = param { "," param } ;
13
14 param = identifier ":" [ "..." ] type [ "=" expression ] ;
15
16 closure = "fn" "(" [ closure_params ] ")" [ "->" type ]
17          ( block | expression ) ;

```

```
18  
19 closure_params = closure_param { "," closure_param } ;  
20  
21 closure_param = identifier [ ":" type ] ;
```


Chapter 7

Error Handling

HAIRA uses explicit error handling inspired by Go. Errors are values, not exceptions. Functions that can fail return both a result and an optional error.

For concise code, HAIRA also provides a `?` propagation operator and `try/catch` blocks built on Go's `panic/recover` mechanism.

7.1 Philosophy

- **Errors are values** – Not special control flow
- **Explicit handling** – Callers must handle or propagate errors
- **Two modes** – Verbose-but-clear tuple checking, or concise `?` propagation
- **Composable** – Errors can be wrapped and chained

7.2 The Error Type

The built-in Error type represents errors:

```
1 struct Error {  
2     message: string  
3     code: int?  
4     source: Error?  
5 }
```

7.2.1 Creating Errors

```
1 // Simple error  
2 err = Error{message: "something went wrong"}  
3  
4 // With error code  
5 err = Error{message: "permission denied", code: 403}  
6  
7 // Wrapped error (error chaining)  
8 err = Error{  
9     message: "failed to read config",  
10    source: original_error  
11 }
```

7.3 Error Return Pattern

Functions that can fail return a tuple of (result, Error?):

```

1 fn divide(a: int, b: int) -> (int, Error?) {
2     if b == 0 {
3         return 0, Error{message: "division by zero"}
4     }
5     return a / b, nil
6 }
7
8 fn read_file(path: string) -> (string, Error?) {
9     // Implementation
10 }
11
12 fn parse_int(s: string) -> (int, Error?) {
13     // Implementation
14 }

```

7.4 Handling Errors

7.4.1 Basic Error Checking

The standard pattern: call a function, check the error, handle or return:

```

1 result, err = divide(10, 0)
2 if err != nil {
3     io.println("Error: " + err.message)
4     return
5 }
6 io.println("Result: " + to_string(result))

```

7.4.2 Propagating Errors

```

1 fn process_file(path: string) -> (Data, Error?) {
2     content, err = read_file(path)
3     if err != nil {
4         return Data{}, err // Propagate error
5     }
6
7     data, err = parse_data(content)
8     if err != nil {
9         return Data{}, err // Propagate error
10    }
11
12    return data, nil
13 }

```

7.4.3 Wrapping Errors

Add context when propagating:

```
1 fn load_config(path: string) -> (Config, Error?) {
2     content, err = read_file(path)
3     if err != nil {
4         return Config{}, Error{
5             message: "failed to load config from " + path,
6             source: err
7         }
8     }
9
10    config, err = parse_config(content)
11    if err != nil {
12        return Config{}, Error{
13            message: "invalid config format",
14            source: err
15        }
16    }
17
18    return config, nil
19 }
```

7.5 Error Propagation Operator (?)

The `?` operator is syntactic sugar for “panic on error.” It extracts the success value from a call that returns `(T, Error?)`. If the error is non-nil, it panics.

```
1 // These two are equivalent:
2
3 // Verbose
4 content, err = read_file(path)
5 if err != nil {
6     panic(err)
7 }
8
9 // Concise
10 content = read_file(path)?
```

The `?` operator makes code much shorter when chaining multiple fallible calls:

```
1 fn setup() {
2     config = load_config("app.toml")?
3     db = connect_database(config.db_url)?
4     cache = init_cache(config.cache_url)?
5     // If any call fails, execution stops with a panic
6 }
```

Warning

The `?` operator panics on error. If used outside a `try` block, an unhandled error will crash the program. Use `?` inside `try/catch` blocks, or in initialization code where failure is fatal.

7.6 Try/Catch

The **try/catch** block recovers from panics caused by the `?` operator (or explicit `panic()` calls). It compiles to Go's `defer/recover` pattern.

```

1  try {
2      config = load_config("app.toml")?
3      db = connect_database(config.db_url)?
4      start_server(db)
5  } catch err {
6      io.println("Startup failed: " + err)
7      os.exit(1)
8  }

```

The catch variable receives the error as a string. Variables assigned inside the **try** block are hoisted and remain accessible after the block.

7.6.1 Try/Catch in Functions

```

1  fn process_batch(items: [string]) -> int {
2      count = 0
3      for item in items {
4          try {
5              result = parse_item(item)?
6              save_result(result)?
7              count = count + 1
8          } catch err {
9              io.eprintln("Skipping item: " + err)
10         }
11     }
12     return count
13 }

```

7.6.2 When to Use Each Pattern

Pattern	Syntax	Use When
Tuple check	<code>val, err = fn()</code>	You need to inspect or wrap the error
<code>?</code> + try/catch	<code>val = fn()?</code>	Multiple calls, same error handling
<code>?</code> without try	<code>val = fn()?</code>	Failure is fatal (init, main)

Table 7.1: Error handling patterns

7.7 Orelse Operator

The **orelse** operator provides a default value when a fallible call fails:

```

1  // If parse fails, use 0 as default
2  count = parse_int(input) orelse 0
3
4  // If config load fails, use defaults
5  config = load_config("app.toml") orelse Config{port: 8080}

```

This is useful for optional operations where a sensible default exists.

7.8 Defer and Errdefer

7.8.1 Defer

Use **defer** to ensure cleanup happens when the enclosing scope exits, regardless of how it exits:

```

1 fn with_file(path: string) -> (string, Error?) {
2     file, err = fs.open(path)
3     if err != nil {
4         return "", err
5     }
6     defer fs.close(file) // Always closes, even on error
7
8     content, err = fs.read_all(file)
9     if err != nil {
10        return "", err // file is closed by defer
11    }
12
13    return content, nil // file is closed by defer
14 }
```

7.8.2 Errdefer

The **errdefer** statement runs cleanup only when the enclosing scope exits via a panic (from `?` or explicit **panic**). If the scope exits normally, the **errdefer** does not run.

```

1 fn setup_resources() {
2     db = connect_database()?
3     errdefer db.close() // Only closes if a later ? panics
4
5     cache = connect_cache()?
6     errdefer cache.close() // Only closes if a later ? panics
7
8     // If migrate fails, both db and cache are closed
9     migrate(db)?
10    start_server(db, cache)
11    // If we get here, errdefers do NOT run -- resources stay
12    // open
13 }
```

7.9 Error Checking Patterns

7.9.1 Guard Pattern

Check errors early and return:

```

1 fn process(input: string) -> (Output, Error?) {
2     // Guard: validate input
3     if string.len(input) == 0 {
4         return Output{}, Error{message: "input cannot be empty"}
```

```

5     }
6
7     // Guard: check prerequisites
8     ready, err = check_ready()
9     if err != nil {
10         return Output{}, err
11     }
12     if not ready {
13         return Output{}, Error{message: "system not ready"}
14     }
15
16     // Main logic
17     result = do_processing(input)
18     return result, nil
19 }

```

7.10 Error Utilities

7.10.1 Checking Error Types

```

1 fn is_not_found(err: Error?) -> bool {
2     if err == nil {
3         return false
4     }
5     return err.code == 404 or string.contains(err.message, "not
6         found")
7 }

```

7.10.2 Error Chain Traversal

```

1 fn root_cause(err: Error?) -> Error? {
2     if err == nil {
3         return nil
4     }
5     current = err
6     while current.source != nil {
7         current = current.source
8     }
9     return current
10 }

```

7.11 Option Types and Errors

For operations that may not find a value (not an error condition), use option types:

```

1 // Use option when "not found" is normal
2 fn find_user(id: int) -> User? {
3     // Returns nil if not found
4 }
5
6 // Use error when "not found" is exceptional

```

```

7 fn get_user(id: int) -> (User, Error?) {
8     // Returns error if not found
9 }

```

7.12 Panic

For truly unrecoverable situations, use `panic`:

```

1 fn assert(condition: bool, message: string) {
2     if not condition {
3         panic(message)
4     }
5 }

```

Warning

`panic` crashes the program unless caught by `try/catch`. Use it only for programming errors and invariant violations. Prefer returning errors for all expected failure conditions.

7.13 Best Practices

- **Return errors, don't log and continue** – Propagate failures to the caller
- **Add context when wrapping** – Use `Error{message: "context", source: err}`
- **Handle at the right level** – Low-level returns specific errors, high-level decides what to do
- **Use ? for chain-or-fail sequences** – Cleaner than repeated `if err != nil`
- **Use `orelse` for optional operations** – When a default is acceptable
- **Use `errdefer` for resource cleanup** – Pairs allocation with deallocation

7.14 Error Handling Grammar

```

1 error_return      = "(" type "," "Error?" ")" ;
2
3 error_check       = "if" identifier "!=" "nil" block ;
4
5 error_create      = "Error" "{"
6                     "message" ":" expression
7                     [ "," "code" ":" expression ]
8                     [ "," "source" ":" expression ]
9                     "}" ;
10
11 propagate_expr    = expression "?" ;
12
13 orelse_expr       = expression "orelse" expression ;
14

```

```
15 try_stmt      = "try" block "catch" identifier block ;
16
17 defer_stmt     = "defer" expression ;
18
19 errdefer_stmt  = "errdefer" expression ;
```


Part II

Concurrency and Modules

Chapter 8

Concurrency

HAIRA provides Go-style concurrency primitives: lightweight spawned tasks and channels for communication. The philosophy is “share memory by communicating” rather than “communicate by sharing memory.”

8.1 Concurrency Model

- **Spawned tasks** – Lightweight concurrent execution units
- **Channels** – Typed conduits for communication
- **Select** – Multiplexing channel operations
- **No shared mutable state** – Communication via channels

8.2 Spawn

The **spawn** keyword starts a new concurrent task:

```
1 import "io"
2 import "time"
3
4 fn main() {
5     spawn {
6         time.sleep(1000)
7         io.println("Hello from spawned task!")
8     }
9
10    io.println("Main continues immediately")
11    time.sleep(2000) // Wait for spawned task
12 }
```

8.2.1 Spawning Functions

```
1 fn worker(id: int) {
2     io.println("Worker " + to_string(id) + " started")
3     time.sleep(1000)
4     io.println("Worker " + to_string(id) + " done")
5 }
6
```

```

7 fn main() {
8     for i in 0..5 {
9         spawn worker(i)
10    }
11    time.sleep(3000) // Wait for all workers
12 }

```

8.3 Channels

Channels are typed conduits for passing values between tasks:

```

1 // Create a channel
2 ch = chan<int>()           // Unbuffered channel
3 ch = chan<int>(10)        // Buffered channel (capacity 10)
4 ch = chan<string>()       // String channel

```

8.3.1 Sending and Receiving

```

1 ch = chan<int>()
2
3 // Send (blocks until received for unbuffered)
4 ch <- 42
5
6 // Receive (blocks until value available)
7 value = <-ch

```

8.3.2 Basic Channel Example

```

1 import "io"
2
3 fn main() {
4     ch = chan<string>()
5
6     spawn {
7         ch <- "Hello from spawned task!"
8     }
9
10    message = <-ch
11    io.println(message)
12 }

```

8.3.3 Buffered Channels

```

1 ch = chan<int>(3) // Buffer size 3
2
3 // These don't block (buffer has space)
4 ch <- 1
5 ch <- 2
6 ch <- 3

```

```
7
8 // This would block (buffer full)
9 // ch <- 4
10
11 // Receive
12 io.println(<-ch) // 1
13 io.println(<-ch) // 2
14 io.println(<-ch) // 3
```

8.3.4 Closing Channels

```
1 ch = chan<int>(10)
2
3 spawn {
4     for i in 0..10 {
5         ch <- i
6     }
7     close(ch) // Signal no more values
8 }
9
10 // Iterate until channel closed
11 for value in ch {
12     io.println(value)
13 }
14
15 io.println("Channel closed, loop exited")
```

8.3.5 Checking Channel State

```
1 // Receive with closed check
2 value, ok = <-ch
3 if not ok {
4     io.println("Channel closed")
5 }
6
7 // Non-blocking receive
8 value, ok = ch.try_receive()
9 if ok {
10     io.println("Got: " + to_string(value))
11 } else {
12     io.println("No value available")
13 }
14
15 // Non-blocking send
16 ok = ch.try_send(42)
17 if not ok {
18     io.println("Channel full or closed")
19 }
```

8.4 Select

The **select** statement waits on multiple channel operations:

```

1 import "io"
2 import "time"
3
4 fn main() {
5     ch1 = chan<string>()
6     ch2 = chan<string>()
7
8     spawn {
9         time.sleep(1000)
10        ch1 <- "from channel 1"
11    }
12
13    spawn {
14        time.sleep(500)
15        ch2 <- "from channel 2"
16    }
17
18    // Wait for first available
19    select {
20        msg = <-ch1 => io.println("Received: " + msg)
21        msg = <-ch2 => io.println("Received: " + msg)
22    }
23 }

```

8.4.1 Select with Default

Non-blocking select:

```

1 select {
2     value = <-ch => io.println("Got: " + to_string(value))
3     default => io.println("No value ready")
4 }

```

8.4.2 Select with Timeout

```

1 timeout = time.after(5000) // 5 seconds
2
3 select {
4     value = <-ch => io.println("Got: " + to_string(value))
5     <-timeout => io.println("Timed out!")
6 }

```

8.4.3 Select in Loop

```

1 fn worker(jobs: chan<Job>, results: chan<Result>, done:
2     chan<bool>) {
3     while true {
4         select {
5             job = <-jobs => {
6                 result = process(job)
7                 results <- result

```

```
7         }
8         <-done => {
9             io.println("Worker shutting down")
10            return
11        }
12    }
13 }
14 }
```

8.5 Common Patterns

8.5.1 Worker Pool

```
1 fn worker(id: int, jobs: chan<int>, results: chan<int>) {
2     for job in jobs {
3         io.println("Worker " + to_string(id) + " processing " +
4             to_string(job))
5         time.sleep(100)
6         results <- job * 2
7     }
8 }
9
10 fn main() {
11     num_jobs = 10
12     num_workers = 3
13
14     jobs = chan<int>(num_jobs)
15     results = chan<int>(num_jobs)
16
17     // Start workers
18     for i in 0..num_workers {
19         spawn worker(i, jobs, results)
20     }
21
22     // Send jobs
23     for j in 0..num_jobs {
24         jobs <- j
25     }
26     close(jobs)
27
28     // Collect results
29     for _ in 0..num_jobs {
30         result = <-results
31         io.println("Result: " + to_string(result))
32     }
33 }
```

8.5.2 Fan-Out, Fan-In

```
1 fn producer(out: chan<int>) {
2     for i in 0..100 {
3         out <- i
4     }
5 }
```

```

5     close(out)
6 }
7
8 fn worker(in: chan<int>, out: chan<int>) {
9     for value in in {
10         out <- value * value
11     }
12 }
13
14 fn main() {
15     input = chan<int>(100)
16     output = chan<int>(100)
17
18     // Producer
19     spawn producer(input)
20
21     // Fan-out to multiple workers
22     for _ in 0..4 {
23         spawn worker(input, output)
24     }
25
26     // Fan-in (collect results)
27     spawn {
28         time.sleep(1000)
29         close(output)
30     }
31
32     for result in output {
33         io.println(result)
34     }
35 }

```

8.5.3 Pipeline

```

1 fn generate(out: chan<int>) {
2     for i in 2..100 {
3         out <- i
4     }
5     close(out)
6 }
7
8 fn filter(in: chan<int>, out: chan<int>, prime: int) {
9     for n in in {
10         if n % prime != 0 {
11             out <- n
12         }
13     }
14     close(out)
15 }
16
17 fn sieve() -> chan<int> {
18     primes = chan<int>()
19
20     spawn {
21         ch = chan<int>(100)
22         spawn generate(ch)

```



```

23         while true {
24             prime, ok = <-ch
25             if not ok {
26                 break
27             }
28             primes <- prime
29
30             next = chan<int>(100)
31             spawn filter(ch, next, prime)
32             ch = next
33         }
34         close(primes)
35     }
36 }
37
38 return primes
39 }
40
41 fn main() {
42     for prime in sieve() {
43         io.println(prime)
44         if prime > 50 {
45             break
46         }
47     }
48 }

```

8.5.4 Semaphore

```

1 struct Semaphore {
2     ch: chan<bool>
3 }
4
5 fn new_semaphore(n: int) -> Semaphore {
6     sem = Semaphore{ch: chan<bool>(n)}
7     for _ in 0..n {
8         sem.ch <- true
9     }
10    return sem
11 }
12
13 Semaphore.acquire() {
14     <-self.ch
15 }
16
17 Semaphore.release() {
18     self.ch <- true
19 }
20
21 // Usage: limit concurrent operations
22 fn main() {
23     sem = new_semaphore(3) // Max 3 concurrent
24
25     for i in 0..10 {
26         spawn {
27             sem.acquire()

```

```

28         defer sem.release()
29
30         io.println("Task " + to_string(i) + " running")
31         time.sleep(1000)
32     }
33 }
34
35 time.sleep(5000)
36 }

```

8.6 Channel Types

8.7 Synchronization

8.7.1 WaitGroup

Wait for multiple tasks to complete:

```

1  import "sync"
2
3  fn main() {
4      wg = sync.WaitGroup{}
5
6      for i in 0..5 {
7          wg.add(1)
8          spawn {
9              defer wg.done()
10             io.println("Task " + to_string(i))
11             time.sleep(1000)
12         }
13     }
14
15     wg.wait() // Block until all done
16     io.println("All tasks completed")
17 }

```

8.7.2 Mutex

For rare cases requiring shared state:

```

1  import "sync"
2
3  struct Counter {
4      mu: sync.Mutex
5      value: int
6  }
7
8  Counter.increment() {
9      self.mu.lock()
10     defer self.mu.unlock()
11     self.value += 1
12 }
13
14 Counter.get() -> int {

```

```
15     self.mu.lock()
16     defer self.mu.unlock()
17     return self.value
18 }
```

Warning

Prefer channels over mutexes. Mutexes are provided for interoperability and performance-critical sections, but channel-based designs are generally safer and more idiomatic.

8.8 Concurrency Grammar

```
1 spawn_expr = "spawn" ( block | function_call ) ;
2
3 channel_type = "chan" "<" type ">"
4               | "chan<-" type          (* send-only *)
5               | "<-chan<" type ">"      (* receive-only *) ;
6
7 channel_create = "chan" "<" type ">" "(" [ expression ] ")" ;
8
9 send_stmt = expression "<-" expression ;
10
11 receive_expr = "<-" expression ;
12
13 select_stmt = "select" "{" { select_case } [ default_case ] "}"
14             ;
15 select_case = pattern "=" receive_expr "=>" ( expression |
16         block )
17             | send_stmt "=>" ( expression | block ) ;
18
19 default_case = "default" "=>" ( expression | block ) ;
```


Chapter 9

Modules and Imports

HAIRA uses explicit imports for all dependencies. Every external symbol must be imported before use.

9.1 Module System Overview

- **Explicit imports** – All dependencies declared at file top
- **No implicit globals** – Everything must be imported
- **Simple resolution** – Standard library, project-local, external
- **Namespaced access** – `module.function()` syntax

9.2 Import Syntax

9.2.1 Basic Import

```
1 import "io"
2 import "json"
3 import "http"
4
5 fn main() {
6     io.println("Hello")
7 }
```

9.2.2 Aliased Import

```
1 import fmt from "io"
2 import db from "haira/postgres"
3
4 fn main() {
5     fmt.println("Hello")
6     db.connect("localhost:5432")
7 }
```

9.2.3 Selective Import

Import specific symbols:

```
1 import { println, readln } from "io"
2 import { User, Post } from "models"
3
4 fn main() {
5     println("Enter name:")
6     name = readln()
7     println("Hello, " + name)
8 }
```

9.2.4 Glob Import

Import all exports (use sparingly):

```
1 import * from "math"
2
3 fn main() {
4     result = sqrt(pow(3.0, 2.0) + pow(4.0, 2.0))
5     io.println(result) // 5.0
6 }
```

Warning

Glob imports can cause naming conflicts and make code harder to read. Prefer explicit imports.

9.3 Module Resolution

Imports are resolved in this order:

1. **Standard library** – Built-in modules ("io", "json", etc.)
2. **Project-local** – Relative to project root ("models/user")
3. **External packages** – From package registry ("github.com/...")

9.3.1 Standard Library

```
1 import "io"           // Standard I/O
2 import "string"        // String manipulation
3 import "math"          // Mathematical functions
4 import "json"          // JSON encoding/decoding
5 import "http"          // HTTP client/server
6 import "fs"            // File system
7 import "os"            // Operating system
8 import "time"          // Time and duration
9 import "crypto"        // Cryptography
10 import "regex"         // Regular expressions
```

9.3.2 Project-Local Imports

```

1 // Project structure:
2 // myapp/
3 //   main.haira
4 //   models/
5 //     user.haira
6 //     post.haira
7 //   utils/
8 //     helpers.haira
9
10 // In main.haira:
11 import "models/user"
12 import "models/post"
13 import "utils/helpers"
14
15 // In models/post.haira:
16 import "models/user" // Absolute from project root

```

9.3.3 External Packages

```

1 import "github.com/haira-libs/aws"
2 import "github.com/haira-libs/postgres"
3 import pg from "github.com/haira-libs/postgres"

```

External packages are declared in `haira.toml`:

```

1 [dependencies]
2 aws = "github.com/haira-libs/aws@1.0.0"
3 postgres = "github.com/haira-libs/postgres@2.1.0"

```

9.4 Module Definitions

9.4.1 File as Module

Each `.haira` file is a module. The file name determines the module name:

```

1 // File: utils/helpers.haira
2 // Module: utils/helpers
3
4 pub fn format_name(first: string, last: string) -> string {
5     return first + " " + last
6 }
7
8 fn validate_email(email: string) -> bool {
9     return string.contains(email, "@")
10 }

```

9.4.2 Visibility: The `pub` Keyword

Declarations are **private by default**. Use the **`pub`** keyword to make them available to importers:

```

1 // Public (exported) -- available to other modules
2 pub fn create_user(name: string) -> User { }
3 pub struct User { name: string, email: string }
4 pub enum Status { Active, Inactive }
5
6 // Private (not exported) -- file-local only
7 fn validate_email(email: string) -> bool { }
8 cache = [:]
```

Note

Agentic declarations (**`provider`**, **`tool`**, **`agent`**, **`workflow`**) are always public — they are inherently part of the module’s external interface.

9.4.3 Package Initialization

Optional `init()` function runs when module is first imported:

```

1 // database.haira
2
3 pool: ConnectionPool?
4
5 fn init() {
6     pool = create_pool()
7     io.println("Database module initialized")
8 }
9
10 pub fn get_connection() -> (Connection, Error?) {
11     if pool == nil {
12         return Connection{}, Error{message: "not initialized"}
13     }
14     return pool.get(), nil
15 }
```

Note

`init()` functions are called in dependency order. Circular dependencies are a compile-time error.

9.5 Package Structure

9.5.1 Single-File Package

```

1 myapp/
2   main.haira
3   haira.toml
```


9.5.2 Multi-File Package

```
1 myapp/  
2   main.haira  
3   models/  
4     user.haira  
5     post.haira  
6     mod.haira      # Package entry point (optional)  
7   services/  
8     auth.haira  
9     api.haira  
10  utils/  
11    helpers.haira  
12  haira.toml
```

9.5.3 The mod.haira File

Re-exports from a directory:

```
1 // models/mod.haira  
2 import { User } from "models/user"  
3 import { Post } from "models/post"  
4  
5 // Re-export  
6 export { User, Post }
```

Now consumers can import from the directory:

```
1 import { User, Post } from "models"
```

9.6 Import Order

Imports should be organized in groups:

```
1 // 1. Standard library  
2 import "io"  
3 import "json"  
4 import "http"  
5  
6 // 2. External packages  
7 import "github.com/haira-libs/aws"  
8  
9 // 3. Project-local  
10 import "models/user"  
11 import "services/auth"
```

9.7 Circular Dependencies

Circular imports are not allowed:

```
1 // a.haira
```

```

2 import "b" // Error if b imports a
3
4 // b.haira
5 import "a" // Creates circular dependency

```

Solution: Extract shared code to a third module:

```

1 // shared.haira
2 struct Common { }
3
4 // a.haira
5 import "shared"
6
7 // b.haira
8 import "shared"

```

9.8 The haira.toml File

Project configuration:

```

1 [package]
2 name = "myapp"
3 version = "1.0.0"
4 authors = ["Your Name <you@example.com>"]
5
6 [dependencies]
7 aws = "github.com/haira-libs/aws@1.0.0"
8 postgres = "github.com/haira-libs/postgres@2.1.0"
9
10 [dev-dependencies]
11 testing = "github.com/haira-libs/testing@1.0.0"
12
13 [build]
14 target = "native"
15 optimization = "release"

```

9.9 Module Grammar

```

1 import_statement = "import" import_spec ;
2
3 import_spec = string_literal (* basic
4 *)
5             | identifier "from" string_literal (*
6             | " {" identifier_list "}" "from" string_literal (*
7             | " {" identifier_list "}" "from" string_literal (*
8             | " {" identifier_list "}" "from" string_literal (*
9             | " {" identifier_list "}" "from" string_literal (*
10            | " {" identifier_list "}" "from" string_literal (*
11            | " {" identifier_list "}" "from" string_literal (*

```

```
12 module = { import_statement } { export_statement } { top_level  
    } ;
```


Chapter 10

Standard Library

The HAIRA standard library provides essential functionality for common programming tasks. All modules must be explicitly imported.

10.1 Core Modules

Module	Description
<code>io</code>	Input/output operations
<code>string</code>	String manipulation
<code>array</code>	Array operations
<code>map</code>	Map/dictionary operations
<code>math</code>	Mathematical functions
<code>conv</code>	Type conversions

Table 10.1: Core modules

10.2 io Module

Basic input/output operations.

```
1 import "io"
2
3 // Output
4 io.print("Hello")           // No newline
5 io.println("Hello")        // With newline
6 io.printf("Value: %d\n", 42) // Formatted
7
8 // Input
9 line = io.readLine()        // Read line from stdin
10
11 // Errors
12 io.eprintln("Error!")       // Print to stderr
```

10.2.1 io Functions

```
1 fn print(s: string)
```

```

2 fn println(s: string)
3 fn printf(format: string, args: ...any)
4 fn readln() -> string
5 fn eprintln(s: string)
6 fn eprintf(format: string, args: ...any)

```

10.3 string Module

String manipulation functions.

```

1 import "string"
2
3 s = "  Hello, World!  "
4
5 // Basic operations
6 string.len(s) // 18
7 string.is_empty("") // true
8
9 // Trimming
10 string.trim(s) // "Hello, World!"
11 string.trim_left(s) // "Hello, World!  "
12 string.trim_right(s) // "  Hello, World!"
13
14 // Case
15 string.to_upper(s) // "  HELLO, WORLD!  "
16 string.to_lower(s) // "  hello, world!  "
17
18 // Search
19 string.contains(s, "World") // true
20 string.starts_with(s, " H") // true
21 string.ends_with(s, "! ") // true
22 string.index_of(s, "o") // 4
23 string.last_index_of(s, "o") // 8
24
25 // Split/Join
26 string.split("a,b,c", ",") // ["a", "b", "c"]
27 string.join(["a", "b"], "-") // "a-b"
28
29 // Substring
30 string.substring(s, 2, 7) // "Hello"
31 string.char_at(s, 2) // "H"
32
33 // Replace
34 string.replace(s, "World", "Haira") // "  Hello, Haira!  "
35 string.replace_all(s, " ", "_") // "__Hello,_World!__"
36
37 // Repeat
38 string.repeat("ab", 3) // "ababab"

```

10.3.1 string Functions

```

1 fn len(s: string) -> int
2 fn is_empty(s: string) -> bool
3 fn trim(s: string) -> string

```

```

4 fn trim_left(s: string) -> string
5 fn trim_right(s: string) -> string
6 fn to_upper(s: string) -> string
7 fn to_lower(s: string) -> string
8 fn contains(s: string, sub: string) -> bool
9 fn starts_with(s: string, prefix: string) -> bool
10 fn ends_with(s: string, suffix: string) -> bool
11 fn index_of(s: string, sub: string) -> int
12 fn last_index_of(s: string, sub: string) -> int
13 fn split(s: string, sep: string) -> []string
14 fn join(parts: []string, sep: string) -> string
15 fn substring(s: string, start: int, end: int) -> string
16 fn char_at(s: string, index: int) -> string
17 fn replace(s: string, old: string, new: string) -> string
18 fn replace_all(s: string, old: string, new: string) -> string
19 fn repeat(s: string, n: int) -> string

```

10.4 array Module

Array operations.

```

1 import "array"
2
3 arr = [1, 2, 3, 4, 5]
4
5 // Basic
6 array.len(arr) // 5
7 array.is_empty([]) // true
8
9 // Access
10 array.first(arr) // 1 (or nil if empty)
11 array.last(arr) // 5 (or nil if empty)
12 array.get(arr, 2) // 3 (or nil if out of bounds)
13
14 // Modification (returns new array)
15 array.push(arr, 6) // [1, 2, 3, 4, 5, 6]
16 array.pop(arr) // ([1, 2, 3, 4], 5)
17 array.insert(arr, 0, 0) // [0, 1, 2, 3, 4, 5]
18 array.remove(arr, 2) // [1, 2, 4, 5]
19
20 // Slicing
21 array.slice(arr, 1, 4) // [2, 3, 4]
22 array.take(arr, 3) // [1, 2, 3]
23 array.drop(arr, 2) // [3, 4, 5]
24
25 // Search
26 array.contains(arr, 3) // true
27 array.index_of(arr, 3) // 2
28 array.find(arr, fn(x) { x > 3 }) // 4
29
30 // Transform
31 array.map(arr, fn(x) { x * 2 }) // [2, 4, 6, 8, 10]
32 array.filter(arr, fn(x) { x > 2 }) // [3, 4, 5]
33 array.reduce(arr, 0, fn(acc, x) { acc + x }) // 15
34
35 // Sort

```

```

36 array.sort(arr) // [1, 2, 3, 4, 5]
37 array.sort_by(arr, fn(a, b) { b - a }) // [5, 4, 3, 2, 1]
38
39 // Other
40 array.reverse(arr) // [5, 4, 3, 2, 1]
41 array.concat(arr, [6, 7]) // [1, 2, 3, 4, 5, 6, 7]
42 array.flatten([[1, 2], [3, 4]]) // [1, 2, 3, 4]
43 array.unique([1, 2, 2, 3]) // [1, 2, 3]

```

10.5 map Module

Map/dictionary operations.

```

1  import "map"
2
3  m = {"a": 1, "b": 2, "c": 3}
4
5  // Basic
6  map.len(m) // 3
7  map.is_empty(m) // false
8
9  // Access
10 map.get(m, "a") // 1 (or nil if not found)
11 map.has(m, "a") // true
12
13 // Modification (returns new map)
14 map.set(m, "d", 4) // {"a": 1, "b": 2, "c": 3, "d": 4}
15 map.remove(m, "b") // {"a": 1, "c": 3}
16
17 // Iteration helpers
18 map.keys(m) // ["a", "b", "c"]
19 map.values(m) // [1, 2, 3]
20 map.entries(m) // [{"a", 1}, {"b", 2}, {"c", 3}]
21
22 // Transform
23 map.map_values(m, fn(v) { v * 2 }) // {"a": 2, "b": 4, "c": 6}
24 map.filter(m, fn(k, v) { v > 1 }) // {"b": 2, "c": 3}
25
26 // Merge
27 map.merge(m, [{"d": 4}]) // {"a": 1, "b": 2, "c": 3, "d": 4}

```

10.6 math Module

Mathematical functions.

```

1  import "math"
2
3  // Constants
4  math.PI // 3.14159265358979
5  math.E // 2.71828182845905
6

```



```

7  // Basic
8  math.abs(-5)                // 5
9  math.min(3, 7)              // 3
10 math.max(3, 7)              // 7
11 math.clamp(5, 0, 10)        // 5
12 math.clamp(-5, 0, 10)       // 0
13
14 // Rounding
15 math.floor(3.7)              // 3.0
16 math.ceil(3.2)              // 4.0
17 math.round(3.5)             // 4.0
18 math.trunc(3.7)             // 3.0
19
20 // Power/Root
21 math.pow(2.0, 10.0)          // 1024.0
22 math.sqrt(16.0)             // 4.0
23 math.cbrt(27.0)             // 3.0
24
25 // Exponential/Logarithm
26 math.exp(1.0)                // 2.718...
27 math.log(math.E)             // 1.0
28 math.log10(100.0)           // 2.0
29 math.log2(8.0)              // 3.0
30
31 // Trigonometry
32 math.sin(0.0)                // 0.0
33 math.cos(0.0)                // 1.0
34 math.tan(0.0)                // 0.0
35 math.asin(0.0)              // 0.0
36 math.acos(1.0)              // 0.0
37 math.atan(0.0)              // 0.0
38 math.atan2(1.0, 1.0)        // 0.785... (PI/4)
39
40 // Random
41 math.random()                // Random float 0.0..1.0
42 math.random_int(1, 100)      // Random int 1..100

```

10.7 conv Module

Type conversion utilities.

```

1  import "conv"
2
3  // To string
4  conv.int_to_string(42)       // "42"
5  conv.float_to_string(3.14)   // "3.14"
6  conv.bool_to_string(true)    // "true"
7
8  // From string
9  conv.string_to_int("42")     // (42, nil)
10 conv.string_to_int("abc")     // (0, Error)
11 conv.string_to_float("3.14")  // (3.14, nil)
12 conv.string_to_bool("true")   // (true, nil)
13
14 // Number conversions
15 conv.int_to_float(42)         // 42.0

```

```

16 conv.float_to_int(3.7)           // 3 (truncates)
17
18 // Base conversions
19 conv.int_to_hex(255)             // "ff"
20 conv.int_to_binary(10)           // "1010"
21 conv.int_to_octal(8)             // "10"
22 conv.hex_to_int("ff")            // (255, nil)

```

10.8 Extended Modules

10.8.1 fs Module

File system operations.

```

1 import "fs"
2
3 // Read/Write
4 content, err = fs.read_file("file.txt")
5 err = fs.write_file("file.txt", "content")
6 err = fs.append_file("file.txt", "more")
7
8 // File operations
9 exists = fs.exists("file.txt")
10 err = fs.remove("file.txt")
11 err = fs.rename("old.txt", "new.txt")
12 err = fs.copy("src.txt", "dst.txt")
13
14 // Directory operations
15 err = fs.mkdir("dir")
16 err = fs.mkdir_all("path/to/dir")
17 entries, err = fs.read_dir("dir")
18 err = fs.remove_all("dir")
19
20 // File info
21 info, err = fs.stat("file.txt")
22 info.size           // Size in bytes
23 info.is_dir         // Is directory?
24 info.modified       // Modification time

```

10.8.2 os Module

Operating system interface.

```

1 import "os"
2
3 // Environment
4 os.getenv("HOME")           // Get env variable
5 os.setenv("KEY", "value")   // Set env variable
6 os.environ()                // All env variables
7
8 // Process
9 os.args                     // Command line arguments
10 os.exit(0)                  // Exit with code
11 os.getpid()                 // Process ID
12

```

```

13 // Execution
14 output, err = os.exec("ls", ["-la"])

```

10.8.3 time Module

Time and duration.

```

1  import "time"
2
3  // Current time
4  now = time.now()
5  now.year           // 2024
6  now.month          // 1-12
7  now.day            // 1-31
8  now.hour           // 0-23
9  now.minute         // 0-59
10 now.second         // 0-59
11
12 // Formatting
13 time.format(now, "2006-01-02") // "2024-03-15"
14
15 // Parsing
16 t, err = time.parse("2024-03-15", "2006-01-02")
17
18 // Duration
19 time.sleep(1000)           // Sleep 1 second (ms)
20 duration = time.since(start) // Time since start
21
22 // Timers
23 timer = time.after(5000)   // Channel that fires after 5s
24 ticker = time.tick(1000)   // Channel that fires every 1s

```

10.8.4 json Module

JSON encoding/decoding.

```

1  import "json"
2
3  struct User {
4      name: string
5      age: int
6  }
7
8  // Encoding
9  user = User{name: "Alice", age: 30}
10 data, err = json.encode(user) // '{"name": "Alice", "age": 30}'
11
12 // Decoding
13 user, err = json.decode<User>(data)
14
15 // Pretty printing
16 data, err = json.encode_pretty(user)
17
18 // Raw JSON
19 value, err = json.parse('{"key": "value"}')

```

```
20 value["key"] // "value"
```

10.8.5 http Module

HTTP client and server.

```
1 import "http"
2
3 // GET request
4 response, err = http.get("https://api.example.com/users")
5 response.status // 200
6 response.body // Response body
7
8 // POST request
9 response, err = http.post(
10     "https://api.example.com/users",
11     [{"Content-Type": "application/json"}],
12     '{"name": "Alice"}'
13 )
14
15 // Server
16 server = http.Server{port: 8080}
17
18 server.handle("/", fn(req: http.Request) -> http.Response {
19     return http.Response{
20         status: 200,
21         body: "Hello, World!"
22     }
23 })
24
25 server.listen()
```

10.8.6 regex Module

Regular expressions.

```
1 import "regex"
2
3 // Compile pattern
4 pattern, err = regex.compile(r"\d+")
5
6 // Match
7 regex.is_match(pattern, "abc123") // true
8
9 // Find
10 match = regex.find(pattern, "abc123def") // "123"
11 matches = regex.find_all(pattern, "a1b2c3") // ["1", "2", "3"]
12
13 // Replace
14 regex.replace(pattern, "a1b2", "X") // "aXb2"
15 regex.replace_all(pattern, "a1b2", "X") // "aXbX"
16
17 // Capture groups
18 pattern, _ = regex.compile(r"(\w+)@(\w+)")
19 captures = regex.captures(pattern, "alice@example")
```

```
20 // captures[0] = "alice@example"
21 // captures[1] = "alice"
22 // captures[2] = "example"
```

10.8.7 encoding Module

Encoding and decoding utilities.

```
1 import "encoding"
2
3 // Base64
4 encoding.base64_encode(data)
5 encoding.base64_decode(string)
6
7 // Hex
8 encoding.hex_encode(data)
9 encoding.hex_decode(string)
```

10.9 Testing

HAIRA has built-in testing support. Tests are declared with the **test** keyword followed by a name and a block of assertions.

10.9.1 Test Blocks

```
1 fn add(a: int, b: int) -> int {
2     return a + b
3 }
4
5 fn is_even(n: int) -> bool {
6     return n % 2 == 0
7 }
8
9 test "add returns correct sum" {
10     assert add(1, 2) == 3, "1+2 should equal 3"
11     assert add(0, 0) == 0
12     assert add(-1, 1) == 0
13 }
14
15 test "is_even checks parity" {
16     assert is_even(4)
17     assert not is_even(3)
18 }
```

10.9.2 Assertions

The **assert** statement checks a condition. If it fails, the test reports the failure and continues.

```
1 // Boolean assertion
2 assert is_valid(input)
3
```

```
4 // Equality check (reports "got X, want Y" on failure)
5 assert result == 42
6
7 // Inequality check
8 assert a != b
9
10 // With custom message
11 assert count > 0, "count should be positive"
```

10.9.3 Running Tests

```
$ haira test file.haira
```

The compiler generates Go test functions and runs them with `go test`. Each `test` block becomes a subtest. Output follows standard test format:

```
--- PASS: TestHaira/add_returns_correct_sum (0.00s)
--- PASS: TestHaira/is_even_checks_parity (0.00s)
PASS
```

Part III

Agentic Orchestration

Chapter 11

Providers

Providers configure LLM backends. They are declared at the top level and referenced by agents.

11.1 Provider Declaration

```
1 provider provider_name {  
2     field: value  
3     field: value  
4 }
```

11.1.1 Cloud Providers

```
1 provider anthropic {  
2     api_key: env("ANTHROPIC_API_KEY")  
3     model: "claude-sonnet-4-20250514"  
4 }  
5  
6 provider openai {  
7     api_key: env("OPENAI_API_KEY")  
8     model: "gpt-4o"  
9 }
```

11.1.2 Local Providers

```
1 provider local {  
2     backend: "ollama"  
3     host: "localhost:11434"  
4     model: "llama3:8b"  
5 }
```

Field	Type	Required	Description
model	string	Yes	Default model identifier
api_key	string	No	API authentication key
backend	string	No	Backend type ("ollama", "llama.cpp")
host	string	No	Backend host address
temperature	float	No	Default temperature (0.0–2.0)
max_tokens	int	No	Default max tokens per response

Table 11.1: Provider fields

11.2 Provider Fields

11.3 Environment Variables

The `env()` function reads environment variables at runtime:

```

1 provider anthropic {
2     api_key: env("ANTHROPIC_API_KEY")
3     model: env("ANTHROPIC_MODEL") or "claude-sonnet-4-20250514"
4 }
```

Warning

Never hardcode API keys in source files. Always use `env()` to read them from the environment.

11.4 Referencing Providers

Providers are referenced by name in agent declarations:

```

1 agent Writer {
2     provider: anthropic           // uses anthropic provider
3     system: "You are a writer."
4 }
5
6 agent FastHelper {
7     provider: local               // uses local Ollama provider
8     system: "You are a fast helper."
9 }
```

11.5 Multiple Providers

A program can declare multiple providers. Agents choose which to use:

```

1 provider anthropic {
2     api_key: env("ANTHROPIC_API_KEY")
3     model: "claude-sonnet-4-20250514"
4 }
5
6 provider local {
7     backend: "ollama"
```

```

8     host: "localhost:11434"
9     model: "llama3:8b"
10 }
11
12 // Use expensive cloud model for important tasks
13 agent Analyst {
14     provider: anthropic
15     system: "You analyze data carefully."
16 }
17
18 // Use cheap local model for simple tasks
19 agent Classifier {
20     provider: local
21     system: "Classify the input into categories."
22 }

```

11.6 MCP Providers

The Model Context Protocol (MCP) enables agents to use tools from external processes. MCP providers spawn a subprocess (or connect via HTTP) and discover its tools automatically.

11.6.1 Stdio Transport

The most common MCP transport. The runtime spawns a subprocess and communicates via JSON-RPC over stdin/stdout:

```

1 provider filesystem {
2     transport: "mcp"
3     command: "npx"
4     args: ["-y", "@modelcontextprotocol/server-filesystem",
5           "/tmp"]
6 }

```

11.6.2 SSE Transport

For remote MCP servers accessible over HTTP:

```

1 provider remote_tools {
2     transport: "sse"
3     endpoint: "http://localhost:9000/sse"
4 }

```

11.6.3 MCP Provider Fields

11.6.4 Using MCP in Agents

Agents reference MCP providers with the `mcp` field. The runtime discovers tools from all listed MCP providers and makes them available alongside regular tools:

```

1 agent Assistant {
2     provider: azure_openai

```

Field	Type	Required	Description
transport	string	Yes	"mcp" (stdio) or "sse" (HTTP)
command	string	Stdio only	Executable to spawn
args	[string]	No	Command arguments
endpoint	string	SSE only	Server URL
env	map	No	Environment variables for subprocess
headers	map	No	HTTP headers (SSE only)

Table 11.2: MCP provider fields

```

3   system: "You are a helpful assistant with file system
      access."
4   tools: [greet]                // regular tools
5   mcp: [filesystem]            // MCP tools (auto-discovered)
6   memory: conversation(max_turns: 10)
7 }

```

11.6.5 MCP Server

Haira workflows can be exposed as MCP tools for other applications to consume:

```

1  import "mcp"
2
3  workflow Summarize(text: string) -> { summary: string } {
4      """Summarize the given text into key points."""
5      summary, err = Summarizer.ask(text)
6      if err != nil { return { summary: "Error" } }
7      return { summary: summary }
8  }
9
10 fn main() {
11     mcp_server = mcp.Server([Summarize])
12     mcp_server.listen(9000) // SSE on port 9000
13 }

```

The `mcp.Server` exposes each workflow as an MCP tool. Tool names, descriptions, and input schemas are derived from the workflow declaration. The server supports both SSE (`.listen(port)`) and stdio (`.serve()`) transports.

11.7 Provider Grammar

```

1  provider_decl = "provider" identifier "{" { provider_field }
      "}" ;
2  provider_field = identifier ":" expression ;

```

Chapter 12

Tools

Tools are typed functions with LLM-visible descriptions. The compiler auto-generates JSON schemas from the type signature, enabling agents to call tools with structured arguments.

12.1 Tool Declaration

```
1 tool tool_name(param: Type, param: Type = default) ->
  ReturnType {
2   """Description visible to the LLM"""
3
4   // implementation
5 }
```

A tool is like **fn** with two additions:

1. The **tool** keyword instead of **fn**
2. A mandatory triple-quoted description as the first element of the body

12.2 Basic Examples

12.2.1 HTTP Tool

```
1 import "tools/http"
2 import "json"
3
4 struct SearchResult {
5   url: string
6   title: string
7   snippet: string
8 }
9
10 tool search(query: string, max_results: int = 5) ->
  [SearchResult] {
11   """
12   Search the web for information.
13   Returns up to max_results results.
14   Supports boolean operators in query.
15   """
```

```

16
17     resp, err =
18         http.get("https://api.search.com?q={query}&n={max_results}")
19     if err != nil { return [], err }
20     return json.decode(resp.body, [SearchResult])

```

12.2.2 Database Tool

```

1 import "tools/postgres"
2
3 tool get_user(user_id: string) -> User {
4     """Look up a user by their ID"""
5
6     user, err = pg.query_one("SELECT * FROM users WHERE id =
7         $1", [user_id])
8     if err != nil { return User{}, err }
9     return user
10 }

```

12.2.3 External Service Tool

```

1 import "tools/http"
2
3 tool send_slack(channel: string, message: string) -> bool {
4     """Send a message to a Slack channel"""
5
6     resp, err =
7         http.post("https://slack.com/api/chat.postMessage", {
8             channel: channel,
9             text: message
10         })
11     return err == nil

```

12.2.4 Agent-Delegating Tool

Tools can call agents internally:

```

1 tool summarize(text: string) -> string {
2     """Summarize text into key points"""
3
4     result, _ = Writer.ask("Summarize concisely: {text}")
5     return result
6 }

```

12.3 Triple-Quote Description

The description is mandatory and must be the first element of the tool body. It is a triple-quoted string ("""...""") that can span multiple lines:

```

1  tool analyze(data: [DataPoint]) -> Analysis {
2      """
3      Analyze a dataset and produce a statistical summary.
4
5      Includes:
6      - Mean, median, and standard deviation
7      - Outlier detection
8      - Trend analysis over time
9
10     Use this tool when the user asks about patterns or
11       statistics.
12     """
13     // implementation
14 }

```

Note

The compiler will emit an error if a tool is missing its description. This is enforced because the description is sent to the LLM as part of the tool-calling protocol.

12.4 Auto-Generated JSON Schema

The compiler generates a JSON schema from the tool's type signature. For example:

```

1  tool search(query: string, max_results: int = 5) ->
2    [SearchResult] {
3      """Search the web for information"""
4      // ...
5  }

```

Generates (conceptually):

```

1  {
2    "name": "search",
3    "description": "Search the web for information",
4    "parameters": {
5      "type": "object",
6      "properties": {
7        "query": { "type": "string" },
8        "max_results": { "type": "integer", "default": 5 }
9      },
10     "required": ["query"]
11   }
12 }

```

This happens at compile time. No hand-written JSON schemas are needed.

12.5 Tool Packs (Standard Library)

HAIRA ships with built-in tool packs for common integrations:

```

1 import "tools/http"           // http.get, http.post, http.put,
   http.delete
2 import "tools/slack"          // slack.send, slack.read_channel
3 import "tools/github"         // github.create_issue,
   github.list_prs, github.get_diff
4 import "tools/postgres"       // pg.query, pg.query_one, pg.insert,
   pg.update
5 import "tools/email"          // email.send
6 import "tools/fs"             // fs.read, fs.write, fs.list

```

Tool packs provide pre-built **tool** declarations. They can be used directly in agent **tools** lists or called from within other tools and workflows.

12.6 Error Handling in Tools

Tools follow HAIRA's standard error handling pattern:

```

1 tool fetch_page(url: string) -> string {
2     """Fetch the content of a web page"""
3
4     resp, err = http.get(url)
5     if err != nil {
6         return "", err
7     }
8     if resp.status != 200 {
9         return "", Error{message: "HTTP {resp.status}"}
10    }
11    return resp.body
12 }

```

When a tool returns an error, the agent receives the error message and can decide how to proceed (retry, try a different tool, or report the error to the user).

12.7 Tool Grammar

```

1 tool_decl      = "tool" identifier "(" [ params ] ")" [ "->"
   type ]
2               "{" triple_string { statement } "}" ;
3
4 triple_string = '"""' { any_char } '"""' ;

```


Chapter 13

Agents

Agents are LLM-powered entities. They have a provider, a system prompt, optional tools, and optional memory. Agents are the primary way to interact with language models in HAIRA.

13.1 Agent Declaration

```
1 agent AgentName {  
2   provider: provider_name  
3   system: "System prompt describing the agent's behavior."  
4   tools: [tool1, tool2]  
5   temperature: 0.5  
6   timeout: 60  
7   memory: conversation(max_turns: 50)  
8 }
```

13.2 Agent Fields

Field	Type	Required	Description
provider	identifier	Yes	Provider to use
system	string	Yes	System prompt
tools	[tool]	No	List of tools the agent can call
temperature	float	No	Sampling temperature (0.0–2.0)
max_tokens	int	No	Max tokens per response
max_steps	int	No	Max tool-calling iterations
memory	memory type	No	Memory configuration (default: none)
handoffs	[agent]	No	Agents this agent can delegate to
timeout	int	No	Timeout in seconds (default: 120)

Table 13.1: Agent fields

13.2.1 Multiline System Prompts

For longer system prompts, use triple-quoted strings:

```
1 agent SupportBot {
```

```

2   provider: anthropic
3   system: """
4       You are a customer support agent for Acme Corp.
5       Use the knowledge base to answer questions.
6       Look up orders when customers ask about purchases.
7       Create tickets for issues you cannot resolve.
8       Always be polite and helpful.
9   """
10  tools: [search_kb, lookup_order, create_ticket]
11  temperature: 0.3
12 }

```

13.3 Agent Methods

Agents have three methods for interacting with the LLM:

13.3.1 .ask() — Text In, Text Out

The simplest interaction. Send a text prompt, get a text response:

```

1  answer, err = Researcher.ask("What is quantum computing?")
2  if err != nil {
3      io.println("Error: " + err.message)
4      return
5  }
6  io.println(answer)

```

Signature: `.ask(prompt: string, session: string?) -> (string, Error?)`

13.3.2 .run() — Structured In, Structured Out

For typed interactions. Send a struct, get a typed response. The output type is inferred from the left-side type annotation:

```

1  struct ResearchRequest {
2      topic: string
3      depth: string = "detailed"
4  }
5
6  struct ResearchResult {
7      summary: string
8      sources: [string]
9      confidence: float
10 }
11
12 research: ResearchResult, err = Researcher.run(ResearchRequest{
13     topic: "quantum computing"
14 })

```

The compiler generates JSON schemas for both the input and output types. The agent is instructed to return structured data matching the output schema.

Signature: `.run(input: T, session: string?) -> (U, Error?)`

Where U is inferred from the left-side type annotation.

13.3.3 .stream() — Text In, Streaming Out

For real-time token-by-token responses. Returns an iterable stream:

```
1 for chunk in Assistant.stream("Tell me a story") {
2     io.print(chunk)
3 }
4 io.println() // newline after stream completes
```

Signature: `.stream(prompt: string, session: string?) -> stream<string>`

The `stream<string>` type is iterable with `for`. Each iteration yields the next token (or chunk) as the LLM generates it.

13.4 Memory

By default, agents are stateless — each call is independent. Adding memory enables agents to remember conversation history across calls.

13.4.1 No Memory (Default)

```
1 agent Classifier {
2     provider: local
3     system: "Classify inputs into categories."
4 }
5
6 // Each call is independent
7 cat1 = Classifier.ask("This is a bug report") // "technical"
8 cat2 = Classifier.ask("What did I just send you?" ) // has no
    context
```

13.4.2 Conversation Memory

Keeps the last N turns of chat history:

```
1 agent Assistant {
2     provider: anthropic
3     system: "You are a helpful assistant."
4     memory: conversation(max_turns: 50)
5 }
```

13.4.3 Summary Memory

Summarizes older messages to stay within a token budget. Useful for long-running conversations:

```
1 agent ProjectHelper {
2     provider: anthropic
3     system: "You help with long-running software projects."
4     memory: conversation(max_turns: 50)
5 }
```

13.4.4 Memory Types

Type	Parameter	Behavior
<code>none</code>	—	Stateless (default)
<code>conversation(max_turns: N)</code>	<code>N: int</code>	Keeps last N message pairs

Table 13.2: Built-in memory types

13.5 Sessions

When an agent has memory, calls are scoped to a **session**. A session groups related interactions — for example, one user’s conversation.

```

1 // With session --- memory persists within same session
2 answer1, _ = Assistant.ask("My name is Alice", session:
3   "user-123")
4 answer2, _ = Assistant.ask("What is my name?", session:
5   "user-123")
6 // answer2 = "Your name is Alice"
7
8 // Different session --- separate memory
9 answer3, _ = Assistant.ask("What is my name?", session:
10  "user-456")
11 // answer3 = "I don't know your name yet"

```

The `session:` parameter is a named argument available on all agent methods:

```

1 // .ask() with session
2 reply, err = Agent.ask("Hello", session: session_id)
3
4 // .run() with session
5 result: T, err = Agent.run(input, session: session_id)
6
7 // .stream() with session
8 for chunk in Agent.stream("Hello", session: session_id) {
9   io.print(chunk)
10 }

```

Without a `session:` parameter, each call starts a fresh conversation — even if the agent has memory configured.

13.6 Handoffs

Agents can transfer a conversation to another agent. This is useful when a generalist agent detects that a specialist should take over.

13.6.1 Declaring Handoffs

Use the `handoffs` field to list agents that this agent can delegate to:

```

1 agent FrontDesk {
2   provider: anthropic
3   system: ""

```

```

4         You are a front desk agent. Greet users and help them.
5         If they have a billing question, hand off to
           BillingAgent.
6         If they have a technical issue, hand off to TechAgent.
7         """
8         handoffs: [BillingAgent, TechAgent]
9         memory: conversation(max_turns: 10)
10    }
11
12    agent BillingAgent {
13        provider: anthropic
14        system: "You handle billing questions. You can look up
           invoices and process refunds."
15        tools: [lookup_invoice, process_refund]
16        memory: conversation(max_turns: 30)
17    }
18
19    agent TechAgent {
20        provider: anthropic
21        system: "You handle technical support. You can search docs
           and check system status."
22        tools: [search_docs, check_status]
23        memory: conversation(max_turns: 30)
24    }

```

13.6.2 How Handoffs Work

When an agent has `handoffs`, the runtime injects synthetic tools for each target agent (e.g., `transfer_to_BillingAgent`). The LLM decides when to call these tools based on the system prompt and conversation context.

The handoff process:

1. The LLM calls a handoff tool (e.g., `transfer_to_BillingAgent`)
2. The runtime transfers the conversation history to the target agent's session
3. The target agent processes the original message with full context
4. The target agent's response is returned to the caller

13.6.3 `.ask()` — Automatic Handoffs

When using `.ask()`, handoffs are automatic. The caller sees a seamless response regardless of which agent ultimately handled it:

```

1 @webhook("/api/chat")
2 workflow Chat(message: string, session_id: string) -> { reply:
   string } {
3     // FrontDesk may hand off to BillingAgent or TechAgent
       internally
4     // The caller just gets the final reply
5     reply, err = FrontDesk.ask(message, session: session_id)
6     if err != nil { return { reply: "Something went wrong." } }
7     return { reply: reply }
8 }

```

13.6.4 .run() — Manual Handoff Control

When using `.run()`, the workflow receives an `AgentResult` that includes handoff information, allowing manual inspection or override:

```

1 @webhook("/api/chat")
2 workflow Chat(message: string, session_id: string) -> { reply:
   string } {
3     result: AgentResult, err = FrontDesk.run(message, session:
       session_id)
4     if err != nil { return { reply: "Something went wrong." } }
5
6     if result.handed_off_to != nil {
7         io.println("Handed off to: " + result.handed_off_to)
8     }
9
10    return { reply: result.reply }
11 }

```

13.6.5 Handoff Chains

Target agents can themselves have handoffs, creating delegation chains. The runtime follows the chain until an agent produces a final response (up to a configurable depth limit):

```

1 agent L1Support {
2     provider: anthropic
3     system: "You handle basic support. Escalate complex issues
       to L2."
4     handoffs: [L2Support]
5 }
6
7 agent L2Support {
8     provider: anthropic
9     system: "You handle complex technical issues. Escalate
       critical outages to L3."
10    tools: [search_docs, check_status]
11    handoffs: [L3Support]
12 }
13
14 agent L3Support {
15     provider: anthropic
16     system: "You handle critical outages. You have full system
       access."
17     tools: [search_docs, check_status, restart_service,
       page_oncall]
18 }

```

13.7 Tool-Calling Loop

When an agent has tools, the runtime executes a loop:

1. Send the prompt (and conversation history) to the LLM
2. If the LLM requests a tool call, execute the tool

3. Send the tool result back to the LLM
4. Repeat until the LLM produces a final response (or `max_steps` is reached)

This is transparent to the caller:

```

1 // The agent may call search() multiple times internally
2 // before producing a final answer
3 answer, err = Researcher.ask("Find recent papers on quantum
   error correction")

```

The `max_steps` field limits how many tool-calling iterations are allowed:

```

1 agent Researcher {
2   provider: anthropic
3   system: "You are a thorough researcher."
4   tools: [search, read_url]
5   max_steps: 10      // at most 10 tool calls per request
6   temperature: 0.2
7 }

```

13.8 Error Handling

Agent calls can fail. Always check for errors:

```

1 answer, err = Researcher.ask("Complex query here")
2 if err != nil {
3   io.println("Agent error: " + err.message)
4   // Handle: retry, fallback, or report
5 }

```

Common error cases:

- LLM provider unreachable or returns an error
- Max steps exceeded (tool-calling loop did not converge)
- Structured output did not match expected schema
- Timeout exceeded

13.9 Examples

13.9.1 Stateless Classifier

```

1 agent Classifier {
2   provider: local
3   system: "Classify the input as: positive, negative, or
   neutral. Reply with just the label."
4   temperature: 0.0
5 }
6
7 fn main() {
8   label, _ = Classifier.ask("I love this product!")

```

```
9     io.println(label) // "positive"  
10 }
```

13.9.2 Research Agent with Tools

```
1  agent Researcher {  
2      provider: anthropic  
3      system: "You are a thorough researcher. Always cite your  
4              sources."  
5      tools: [search, read_url]  
6      temperature: 0.2  
7      max_steps: 15  
8  }  
9  fn main() {  
10     answer, err = Researcher.ask("What are the latest advances  
11     in fusion energy?")  
12     if err != nil {  
13         io.println("Error: " + err.message)  
14         return  
15     }  
16     io.println(answer)  
17 }
```

13.9.3 Conversational Assistant

```
1  agent Tutor {  
2      provider: anthropic  
3      system: "You are a patient math tutor. Explain concepts  
4              step by step."  
5      memory: conversation(max_turns: 30)  
6      temperature: 0.5  
7  }  
8  fn main() {  
9      session = "student-001"  
10  
11     reply1, _ = Tutor.ask("What is a derivative?", session:  
12     session)  
13     io.println(reply1)  
14  
15     reply2, _ = Tutor.ask("Can you give me an example?",  
16     session: session)  
17     io.println(reply2)  
18  
19     reply3, _ = Tutor.ask("Now explain the chain rule",  
20     session: session)  
21     io.println(reply3)  
22     // Tutor remembers the full conversation context  
23 }
```


13.10 Agent Grammar

```
1 agent_decl      = "agent" identifier "{" { agent_field } "}" ;
2
3 agent_field     = identifier ":" agent_value ;
4
5 agent_value     = expression
6                 | string_lit
7                 | triple_string
8                 | "[" ident_list "]"
9                 | memory_value ;
10
11 memory_value    = "conversation" "(" [ arguments ] ")"
12                 | "summary" "(" [ arguments ] ")"
13                 | "none" ;
```


Chapter 14

Workflows

Workflows are the central abstraction in HAIRA. A workflow is a composable function with a trigger, typed parameters, and a typed return value. Workflows replace both n8n-style visual automations and LangGraph-style agent graphs.

14.1 Workflow Declaration

A workflow is a function preceded by a trigger decorator:

```
1 @trigger_type("config")
2 workflow WorkflowName(param: Type, param: Type) -> ReturnType {
3     // body
4 }
```

Workflows without a trigger decorator can be called as sub-workflows but cannot be triggered externally.

14.2 Triggers

Triggers define how a workflow is invoked. They use the @ decorator syntax.

14.2.1 @webhook — HTTP Endpoint

```
1 @webhook("/summarize")
2 workflow Summarizer(url: string) -> Summary {
3     content = fetch_page(url)
4     summary = Researcher.ask("Summarize: {content}")
5     return Summary{ text: summary, word_count: summary.len() }
6 }
```

The workflow parameters become the JSON request body. The return type becomes the JSON response body.

```
$ curl -X POST http://localhost:8080/summarize \
    -d '{"url": "https://example.com/article"}'

{"text": "...", "word_count": 150}
```

Options:

```

1 @webhook("/path") // POST (default)
2 @webhook("/path", method: "GET") // GET
3 @webhook("/path", method: "PUT") // PUT

```

14.2.2 @websocket — Bidirectional Streaming

```

1 @websocket("/chat")
2 workflow Chat(message: string, session_id: string) ->
3   stream<string> {
4     return Assistant.stream(message, session: session_id)
5   }

```

WebSocket workflows receive messages and can return `stream<string>` for real-time streaming responses.

14.2.3 @cron — Scheduled Execution

```

1 @cron("0 9 * * *") // every day at 9 AM
2 workflow DailyReport() -> Report {
3   data = fetch_metrics()
4   report: Report, _ = Analyst.run(AnalyzeRequest{ data: data
5   })
6   send_email("team@company.com", "Daily Report",
7   report.summary)
8   return report
9 }

```

Cron expressions follow standard cron syntax (minute, hour, day-of-month, month, day-of-week).

14.2.4 @event — Event-Driven

```

1 @event("order.created")
2 workflow ProcessOrder(order: Order) -> OrderResult {
3   validated = validate_order(order)
4   if validated.has_errors {
5     return OrderResult{ status: "rejected" }
6   }
7   charge_payment(order)
8   return OrderResult{ status: "confirmed" }
9 }

```

Events are published using `haira.emit()`:

```

1 haira.emit("order.created", Order{ id: "123", total: 99.99 })

```

14.2.5 @manual — CLI Only

```

1 @manual

```

```

2 workflow Migrate(version: string) -> { status: string } {
3     // only invoked from CLI or main()
4     run_migrations(version)
5     return { status: "done" }
6 }

```

```
$ haira run Migrate --input '{"version": "v2"}'
```

14.2.6 No Trigger (Sub-Workflow)

Workflows without a trigger can only be called from other workflows or from `main()`:

```

1 workflow EnrichLead(email: string) -> EnrichedLead {
2     company = lookup_company(email)
3     social = lookup_social(email)
4     return EnrichedLead{ email: email, company: company,
5         social: social }
5 }

```

14.3 Sequential Steps

The workflow body is sequential by default. Each line can use the results of previous lines:

```

1 @manual
2 workflow ArticlePipeline(topic: string) -> Article {
3     // Step 1: Research
4     research: ResearchResult, err =
5         Researcher.run(ResearchRequest{ topic: topic })
6     if err != nil { return Article{}, err }
7
8     // Step 2: Write draft
9     draft: Article, err = Writer.run(WriteRequest{
10         content: research.summary,
11         sources: research.sources
12     })
13     if err != nil { return Article{}, err }
14     return draft
15 }

```

14.4 Named Steps

The **step** keyword creates named, observable regions within a workflow body. Each step has a string label and a block of statements. The runtime automatically emits structured telemetry for every step—start time, end time, duration, and success/failure status—giving production-grade observability with zero manual logging.

14.4.1 Syntax

```

1 step "Step Name" {

```

```

2      // statements
3  }

```

A **step** is only valid inside a **workflow** body (including inside **spawn** blocks within workflows). The step name must be a string literal.

14.4.2 Transparent Scope

Steps are *not* functions—they are labeled regions of the enclosing workflow. Variables assigned inside a step are visible to all subsequent steps and to the rest of the workflow body. A **return** inside a step returns from the enclosing workflow.

```

1  @webhook("/api/process")
2  workflow ProcessData(input: string) -> { status: string, count:
3      int } {
4      step "Parse input" {
5          records = parse_csv(input)?
6          count = len(records)
7      }
8      step "Validate" {
9          // 'records' and 'count' are visible here
10         errors = validate_records(records)
11         if len(errors) > 0 {
12             // returns from ProcessData, not just the step
13             return { status: "invalid", count: 0 }
14         }
15     }
16     step "Save to database" {
17         save_all(records)?
18     }
19     return { status: "ok", count: count }
20 }
21 }
22 }

```

This matches the n8n/Zapier mental model: every node's output flows to downstream nodes.

14.4.3 Automatic Telemetry

The runtime emits structured log events for each step without any explicit logging code:

```

[ProcessData] step:start  "Parse input"
[ProcessData] step:end    "Parse input"      duration=42ms
              status=success
[ProcessData] step:start  "Validate"
[ProcessData] step:end    "Validate"          duration=3ms
              status=success
[ProcessData] step:start  "Save to database"
[ProcessData] step:end    "Save to database"   duration=156ms
              status=success

```

If a step fails (via error propagation or early return), the telemetry reflects it:

```

[ProcessData] step:start  "Validate"

```

```
[ProcessData] step:end      "Validate"          duration=3ms
               status=failed  error="Validation failed"
```

This gives operations teams per-step visibility in production:

- Which step is slow (duration)
- Which step failed (status + error message)
- Dashboards and alerts per workflow per step

14.4.4 Parallel Steps

Use existing **spawn** blocks to run steps concurrently—no new syntax needed:

```
1 workflow ProcessOrder(order: Order) -> Result {
2   step "Validate order" {
3     validate(order)?
4   }
5
6   // These run in parallel
7   spawn {
8     step "Check inventory" {
9       stock = check_stock(order.items)
10    }
11    step "Verify payment" {
12      payment = verify_payment(order.card)
13    }
14  }
15
16  // Back to sequential --- stock, payment available after
   // spawn completes
17  step "Fulfill" {
18    fulfill(order, stock, payment)
19  }
20
21  return Result{ status: "fulfilled" }
22 }
```

Telemetry shows overlapping timestamps for parallel steps:

```
[ProcessOrder] step:start  "Check inventory"
[ProcessOrder] step:start  "Verify payment"
[ProcessOrder] step:end    "Check inventory"    duration=120ms
               status=success
[ProcessOrder] step:end    "Verify payment"    duration=200ms
               status=success
```

Variables from parallel steps flow to workflow scope after the **spawn** block completes (spawn is a synchronization point).

14.4.5 Real-World Example

A data pipeline with multiple stages, each automatically traced:

```
1 @webhook("/api/upload-config")
2 workflow UploadConfig(file_path: string) -> { status: string,
   message: string } {
```

```

3     filename = file_path | string.split("/") | last()
4
5     step "Open Excel" {
6         wb = excel.open(file_path)?
7         defer wb.close()
8         index_rows = wb.read_sheet("index")?
9     }
10
11    step "Extract table data" {
12        table_data = {}
13        for row in index_rows {
14            sheet = wb.read_sheet(row["sheet_name"])?
15            table_data[row["table_name"]] = sheet
16        }
17    }
18
19    step "Validate" {
20        validation = validate_data(table_data)
21        if !validation.is_valid {
22            return { status: "error", message: "Validation
23                failed: ${validation.errors}" }
24        }
25    }
26
27    step "Push to GitLab" {
28        branch = "feature/${filename}"
29        create_branch(branch)?
30        for name, data in table_data {
31            sql = generate_sql(name, data)
32            commit_file(branch, "${name}.sql", sql)?
33        }
34        create_merge_request(branch, "Update ${filename}")?
35    }
36
37    step "Wait for CI/CD" {
38        poll_pipeline(branch)?
39    }
40
41    return { status: "success", message: "Deployed ${filename}"
42    }
43 }

```

Without **step**, this workflow would need 10+ manual `io.println` calls to achieve the same observability. With **step**, every stage is automatically traced in production.

14.5 Parallel Execution

Use **spawn** blocks to run steps in parallel:

```

1 @manual
2 workflow ReviewArticle(article: Article) -> [Review] {
3     reviews = spawn {
4         Reviewer.run(ReviewRequest{ article: article, focus:
5             "accuracy" })
6         Reviewer.run(ReviewRequest{ article: article, focus:
7             "clarity" })
8         Reviewer.run(ReviewRequest{ article: article, focus:

```



```

        "engagement" })
7     }
8     return reviews
9 }

```

A **spawn** block runs all statements concurrently and returns a list of results when all complete. This is equivalent to `Promise.all()` in JavaScript.

14.6 Branching

Standard **if/else** and **match** work inside workflows:

```

1  @event("order.created")
2  workflow ProcessOrder(order: Order) -> OrderResult {
3      validated = validate_order(order)
4
5      if validated.has_errors {
6          send_slack("#alerts", "Order {order.id} failed")
7          return OrderResult{ status: "rejected" }
8      }
9
10     stock = check_stock(validated.items)
11
12     if stock.available {
13         charge_payment(order)
14         send_email(order.customer_email, "Order confirmed!")
15         return OrderResult{ status: "confirmed" }
16     } else {
17         send_email(order.customer_email, "Items out of stock")
18         return OrderResult{ status: "backordered" }
19     }
20 }

```

14.7 Loops

```

1  @manual
2  workflow ProcessBatch(urls: [string]) -> [string] {
3      results = for url in urls {
4          content = fetch_page(url)
5          Researcher.ask("Summarize: {content}")
6      }
7      return results
8  }

```

Loops inside workflows are sequential by default. For parallel iteration, use **spawn**:

```

1  @manual
2  workflow ProcessBatchParallel(urls: [string]) -> [string] {
3      results = spawn {
4          for url in urls {
5              fetch_page(url) | Researcher.ask("Summarize: {_}")
6          }
7      }

```

```

8     return results
9 }

```

14.8 Sub-Workflows

Workflows can call other workflows like functions:

```

1  workflow ScoreLead(company: Company, social: SocialProfile) ->
    ScoreResult {
2      score: ScoreResult, err = Analyst.run(ScoreRequest{
3          company: company,
4          social: social
5      })
6      return score
7  }
8
9  workflow EnrichLead(email: string) -> EnrichedLead {
10     company = lookup_company(email)
11     social = lookup_social(email)
12     score = ScoreLead(company, social)
13     return EnrichedLead{ email: email, company: company, score:
        score.value }
14 }
15
16 @webhook("/new-lead")
17 workflow SalesAutomation(email: string) -> { result: string } {
18     lead = EnrichLead(email)
19
20     if lead.score > 0.8 {
21         send_slack("#sales", "Hot lead: {email}")
22     }
23
24     return { result: "processed" }
25 }

```

Sub-workflows compose naturally. The compiler validates type compatibility at each call site.

14.9 Error Handling

Workflows use standard HAIRA error handling:

```

1  @manual
2  workflow SafeProcess(data: string) -> { result: string } {
3      result, err = risky_operation(data)
4      if err != nil {
5          send_slack("#errors", "Failed: {err.message}")
6          result = fallback_operation(data)
7      }
8      return { result: result }
9  }

```

14.10 Running Workflows

14.10.1 From main()

```

1 fn main() {
2     result, err = ArticlePipeline("AI agents")
3     if err != nil {
4         io.println("Failed: " + err.message)
5         return
6     }
7     io.println(result.title)
8 }

```

14.10.2 Server Mode

Start a server that listens for all triggered workflows:

```

1 fn main() {
2     server = http.Server([
3         Summarizer,           // POST /summarize
4         Chat,                 // WebSocket /chat
5         ProcessOrder,         // on order.created events
6         DailyReport,          // cron 0 9 * * *
7     ])
8
9     io.println("Haira server running on :8080")
10    server.listen(8080)
11 }

```

14.10.3 CLI

```

# Run a workflow directly
$ haira run ArticlePipeline --input '{"topic": "AI agents"}'

# Start server mode
$ haira serve main.haira --port 8080

# Run with environment
$ ANTHROPIC_API_KEY=sk-... haira serve main.haira

```

14.11 Orchestration Patterns

HAIRA workflows compose naturally into common orchestration patterns using existing language primitives. No special keywords or frameworks are needed.

14.11.1 Sequential Chain

Pipe the output of one agent into the next:

```

1 workflow Analyze(text: string) -> { result: string } {
2     summary, err = Summarizer.ask(text)

```

```

3     sentiment, err = SentimentAnalyzer.ask(summary)
4     report, err = ReportWriter.ask(sentiment)
5     return { result: report }
6 }

```

14.11.2 Parallel Fan-Out

Run multiple agents concurrently and collect results:

```

1 workflow Research(topic: string) -> { synthesis: string } {
2     results = spawn {
3         WebSearcher.ask(topic)
4         PaperAnalyzer.ask(topic)
5         NewsScanner.ask(topic)
6     }
7     synthesis, err = Synthesizer.ask(results)
8     return { synthesis: synthesis }
9 }

```

14.11.3 Router / Dispatcher

One agent classifies intent, then **match** routes to a specialist:

```

1 workflow Support(message: string, session_id: string) -> {
2     reply: string } {
3     intent, err = Router.ask(message)
4     reply, err = match intent {
5         "billing" => BillingAgent.ask(message, session:
6             session_id)
7         "technical" => TechAgent.ask(message, session:
8             session_id)
9         - => GeneralAgent.ask(message, session:
10             session_id)
11     }
12     return { reply: reply }
13 }

```

14.11.4 Handoff

Agents delegate to other agents automatically via the **handoffs** field. See Chapter 13, Handoffs section.

14.11.5 Iterative Refinement

An agent produces output, a critic evaluates it, and the loop repeats until approved:

```

1 workflow Refine(draft: string) -> { result: string } {
2     current = draft
3     for i in 0..3 {
4         critique, err = Critic.ask(current)
5         if critique == "approved" { break }
6         current, err = Writer.ask("Improve based on:
7             ${critique}\n\n${current}")
8     }
9     return { result: current }
10 }

```

```

7     }
8     return { result: current }
9 }

```

14.11.6 Supervisor

A planner creates a task list, workers execute each step, and a supervisor evaluates results:

```

1 workflow ComplexTask(goal: string) -> { result: string } {
2     plan, err = Planner.ask(goal)
3
4     for step in plan.steps {
5         worker = match step.type {
6             "code"      => Coder
7             "research"  => Researcher
8             "review"    => Reviewer
9         }
10        result, err = worker.ask(step.instruction)
11
12        evaluation, err = Supervisor.ask("Evaluate: ${result}")
13        if evaluation == "retry" {
14            result, err = worker.ask("Try again:
15                                     ${step.instruction}")
16        }
17    }
18    return { result: result }
19 }

```

14.11.7 Voting / Consensus

Multiple agents answer the same question, then an aggregator picks the best:

```

1 workflow Decide(question: string) -> { answer: string } {
2     votes = spawn {
3         ExpertA.ask(question)
4         ExpertB.ask(question)
5         ExpertC.ask(question)
6     }
7     consensus, err = Aggregator.ask(votes)
8     return { answer: consensus }
9 }

```

14.11.8 Human-in-the-Loop

For workflows that require human approval, split into two workflows connected by shared state. The first workflow does the AI work and sends a notification. The second workflow handles the human's response:

```

1 tool send_email(to: string, subject: string, body: string) ->
2     string {
3     """Send an email notification."""
4 }

```

```

3     resp, err =
4         http.post("https://api.sendgrid.com/v3/mail/send", {
5             to: to, subject: subject, body: body
6         })
7     return resp.body
8 }
9 @webhook("/api/publish")
10 workflow SubmitForReview(content: string) -> { review_id:
11     string } {
12     edited, err = Editor.ask(content)
13     review_id = save_review(edited)
14
15     send_email(
16         to: "admin@company.com",
17         subject: "Review needed",
18         body: "Approve:
19             https://app.com/api/review?id=${review_id}&action=approve"
20     )
21
22     return { review_id: review_id }
23 }
24 @webhook("/api/review")
25 workflow HandleReview(id: string, action: string) -> { status:
26     string } {
27     if action == "approve" {
28         content = get_review(id)
29         publish(content)
30         return { status: "published" }
31     }
32     return { status: "rejected" }
33 }

```

No new primitives are needed. The “event” is just another HTTP request hitting another workflow. Notifications go out via tools (email, Slack, etc.), and the callback URL points back to a `@webhook` workflow.

14.11.9 Pattern Summary

Pattern	Primitives Used	New Syntax?
Sequential chain	statements	No
Named steps	step blocks	Keyword
Parallel fan-out	spawn	No
Router / dispatcher	match + agents	No
Handoff	handoffs agent field	Field only
Iterative refinement	for + break	No
Supervisor	loop + match + agents	No
Voting / consensus	spawn + aggregation	No
Human-in-the-loop	two workflows + tools	No

Table 14.1: Orchestration patterns and their primitives

14.12 Workflow Grammar

```
1 workflow_decl = { decorator } "workflow" identifier
2               "(" [ params ] ")" [ "->" type ] block ;
3
4 decorator     = "@" identifier [ "(" [ arguments ] ")" ] ;
5
6 spawn_block   = "spawn" "{" { statement } "}" ;
7
8 step_stmt     = "step" string_literal block ;
```


Part IV

Implementation

Chapter 15

Complete Grammar

This chapter provides the complete formal grammar for HAIRA in Extended Backus-Naur Form (EBNF), including all core language constructs and agentic extensions (providers, tools, agents, and workflows).

15.1 Notation

Notation	Meaning
=	Definition
	Alternative
[...]	Optional (0 or 1)
{ ... }	Repetition (0 or more)
(...)	Grouping
"..."	Terminal string
;	End of production

Table 15.1: EBNF notation

15.2 Lexical Grammar

15.2.1 Characters

```
1 letter      = "a".."z" | "A".."Z" | "_" ;
2 digit       = "0".."9" ;
3 hex_digit   = digit | "a".."f" | "A".."F" ;
4 octal_digit = "0".."7" ;
5 binary_digit = "0" | "1" ;
6 any_char    = (* any Unicode character *) ;
```

15.2.2 Whitespace and Comments

```
1 whitespace = " " | "\t" | "\r" ;
2 newline    = "\n" ;
3 line_comment = "//" { any_char } newline ;
4 block_comment = "/*" { any_char | block_comment } "*/" ;
```

```
5 doc_comment    = "///" { any_char } newline ;
```

Note

Block comments nest: `/* outer /* inner */ still outer */` is valid. Doc comments (`///`) attach to the immediately following declaration and are preserved in compiled metadata.

15.2.3 Identifiers

```
1 identifier      = letter { letter | digit } ;
```

15.2.4 Keywords

```
1 keyword         = "agent" | "and" | "as" | "break" | "catch"
2                 | "chan" | "const" | "continue" | "defer"
3                 | "else" | "enum" | "false" | "fn" | "for"
4                 | "from" | "if" | "impl" | "import" | "in"
5                 | "match" | "nil" | "not" | "or" | "provider"
6                 | "return" | "select" | "spawn" | "step" |
7                 | "struct"
8                 | "tool" | "true" | "try" | "type" | "unsafe"
9                 | "where" | "while" | "workflow" ;
```

Note

The keywords **agent**, **provider**, **tool**, and **workflow** are reserved for agentic declarations. The keyword **unsafe** is reserved for future use. All keywords are case-sensitive.

15.2.5 Literals

```
1 integer_lit     = decimal_lit | hex_lit | octal_lit | binary_lit ;
2 decimal_lit    = digit { digit | "_" } ;
3 hex_lit        = "0" ("x"|"X") hex_digit { hex_digit | "_" } ;
4 octal_lit      = "0" ("o"|"O") octal_digit { octal_digit | "_" }
5               ;
6 binary_lit     = "0" ("b"|"B") binary_digit { binary_digit | "_"
7               } ;
8
9 float_lit      = decimal_lit "." decimal_lit [ exponent ]
10               | decimal_lit exponent ;
11 exponent       = ("e"|"E") ["+"|"-"] decimal_lit ;
12
13 string_lit     = ''' { string_char } '''
14               | "r" ''' { raw_char } '''
15               | triple_string ;
16 triple_string  = '"""' { any_char } '"""' ;
17 string_char    = any_char - ('"' | "'" | "\n" | newline)
18               | escape_seq ;
19 escape_seq     = "\" ( "n" | "r" | "t" | "\" | "'" | "0"
```

```

18         | "x" hex_digit hex_digit
19         | "u" "{" hex_digit { hex_digit } "}" ) ;
20 raw_char    = any_char - ',' ;
21
22 bool_lit    = "true" | "false" ;
23 nil_lit     = "nil" ;

```

Note

Triple-quoted strings ("\"...\"") preserve newlines and leading indentation. They are used for tool descriptions, agent system prompts, and multi-line string literals. The common leading whitespace is stripped at compile time.

15.2.6 Operators

```

1 operator    = arith_op | comp_op | logic_op | bitwise_op
2             | shift_op | assign_op | other_op ;
3
4 arith_op    = "+" | "-" | "*" | "/" | "%" ;
5 comp_op     = "==" | "!=" | "<" | ">" | "<=" | ">=" ;
6 logic_op    = "&&" | "||" | "!" ;
7 bitwise_op  = "&" | "|" | "^" | "~" ;
8 shift_op    = "<<" | ">>" ;
9 assign_op   = "=" | "+=" | "-=" | "*=" | "/="
10            | "&=" | "|=" | "^=" | "<<=" | ">>=" | ">>>=" ;
11 other_op    = "|>" | ".." | "..=" | "<-" | "->" | "=>" | "?"
              | "@" ;

```

Note

The `|>` operator is the pipe operator for function composition. The `@` symbol is used for decorator syntax on workflow declarations. The `<-` operator is used for channel send and receive operations.

15.2.7 Delimiters

```

1 delimiter   = "(" | ")" | "[" | "]" | "{" | "}"
2             | "," | ":" | "." | ";" | "#" ;

```

15.3 Syntactic Grammar

15.3.1 Program Structure

```

1 program     = { import_stmt } { top_level } ;
2
3 top_level   = fn_decl
4             | struct_decl
5             | enum_decl
6             | type_alias
7             | const_decl

```

```

8         | impl_block
9         | constraint_decl
10        | provider_decl
11        | tool_decl
12        | agent_decl
13        | workflow_decl ;

```

15.3.2 Imports

```

1 import_stmt  = "import" import_spec ;
2
3 import_spec  = string_lit
4               | identifier "from" string_lit
5               | "{" ident_list "}" "from" string_lit
6               | "*" "from" string_lit ;
7
8 ident_list   = identifier { "," identifier } ;

```

15.3.3 Core Declarations

```

1 fn_decl      = [ attributes ] "fn" [ receiver ] identifier
2               [ generic_params ] "(" [ params ] ")"
3               [ "->" type ] [ where_clause ] block ;
4
5 receiver     = "(" identifier ":" [ "*" ] type ")" ;
6
7 generic_params = "<" generic_param { "," generic_param } ">" ;
8
9 generic_param = identifier [ ":" constraint_list ] ;
10
11 constraint_list = identifier { "+" identifier } ;
12
13 where_clause  = "where" where_bound { "," where_bound } ;
14
15 where_bound   = identifier ":" constraint_list ;
16
17 params        = param { "," param } ;
18
19 param         = identifier ":" [ "..." ] type [ "=" expression
20               ] ;
21
22 struct_decl   = [ attributes ] "struct" identifier
23               [ generic_params ] "{ { field_decl } }" ;
24
25 field_decl    = [ attributes ] identifier ":" type ;
26
27 enum_decl     = [ attributes ] "enum" identifier
28               [ generic_params ] "{ enum_variants }" ;
29
30 enum_variants = enum_variant { "," enum_variant } [ "," ] ;
31
32 enum_variant  = identifier [ "(" type_list ")" ] ;

```

```

33 type_alias      = "type" identifier [ generic_params ] "=" type ;
34
35 const_decl      = "const" identifier [ ":" type ] "=" expression ;
36
37 impl_block      = "impl" [ generic_params ] identifier
38                  "for" type [ where_clause ]
39                  "{" { fn_decl } "}" ;
40
41 constraint_decl  = "constraint" identifier [ generic_params ]
42                  "{" { fn_signature } "}" ;
43
44 fn_signature     = "fn" identifier "(" [ type_params ] ")" [ "->"
45                  type ] ;
46
47 type_params      = type { "," type } ;

```

15.3.4 Attributes

Attributes use the `#[...]` syntax and can be applied to functions, structs, enums, and fields. They are distinct from the `@` decorator syntax used for workflow triggers.

```

1 attributes      = { attribute } ;
2
3 attribute       = "#[" identifier [ "(" attr_args ")" ] "]" ;
4
5 attr_args       = attr_arg { "," attr_arg } ;
6
7 attr_arg        = identifier [ "=" literal ] ;

```

15.3.5 Decorators

Decorators use the `@` syntax and are used exclusively for workflow trigger declarations.

```

1 decorator       = "@" identifier [ "(" [ arguments ] ")" ] ;

```

15.3.6 Provider Declarations

Providers configure LLM backends. They are declared at the top level and referenced by name in agent declarations.

```

1 provider_decl   = "provider" identifier "{" { provider_field }
2                  "}" ;
3
4 provider_field  = identifier ":" expression ;

```

15.3.7 Tool Declarations

Tools are typed functions with a mandatory triple-quoted description. The compiler auto-generates JSON schemas from the type signature.

```

1 tool_decl       = "tool" identifier "(" [ params ] ")" [ "->"
2                  type ]

```

```

2         "{" triple_string { statement } "}" ;
3
4 triple_string = '"""' { any_char } '"""' ;

```

Note

The `triple_string` must be the first element of the tool body. The compiler emits an error if a tool is missing its description.

15.3.8 Agent Declarations

Agents are LLM-powered entities with a model, system prompt, optional tools, and optional memory.

```

1 agent_decl    = "agent" identifier "{" { agent_field } "}" ;
2
3 agent_field   = identifier ":" agent_value ;
4
5 agent_value   = expression
6               | string_lit
7               | triple_string
8               | "[" ident_list "]"
9               | memory_value ;
10
11 memory_value  = "conversation" "(" [ arguments ] ")"
12               | "summary" "(" [ arguments ] ")"
13               | "none" ;

```

Note

Agent fields include `model` (required), `system` (required), and optional fields such as `tools`, `temperature`, `max_tokens`, `max_steps`, and `memory`. The `model` field references a provider by name.

15.3.9 Workflow Declarations

Workflows are composable functions with optional trigger decorators. A workflow without a decorator can only be called as a sub-workflow.

```

1 workflow_decl = { decorator } "workflow" identifier
2               "(" [ params ] ")" [ "->" type ] block ;

```

15.3.10 Types

```

1 type          = simple_type
2               | array_type
3               | map_type
4               | tuple_type
5               | option_type
6               | fn_type
7               | generic_type
8               | stream_type

```



```

 9         | pointer_type
10         | union_type
11         | channel_type ;
12
13 simple_type = "int" | "i8" | "i16" | "i32" | "i64"
14             | "u8" | "u16" | "u32" | "u64"
15             | "float" | "f32" | "f64"
16             | "bool" | "string"
17             | identifier ;
18
19 array_type  = "[" "]" type ;
20
21 map_type    = "[" type ":" type "]" ;
22
23 tuple_type  = "(" type { "," type } ")" ;
24
25 option_type = type "?" ;
26
27 fn_type     = "fn" "(" [ type_list ] ")" [ "->" type ] ;
28
29 type_list   = type { "," type } ;
30
31 generic_type = identifier "<" type_list ">" ;
32
33 stream_type  = "stream" "<" type ">" ;
34
35 pointer_type = "*" type ;
36
37 union_type   = type { "|" type } ;
38
39 channel_type = "chan" "<" type ">" ;

```

Note

The `stream<T>` type represents a lazy, asynchronous, single-use sequence of values. It is distinct from `chan<T>`, which is a bidirectional communication channel between concurrent tasks. Streams are typically produced by agent `.stream()` calls and consumed with `for` loops.

15.3.11 Statements

```

1 statement = var_decl
2           | assignment
3           | expr_stmt
4           | if_stmt
5           | for_stmt
6           | while_stmt
7           | match_stmt
8           | return_stmt
9           | break_stmt
10          | continue_stmt
11          | defer_stmt
12          | spawn_stmt
13          | step_stmt
14          | send_stmt

```

```

15         | select_stmt
16         | block ;
17
18 var_decl    = ident_list [ ":" type ] "=" expr_list ;
19
20 ident_list  = identifier { "," identifier } ;
21
22 expr_list   = expression { "," expression } ;
23
24 assignment  = lvalue_list assign_op expr_list ;
25
26 lvalue_list = lvalue { "," lvalue } ;
27
28 lvalue      = identifier
29             | lvalue "." identifier
30             | lvalue "[" expression "]" ;
31
32 expr_stmt   = expression ;
33
34 if_stmt     = "if" [ simple_stmt ";" ] expression block
35             [ "else" ( if_stmt | block ) ] ;
36
37 simple_stmt = var_decl | assignment | expr_stmt ;
38
39 for_stmt    = "for" [ identifier "," ] identifier "in"
40             expression block ;
41
42 while_stmt  = "while" expression block ;
43
44 match_stmt  = "match" expression "{" { match_arm } "}" ;
45
46 match_arm   = pattern [ "if" expression ] "=>" ( expression |
47             block ) ;
48
49 return_stmt = "return" [ expr_list ] ;
50
51 break_stmt  = "break" [ identifier ] ;
52
53 continue_stmt = "continue" [ identifier ] ;
54
55 defer_stmt  = "defer" statement ;
56
57 spawn_stmt  = "spawn" ( block | fn_call ) ;
58
59 step_stmt   = "step" string_lit block ;
60
61 send_stmt   = expression "<-" expression ;
62
63 select_stmt = "select" "{" { select_case } [ default_case ]
64             "}" ;
65
66 select_case = pattern "=" "<-" expression "=>" ( expression |
67             block )
68             | expression "<-" expression "=>" ( expression |
69             block ) ;
70
71 default_case = "default" "=>" ( expression | block ) ;

```

```

68     block      = "{" { statement } "}" ;
69
70     fn_call    = identifier "(" [ arguments ] ")"
71                 | identifier "." identifier "(" [ arguments ] ")"
                    ;

```

Note

Inside a workflow, a **spawn** block runs all its statements concurrently and returns a list of results when all complete. This is equivalent to `Promise.all()` in JavaScript. Outside a workflow, **spawn** launches a concurrent goroutine-style task. A **step** block is a named region with automatic telemetry; it is only valid inside **workflow** bodies. Variables assigned inside a step are visible in subsequent steps and the enclosing workflow scope.

15.3.12 Expressions

```

1 expression      = assign_expr ;
2
3 assign_expr     = or_expr [ ("=" | "+=" | "-=" | "*=" | "/="
4 | "&=" | "|=" | "^=" | "<<=" | ">>=" | ">>>=")
5 assign_expr ] ;
6
7 or_expr        = and_expr { ("or" | "||") and_expr } ;
8
9 and_expr       = equality { ("and" | "&&") equality } ;
10
11 equality       = comparison { ("==" | "!=") comparison } ;
12
13 comparison    = pipe { ("<" | ">" | "<=" | ">=") pipe } ;
14
15 pipe          = range { ">" range } ;
16
17 range         = bitwise_or [ (".." | "..=") bitwise_or ] ;
18
19 bitwise_or    = bitwise_xor { "|" bitwise_xor } ;
20
21 bitwise_xor   = bitwise_and { "^" bitwise_and } ;
22
23 bitwise_and   = shift { "&" shift } ;
24
25 shift        = additive { ("<<" | ">>") additive } ;
26
27 additive     = multiplicative { ("+" | "-") multiplicative } ;
28
29 multiplicative = unary { ("*" | "/" | "%") unary } ;
30
31 unary        = ("!" | "-" | "~" | "not") unary
32 | postfix ;
33
34 postfix     = primary { postfix_op } ;
35
36 postfix_op  = call
37 | index
38 | field

```

```

39         | optional_chain ;
40
41 call      = "(" [ arguments ] ")" ;
42
43 arguments = argument { "," argument } ;
44
45 argument  = [ identifier ":" ] expression ;
46
47 index     = "[" expression "]" ;
48
49 field     = "." identifier ;
50
51 optional_chain = "?" "." identifier ;
52
53 primary   = identifier
54             | literal
55             | "(" expression ")"
56             | array_lit
57             | map_lit
58             | struct_lit
59             | closure
60             | if_expr
61             | match_expr
62             | receive_expr
63             | block ;
64
65 literal   = integer_lit | float_lit | string_lit
66             | bool_lit | nil_lit ;
67
68 array_lit = "[" [ expr_list ] "]" ;
69
70 map_lit   = "[" ":" "]"
71             | "[" map_entry { "," map_entry } [ "," ] "]" ;
72
73 map_entry = expression ":" expression ;
74
75 struct_lit = identifier "{" [ field_inits ] "}";
76
77 field_inits = field_init { "," field_init } [ "," ] ;
78
79 field_init  = identifier [ ":" expression ] ;
80
81 closure     = "fn" "(" [ closure_params ] ")" [ "->" type ]
82             ( block | "{" expression }" ) ;
83
84 closure_params = closure_param { "," closure_param } ;
85
86 closure_param  = identifier [ ":" type ] ;
87
88 if_expr        = "if" expression block "else" ( if_expr | block
89             ) ;
90
91 match_expr     = "match" expression "{" { match_arm } "}";
92
93 receive_expr   = "<-" expression ;

```

15.3.13 Patterns

```

1 pattern      = literal_pat
2               | ident_pat
3               | wildcard_pat
4               | tuple_pat
5               | struct_pat
6               | enum_pat
7               | array_pat
8               | range_pat
9               | or_pat ;
10
11 literal_pat  = integer_lit | float_lit | string_lit | bool_lit
12               ;
13 ident_pat    = identifier ;
14
15 wildcard_pat = "_" ;
16
17 tuple_pat    = "(" pattern { "," pattern } ")" ;
18
19 struct_pat   = identifier "{" [ field_pats ] "}" ;
20
21 field_pats   = field_pat { "," field_pat } ;
22
23 field_pat    = identifier [ ":" pattern ] ;
24
25 enum_pat     = identifier "." identifier [ "(" [ pattern_list
26               ] ")" ] ;
27
28 pattern_list = pattern { "," pattern } ;
29
30 array_pat    = "[" [ pattern_list ] [ "," "..." [ identifier ]
31               ] "]" ;
32
33 range_pat    = literal ".." [ "=" ] literal ;
34
35 or_pat       = pattern { "|" pattern } ;

```

15.4 Grammar Summary

15.4.1 Entry Points

- **program** — Complete source file
- **expression** — Single expression (REPL)
- **statement** — Single statement
- **type** — Type annotation

15.4.2 Precedence (High to Low)

1. Postfix: `() [] . ?`.
2. Unary: `! - ~ not`

3. Multiplicative: `*` `/` `%`
4. Additive: `+` `-`
5. Shift: `<<` `>>` `>>>`
6. Bitwise AND: `&`
7. Bitwise XOR: `^`
8. Bitwise OR: `|`
9. Range: `...` `...=`
10. Pipe: `|>`
11. Comparison: `<` `>` `<=` `>=`
12. Equality: `==` `!=`
13. Logical AND: `and` `&&`
14. Logical OR: `or` `||`
15. Assignment: `=` `+=` `-=` `*=` `/=` `&=` `|=` `^=` `<=>` `>=>` `>>=`

15.4.3 Associativity

- Left-associative: All binary operators except assignment
- Right-associative: Assignment operators, unary operators
- Non-associative: Comparison operators, range operators

Chapter 16

Compiler Architecture

This chapter describes the architecture and implementation of the HAIRA compiler.

16.1 Overview

The HAIRA compiler is written in Go and transforms `.haira` source files into native binaries by generating Go code and invoking the Go toolchain. Agentic constructs — **provider**, **tool**, **agent**, and **workflow** with `@` decorator triggers — are first-class elements of the compilation pipeline.

The compiler pipeline:



The key design decision: HAIRA compiles *to Go*, not to machine code directly. This means every HAIRA program benefits from Go’s battle-tested runtime (goroutines, garbage collector, HTTP stack, TLS) without reimplementing any of it. The generated Go code is readable and can be inspected via `haira emit`.

Note

There is no “AI Phase” in the pipeline. All LLM interaction happens at *runtime* through agent method calls. The compiler validates, type-checks, and generates schemas for agentic constructs, then emits runtime library calls.

16.2 Lexer

The lexer is a hand-written scanner in `compiler/internal/lexer/` that converts UTF-8 source text into a stream of tokens.

16.2.1 Token Types

Tokens are defined in `compiler/internal/token/`:

```
// Literals
Int, Float, String, InterpolatedString, TripleQuoteString

// Keywords (agentic)
Provider, Tool, Agent, Workflow

// Keywords (core)
Fn, Struct, Enum, Type, Import, Export, From, Pub,
If, Else, For, While, Match, Return, Break, Continue,
True, False, None, Some, And, Or, Not, In,
Spawn, Select, Try, Catch, Defer, Errdefer,
Step, Test, Assert, Let, Const

// Keywords (reserved for future)
Trait, Impl, Async

// Operators
Plus, Minus, Star, Slash, Percent,
Eq, EqEq, Ne, Lt, Gt, Le, Ge,
PlusEq, MinusEq, StarEq, SlashEq, PercentEq,
Pipe, PipeArrow, Amp, AmpEq, Caret, CaretEq, Tilde,
Shl, ShlEq, Shr, ShrEq, PipeEq,
Arrow, FatArrow, DotDot, DotDotEq, Dot, Question, At, Ellipsis

// Delimiters
LParen, RParen, LBracket, RBracket, LBrace, RBrace, Comma, Colon
```

16.2.2 Lexer Features

- **Numeric literals** — Decimal, hex (`0xFF`), binary (`0b1010`), octal (`0o77`), with underscores (`1_000_000`)
- **String interpolation** — `${expr}` syntax with nested brace tracking
- **Triple-quoted strings** — `"""..."""` with automatic leading-whitespace dedenting

- **Nested block comments** — `/* ... /* nested */ ... */`
- **Escape sequences** —
 - `n`,
 - `t`,
 - `r`,
 - `,`
 - `"`,
 - `{`,
 - `}`

16.3 Parser

The parser in `compiler/internal/parser/` uses recursive descent for statements and declarations, combined with Pratt (precedence-climbing) parsing for expressions.

16.3.1 AST Nodes

The AST is defined in `compiler/internal/ast/` using Go interfaces as sum types. All nodes carry a `Span` for error reporting.

Top-level declarations:

- `TypeDef` — struct with typed fields and optional defaults
- `EnumDef` — enum with variants (optionally carrying fields)
- `TypeAlias` — `type Name = Type`
- `FunctionDef` — `fn name(params) -> Type { body }`
- `MethodDef` — `Type.method(params) -> Type { body }` with implicit `self`
- `ImportDecl` — four forms: basic, aliased, selective, glob
- `ExportDecl` — `export { Name1, Name2 }`
- `ProviderDecl` — `provider name { key: value }`
- `ToolDecl` — tool with params, return type, triple-quoted description, body
- `AgentDecl` — `agent name { key: value }`
- `WorkflowDecl` — workflow with optional trigger decorator, params, lifecycle hooks, body
- `TestDecl` — `test "name" { body }`

Expressions (27 types): Literals, identifiers, binary/unary ops, calls, method calls, field access, indexing, pipe, lambda, match, if-expressions, lists, maps, struct instances, ranges, error propagation (`?`), `orElse`, `some/none`, `spawn`, `select`, `async` blocks.

Statements (14 types): Assignment, `let/const`, `if/else`, `for-in`, `while`, `return`, `try/catch`, `defer/errdefer`, `match`, `break/continue`, `step`, `assert`, expression statements.

Patterns (6 types): Wildcard (`_`), literal, identifier, constructor, or-pattern (`A | B`), range (`1..5`).

Level	Operators
1	<code>orelse</code>
2	<code> ></code> (pipe)
3	<code>or</code> (logical)
4	<code>and</code> (logical)
5	<code> </code> (bitwise OR)
6	<code>^</code> (bitwise XOR)
7	<code>&</code> (bitwise AND)
8	<code>==, !=</code>
9	<code><, >, <=, >=, .., ..=</code>
10	<code>«, »</code>
11	<code>+, -</code>
12	<code>*, /, %</code>
13	<code>-, not, ~</code> (unary)
14	<code>()</code> , <code>[]</code> , <code>.</code> , <code>?</code> (postfix)

Table 16.1: Operator precedence (lowest to highest)

16.3.2 Operator Precedence

The Pratt parser implements 14 precedence levels (lowest to highest):

16.4 Resolver

The resolver in `compiler/internal/resolver/` handles multi-file module resolution:

1. Reads the main source file and extracts import declarations.
2. Resolves each import: standard library (built-in), project-local (`path/file.haira` or `path/mod.haira`), or external.
3. Recursively resolves transitive imports.
4. Detects circular dependencies (compile-time error).
5. Filters exports: only `pub` items and agentic declarations are visible to importers.
6. Produces a `Program` with a merged item list in dependency order.

```
// Resolution order for merged items:
// 1. Transitive module items (deepest dependencies first)
// 2. Direct import items (in source order)
// 3. Main file items
```

16.5 Checker

The type checker in `compiler/internal/checker/` performs semantic analysis in two passes:

Pass 1 (Registration): Walk all top-level items and register:

- Struct types with field types

- Enum types with variants
- Function signatures
- Provider, tool, agent, workflow declarations
- Global variables (inferred from initializers)

Pass 2 (Validation): Check function/method bodies:

- Type inference for expressions (literals, binary ops, calls, field access, indexing)
- Variable definition tracking (warn on undefined variables)
- Const immutability enforcement
- Agent field validation (model references a provider, tools exist, known fields)
- Enum exhaustiveness warnings in match expressions
- Return type checking
- Method `self` binding

16.5.1 Type System

```
Primitive types: int, float, string, bool, any, void, error
Compound types: [T] (list), {K: V} (map)
Named types:    struct, enum
Function types: (T, U) -> V
```

Note

Generic types are parse-accepted but erased to `any` in v1. Full generics are planned for a future release.

16.5.2 Agentic Validation

The checker validates agentic declarations:

- **Providers:** Warn on unknown fields. Valid fields: `model`, `api_key`, `backend`, `host`, `temperature`, `max_tokens`, `transport`, `command`, `args`, `url`.
- **Agents:** Require `model` field referencing a declared provider. Validate tool references. Check handoff targets exist. Warn on unknown fields.
- **Tools:** Enforce mandatory triple-quoted description (parser-level). Register parameters for schema generation.
- **Workflows:** Validate decorator triggers. Type-check body.

16.6 Code Generation

The code generator in `compiler/internal/codegen/` transforms the typed AST into Go source code. This is the core of HAIRA's compilation strategy: generate idiomatic Go that calls into the HAIRA runtime library.

16.6.1 Emission Strategy

Code is emitted in 8 ordered passes to satisfy Go's declaration requirements:

1. **Providers** — `var Claude = &haira.Provider{...}`
2. **Tools** — Tool functions + registration
3. **Agents** — Agent variables + initialization functions
4. **Workflows** — HTTP handler functions + SSE streaming handlers
5. **Top-level variables** — Global `var` declarations
6. **Types and enums** — Struct definitions, enum iota constants
7. **Methods** — Receiver methods (dot-attach syntax)
8. **Functions** — User functions (non-main)
9. **Main function** — Entry point with agent initialization in topological order

16.6.2 Agent Initialization Order

Agents with handoff targets must be initialized after their targets. The codegen performs a topological sort on the handoff graph to determine initialization order:

```

1 // Haira source:
2 agent Triage { handoffs: [Billing, Support] }
3 agent Billing { provider: Claude }
4 agent Support { provider: Claude }
5
6 // Generated init order:
7 // 1. Billing (no dependencies)
8 // 2. Support (no dependencies)
9 // 3. Triage (depends on Billing, Support)

```

16.6.3 Workflow Code Generation

Workflows with triggers generate HTTP handlers. Streaming workflows generate both a `StreamHandler` (SSE) and a `Handler` (JSON fallback):

```

// @webhook("/summarize")
// workflow Summarize(req: SumReq) -> Summary { ... }

// Generates:
// func SummarizeHandler(w, r)           -- JSON response
// func SummarizeStreamHandler(w, r) -- SSE streaming
// Route registration in main()

```

16.6.4 Tool Schema Generation

Each tool declaration generates a JSON schema from its parameter types:

Haira Type	JSON Schema Type
int	"integer"
float	"number"
bool	"boolean"
string	"string"
[T]	"array" with items
struct	"object" with properties

Table 16.2: Type-to-JSON-schema mapping

16.6.5 Import Detection

The codegen scans the AST to determine which Go imports are needed:

- `fmt` — string interpolation, print calls
- `encoding/json` — JSON marshal/unmarshal calls
- `sync` — spawn blocks (WaitGroup)
- `haira` — runtime library (agents, providers, tools, workflows, HTTP)

16.6.6 Go Compilation

After generating Go source code, the codegen:

1. Creates a temporary Go project directory with `go.mod`
2. Extracts the embedded runtime bundle (`bundle.tar.gz`) containing the haira Go package
3. Writes the generated `main.go`
4. Runs `go build` to produce a native binary
5. Moves the binary to the output path and cleans up

16.7 Runtime Library

The HAIRA runtime is a Go package (`package haira`) bundled into the compiler binary. It provides the execution environment for all agentic constructs.

16.7.1 Runtime Structure

The runtime is split into two layers:

- **Primitive** (`primitive/haira/`) — Core runtime: agent execution, provider management, tool registry, workflow orchestration, HTTP server, memory, I/O, strings, arrays, maps, math, JSON, regex, file system, time, environment variables.
- **Stdlib** (`stdlib/haira/`) — External integrations: PostgreSQL, SQLite, vector databases, Excel, Slack, GitHub, GitLab.

Both layers use `package haira` and are merged at bundle time into a single Go package, embedded in the compiler binary as `bundle.tar.gz`.

16.7.2 Provider Management

The runtime manages LLM provider connections:

- API key resolution from environment variables at startup
- Model routing based on provider configuration
- HTTP connection pooling to provider APIs
- Support for Anthropic, OpenAI, Ollama, and MCP (Model Context Protocol) transports

16.7.3 Agent Execution Loop

1. Assemble message payload (system prompt, conversation history, user message).
2. Send request to the LLM provider.
3. If response contains tool calls, execute each tool and collect results.
4. Send tool results back to the LLM.
5. Repeat steps 2–4 until final response or `max_steps` reached.
6. For `.run()` calls, parse JSON response into the annotated output type.
7. For `.stream()` calls, yield tokens incrementally via SSE.

16.7.4 Workflow Orchestration

- **HTTP server** — Built on Go's `net/http`. Handles `@webhook` (POST/GET/PUT/DELETE) and `@websocket` triggers.
- **Cron scheduler** — Executes `@cron`-triggered workflows on 5-field UNIX cron schedules.
- **Event bus** — In-process event dispatch for `@event`-triggered workflows via `haira.emit()`.
- **Step telemetry** — Named `step` blocks emit timing and status events for observability.
- **Run tracking** — Each workflow execution gets a unique run ID with step-level SSE streaming.

16.7.5 Session Management

- Each session maintains an isolated conversation history.
- `conversation(max_turns: N)` memory trims to the last N message pairs.
- Sessions are stored in-memory by default, with optional persistence via the Store interface.

16.8 Project Structure

The compiler is organized as a Go module:

```

compiler/
  main.go                # CLI entry point
  internal/
    token/               # Token type definitions
    lexer/               # Hand-written scanner
    ast/                 # AST node definitions
    parser/              # Recursive descent + Pratt
      parser
    resolver/            # Multi-file import
      resolution
    checker/             # Type checking + semantic
      analysis
    codegen/             # Go code generation
      codegen.go         # 8-pass emission
      orchestrator
    emitter.go           # Output buffer +
      indentation
    expressions.go       # Expression-to-Go
      conversion
    statements.go        # Statement emission
    agentic.go           #
      Agent/tool/provider/workflow emission
    stdlib.go            # Standard library
      function mapping
    types.go             # Haira-type-to-Go-type
      conversion
    project.go           # Temp project setup + go
      build
    test_codegen.go      # Test harness generation
  driver/               # Pipeline orchestration
  errors/               # Diagnostic system
    (pretty-printing)
  formatter/            # Source code formatter
  lsp/                  # Language server protocol
  manifest/             # package.haira handling
  runtime/              # Embedded runtime bundle
    (bundle.tar.gz)

primitive/haira/        # Core runtime (Go)
stdlib/haira/           # External integrations (Go)
ui/sdk/                 # Web component SDK
  (TypeScript)
ui/application/         # Default HTML templates

```

16.9 CLI Commands

```

haira build [file] [-o output] # Compile to native binary
haira run [file]               # Compile and execute
haira check [file]             # Type-check without codegen
haira emit [file]              # Show generated Go code
haira test [file]              # Run test blocks

```

Package	Responsibility
<code>token</code>	Token kind constants and Token struct
<code>lexer</code>	Tokenizes source text
<code>ast</code>	AST node types (Go interfaces as sum types)
<code>parser</code>	Builds AST from tokens
<code>resolver</code>	Resolves imports and produces merged program
<code>checker</code>	Type checking, inference, and semantic validation
<code>codegen</code>	Generates Go source code from typed AST
<code>driver</code>	Orchestrates the full compilation pipeline
<code>errors</code>	Diagnostic formatting with source context
<code>formatter</code>	Haira source code formatting
<code>lsp</code>	Language server (hover, completion, go-to-definition)
<code>manifest</code>	Package manifest (package.haira) support

Table 16.3: Compiler package responsibilities

```

haira fmt [file]           # Format source in-place
haira parse [file]         # Show the AST
haira lex [file]           # Show tokens
haira init                 # Create package.haira
    manifest
haira lsp                  # Start language server
    (stdio)
haira version              # Show version

```

If no file is specified, `haira` looks for `package.haira` in the current directory and uses its entry point.

16.10 Error Messages

The compiler produces contextual error messages with source location, underlines, and help text.

16.10.1 Missing Tool Description

```

error: tool is missing a description
--> tools.haira:5:1
  |
5 | tool search(query: string) -> [Result] {
  | ~~~~~
  |
  = help: add a triple-quoted description: """Search the web"""
  = note: tool descriptions are sent to the LLM and cannot be
         omitted

```

16.10.2 Agent Referencing Undefined Provider

```

error: undefined provider 'openai'
--> agents.haira:8:12
  |

```



```

8 |         provider: openai
  |         ~~~~~~ not found in this scope
  |
  = help: declare a provider with: provider openai { model:
    "gpt-4o" ... }

```

16.10.3 Undefined Variable

```

warning: undefined variable 'foo'
--> main.haira:10:5
  |
10 |         io.println(foo)
   |         ~~~
   = hint: did you mean to declare it?

```

16.10.4 Workflow with Invalid Trigger

```

error: unknown trigger '@daily'
--> workflows.haira:12:1
  |
12 | @daily
   | ~~~~~~ not a recognized trigger
   |
   = help: valid triggers: @webhook, @websocket, @cron, @event,
     @manual
   = help: for daily execution, use: @cron("0 0 * * *")

```

16.11 Runtime Bundling

The runtime is bundled into the compiler binary at build time:

1. make `bundle-runtime` copies `primitive/haira/*.go` and `stdlib/haira/*.go` into a staging directory.
2. The UI SDK is built (`bun build`) and its output included.
3. Everything is archived as `bundle.tar.gz`.
4. The archive is embedded in the compiler binary via Go's `go:embed` directive.
5. At compile time, the codegen extracts this bundle into a temporary Go project, writes the generated `main.go`, and runs `go build`.

This approach produces fully standalone binaries: the compiled HAIRA program contains everything it needs, including the full runtime library. No separate runtime installation is required.

Appendix A

Chatbot Patterns

One of the most common use cases for HAIRA is building chatbot backends. This chapter shows patterns from simple to advanced.

A.1 Simple REST Chatbot

A chatbot accessible via HTTP POST:

```
1 import "io"
2 import "http"
3
4 provider anthropic {
5     api_key: env("ANTHROPIC_API_KEY")
6     model: "claude-sonnet-4-20250514"
7 }
8
9 agent Assistant {
10     provider: anthropic
11     system: "You are a helpful assistant."
12     memory: conversation(max_turns: 50)
13     temperature: 0.7
14 }
15
16 @webhook("/chat")
17 workflow Chat(message: string, session_id: string) -> { reply:
18     string } {
19     reply, err = Assistant.ask(message, session: session_id)
20     if err != nil {
21         return { reply: "Sorry, something went wrong." }
22     }
23     return { reply: reply }
24 }
25
26 fn main() {
27     server = http.Server([Chat])
28     io.println("Chat server on :8080")
29     server.listen(8080)
30 }
```

Usage:

```
$ curl -X POST http://localhost:8080/chat \
  -d '{"message": "Hello!", "session_id": "user-123"}'
```

```

{"reply": "Hi there! How can I help you today?"}

$ curl -X POST http://localhost:8080/chat \
  -d '{"message": "What did I just say?", "session_id":
      "user-123"}'

{"reply": "You said \"Hello!\""}

```

The agent remembers the conversation because both calls share the same `session_id`.

A.2 Streaming WebSocket Chatbot

For real-time token-by-token responses over WebSocket:

```

1 @websocket("/chat")
2 workflow StreamingChat(message: string, session_id: string) ->
3   stream<string> {
4     return Assistant.stream(message, session: session_id)
5   }
6
7 fn main() {
8   server = http.Server([StreamingChat])
9   server.listen(8080)
10 }

```

The `stream<string>` return type tells HAIRA to stream each token over the WebSocket as it arrives. No buffering — the client sees tokens in real time.

A.3 Dual-Mode Chatbot

Expose the same agent over both REST (for simple integrations) and WebSocket (for real-time UIs):

```

1 agent Assistant {
2   provider: anthropic
3   system: "You are a helpful assistant."
4   memory: conversation(max_turns: 50)
5 }
6
7 @webhook("/chat")
8 workflow ChatREST(message: string, session_id: string) -> {
9   reply: string } {
10    reply, err = Assistant.ask(message, session: session_id)
11    return { reply: reply or "Error occurred." }
12 }
13
14 @websocket("/chat/stream")
15 workflow ChatStream(message: string, session_id: string) ->
16   stream<string> {
17     return Assistant.stream(message, session: session_id)
18   }
19
20 fn main() {
21   server = http.Server([ChatREST, ChatStream])
22 }

```

```

20     server.listen(8080)
21 }

```

Both endpoints share the same agent and session memory.

A.4 RAG Chatbot

A chatbot that searches a knowledge base before answering:

```

1  import "tools/postgres"
2  import "tools/http"
3
4  tool search_kb(query: string) -> [KBArticle] {
5      """Search the knowledge base for relevant articles"""
6
7      results, err = pg.query(
8          "SELECT * FROM articles WHERE content @@ to_tsquery($1)
9          LIMIT 5",
10         [query]
11     )
12     if err != nil { return [], err }
13     return results
14 }
15
16 tool lookup_order(order_id: string) -> Order {
17     """Look up a customer order by ID"""
18
19     order, err = pg.query_one(
20         "SELECT * FROM orders WHERE id = $1",
21         [order_id]
22     )
23     if err != nil { return Order{}, err }
24     return order
25 }
26
27 tool create_ticket(subject: string, body: string, priority:
28     string = "normal") -> Ticket {
29     """Create a support ticket"""
30
31     resp, err = http.post("https://helpdesk.api/tickets", {
32         subject: subject, body: body, priority: priority
33     })
34     if err != nil { return Ticket{}, err }
35     return json.decode(resp.body, Ticket)
36 }
37
38 agent SupportBot {
39     provider: anthropic
40     system: """
41         You are a customer support agent for Acme Corp.
42         Use the knowledge base to answer questions.
43         Look up orders when customers ask about purchases.
44         Create tickets for issues you cannot resolve directly.
45         Always be polite and helpful.
46     """
47     tools: [search_kb, lookup_order, create_ticket]
48     memory: conversation(max_turns: 100)

```

```

47     temperature: 0.3
48 }
49
50 @websocket("/support")
51 workflow CustomerSupport(message: string, session_id: string)
52   -> stream<string> {
53     return SupportBot.stream(message, session: session_id)
54 }

```

The agent automatically decides when to search the knowledge base, look up an order, or create a ticket based on the user's message.

A.5 Multi-Agent Routing

Route user messages to specialized agents based on intent:

```

1  agent Router {
2    provider: anthropic
3    system: ""
4      Classify the user message into one category: billing,
5      technical, general.
6      Reply with just the category name, nothing else.
7    ""
8    temperature: 0.0
9  }
10
11 agent BillingAgent {
12   provider: anthropic
13   system: "You handle billing questions. You can look up
14     invoices and process refunds."
15   tools: [lookup_invoice, process_refund]
16   memory: conversation(max_turns: 30)
17 }
18
19 agent TechAgent {
20   provider: anthropic
21   system: "You handle technical support. You can search docs
22     and check system status."
23   tools: [search_docs, check_status]
24   memory: conversation(max_turns: 30)
25 }
26
27 agent GeneralAgent {
28   provider: anthropic
29   system: "You handle general inquiries."
30   memory: conversation(max_turns: 30)
31 }
32
33 @websocket("/chat")
34 workflow SmartChat(message: string, session_id: string) ->
35   stream<string> {
36     category, _ = Router.ask(message)
37
38     match category {
39       "billing" => return BillingAgent.stream(message,
40         session: session_id)
41       "technical" => return TechAgent.stream(message,

```

```

        session: session_id)
37     -     => return GeneralAgent.stream(message,
        session: session_id)
38     }
39 }

```

A.6 Agent Handoffs

For complex support bots, use handoffs instead of explicit routing. The front desk agent decides when to delegate, and the runtime handles the transfer automatically:

```

1  agent FrontDesk {
2      provider: anthropic
3      system: """
4          You are a front desk agent. Greet users and help with
5          general questions.
6          If they have a billing question, hand off to
7          BillingAgent.
8          If they have a technical issue, hand off to TechAgent.
9          """
10     handoffs: [BillingAgent, TechAgent]
11     memory: conversation(max_turns: 10)
12 }
13
14 agent BillingAgent {
15     provider: anthropic
16     system: "You handle billing questions. You can look up
17         invoices and process refunds."
18     tools: [lookup_invoice, process_refund]
19     memory: conversation(max_turns: 30)
20 }
21
22 agent TechAgent {
23     provider: anthropic
24     system: "You handle technical support. You can search docs
25         and check system status."
26     tools: [search_docs, check_status]
27     memory: conversation(max_turns: 30)
28 }
29
30 @webhook("/chat")
31 workflow SmartSupport(message: string, session_id: string) -> {
32     reply: string {
33         // FrontDesk automatically hands off when appropriate
34         reply, err = FrontDesk.ask(message, session: session_id)
35         if err != nil { return { reply: "Sorry, something went
36             wrong." } }
37         return { reply: reply }
38     }
39 }
40
41 fn main() {
42     server = http.Server([SmartSupport])
43     io.println("Support server on :8080")
44     server.listen(8080)
45 }

```

Compare this to the explicit routing in the previous section. With handoffs, the workflow code stays simple — the agent decides when to delegate, not the workflow. This is ideal for conversational bots where the routing logic is nuanced and best handled by the LLM itself.

A.7 Chatbot with Structured Actions

Sometimes you want the chatbot to return structured data alongside text:

```

1  struct ChatResponse {
2      reply: string
3      actions: [Action]
4  }
5
6  struct Action {
7      type: string          // "navigate", "show_modal", "refresh"
8      target: string
9  }
10
11 agent UIAssistant {
12     provider: anthropic
13     system: """
14         You are a UI assistant. When answering, also suggest UI
15         actions.
16         For example, if the user asks about their order,
17         suggest navigating
18         to the orders page.
19     """
20     tools: [search_kb, lookup_order]
21     memory: conversation(max_turns: 50)
22 }
23
24 @webhook("/assistant")
25 workflow UIChat(message: string, session_id: string) ->
26     ChatResponse {
27         response: ChatResponse, err = UIAssistant.run(
28             { message: message },
29             session: session_id
30         )
31         if err != nil {
32             return ChatResponse{ reply: "Sorry, an error
33                 occurred.", actions: [] }
34         }
35         return response
36     }

```

A.8 Best Practices

1. **Always use sessions for conversation** — without a session, memory agents start fresh every call
2. **Set appropriate max_turns** — too high wastes tokens, too low loses context
3. **Use streaming for interactive UIs** — REST is fine for backend-to-backend, but users expect real-time responses

-
4. **Give tools clear descriptions** — the agent decides when to use tools based on descriptions
 5. **Use a Router agent for complex bots** — instead of one agent doing everything, route to specialists
 6. **Handle errors gracefully** — always provide a fallback response when agents fail

Appendix B

Generative UI

Generative UI allows agents to render rich, interactive components inline in a chat conversation instead of returning only text. When an agent calls a tool, the tool's result can be rendered as a structured UI component—a table, a card, a diff view, a form—rather than a plain text bubble.

HAIRA approaches generative UI at the language level. Tools declare what component renders their output. The runtime ships a built-in component catalog. No frontend framework, bundler, or client-side JavaScript is required.

B.1 Motivation

Consider a migration tool that returns a dry-run result:

```
1 tool dry_run(file: string) -> string {
2     """Run SQL against the database in dry-run mode"""
3     // ... returns "DRY-RUN FAILED on seeds.\n\nROOT CAUSE: ..."
4 }
```

The agent receives a string, streams it as markdown, and the user sees a wall of text. The information is there, but the presentation is poor—error details, affected tables, and fix SQL are all flattened into one block.

With generative UI, the same tool returns structured data and declares a renderer:

```
1 @render("status-card")
2 tool dry_run(file: string) -> DryRunResult {
3     """Run SQL against the database in dry-run mode"""
4     // ... returns structured result
5 }
```

The chat UI renders a status card with colored indicators, collapsible sections, and copy-paste SQL blocks—all from the same tool declaration, with zero frontend code.

B.2 The @render Decorator

Tools opt into generative UI with the @render decorator. It takes a component name from the built-in catalog:

```
1 @render("component-name")
2 tool tool_name(params) -> ReturnType {
3     """Description"""
```

```

4  // body
5  }

```

The decorator does three things:

1. Tells the compiler to include component metadata in the tool definition
2. Tells the runtime to emit a `tool_render` SSE event with the structured result
3. Tells the chat UI to render the named component instead of showing raw text

Tools without `@render` behave as before: the agent receives the result, interprets it, and streams a text response. Tools with `@render` still return their result to the agent for reasoning, but the UI additionally renders the component inline.

Note

The `@render` decorator is optional. Existing tools continue to work unchanged. Generative UI is additive.

B.3 Built-in Component Catalog

HAIRA ships a set of built-in components that cover common patterns. Each component expects a specific data shape and renders accordingly.

B.3.1 table — Tabular Data

Renders a table with headers and rows. Supports optional column alignment and row highlighting.

```

1  struct TableData {
2      title: string
3      headers: [string]
4      rows: [[string]]
5      highlight: [int]           // row indices to highlight
                                   (optional)
6  }
7
8  @render("table")
9  tool list_tables(file: string) -> TableData {
10     """List all tables found in the Excel config file"""
11
12     // ... extract tables ...
13     return TableData{
14         title: "Tables in " + filename,
15         headers: ["Table", "Rows", "Type"],
16         rows: [
17             ["parameters", "42", "configuration"],
18             ["batches", "8", "run"],
19         ],
20         highlight: [],
21     }
22 }

```

B.3.2 status-card — Result Summary

Renders a card with a status indicator (success, error, warning), title, and detail sections.

```

1 struct StatusCard {
2     status: string           // "success", "error", "warning",
                             // "info"
3     title: string
4     message: string
5     sections: [CardSection]
6 }
7
8 struct CardSection {
9     label: string
10    content: string           // markdown supported
11    style: string             // "default", "code", "error",
                             // "success"
12 }
13
14 @render("status-card")
15 tool validate(file: string) -> StatusCard {
16     """Validate Excel data against the database schema"""
17
18     // ... run validation ...
19     return StatusCard{
20         status: "error",
21         title: "Validation Failed",
22         message: "3 errors, 1 warning",
23         sections: [
24             CardSection{
25                 label: "Missing Columns",
26                 content: "'id_plant' in table 'parameters'",
27                 style: "error",
28             },
29         ],
30     }
31 }

```

B.3.3 code-block — Source Code or SQL

Renders a syntax-highlighted code block with a copy button and optional title.

```

1 struct CodeBlock {
2     title: string           // optional header
3     language: string        // "sql", "json", "haira", etc.
4     code: string
5 }
6
7 @render("code-block")
8 tool generate_sql(file: string) -> CodeBlock {
9     """Generate SQL INSERT statements from the config file"""
10
11    // ... generate SQL ...
12    return CodeBlock{
13        title: "Generated Seeds SQL",
14        language: "sql",
15        code: sql_output,

```

```

16     }
17 }

```

B.3.4 diff — Before/After Comparison

Renders a side-by-side or unified diff view.

```

1  struct DiffView {
2      title: string
3      before_label: string
4      after_label: string
5      before: string
6      after: string
7      language: string // syntax highlighting language
8  }
9
10 @render("diff")
11 tool compare_schemas(table: string) -> DiffView {
12     """Compare Excel schema with database schema for a table"""
13
14     return DiffView{
15         title: "Schema Diff: " + table,
16         before_label: "Database",
17         after_label: "Excel",
18         before: db_schema,
19         after: excel_schema,
20         language: "sql",
21     }
22 }

```

B.3.5 key-value — Property List

Renders a compact list of labeled values. Useful for metadata, summaries, and configuration display.

```

1  struct KeyValueList {
2      title: string
3      items: [KVItem]
4  }
5
6  struct KVItem {
7      key: string
8      value: string
9      style: string // "default", "success", "error",
10                  "muted"
11 }
12
13 @render("key-value")
14 tool file_info(file: string) -> KeyValueList {
15     """Show metadata about the uploaded config file"""
16
17     return KeyValueList{
18         title: "File Info",
19         items: [

```

```

19         KVItem{ key: "Filename", value: filename, style:
20             "default" },
21         KVItem{ key: "Tables", value: "12", style:
22             "default" },
23         KVItem{ key: "Rows", value: "347", style: "default"
24             },
25         KVItem{ key: "Status", value: "Valid", style:
26             "success" },
27     ],
28 }

```

B.3.6 progress — Multi-Step Progress

Renders a vertical progress tracker showing completed, active, and pending steps. Useful for tools that report pipeline state.

```

1 struct ProgressView {
2     title: string
3     steps: [ProgressStep]
4 }
5
6 struct ProgressStep {
7     name: string
8     status: string // "done", "active", "pending",
9                 "failed"
10    detail: string // optional detail text
11 }
12
13 @render("progress")
14 tool pipeline_status(mr_id: string) -> ProgressView {
15     "" "Check the deployment pipeline status""
16
17     return ProgressView{
18         title: "Deployment Pipeline",
19         steps: [
20             ProgressStep{ name: "Branch Created", status:
21                 "done", detail: "" },
22             ProgressStep{ name: "CI/CD Running", status:
23                 "active", detail: "2m 15s" },
24             ProgressStep{ name: "Merge", status: "pending",
25                 detail: "" },
26             ProgressStep{ name: "Deploy to QUA", status:
27                 "pending", detail: "" },
28         ],
29     }
30 }

```

B.3.7 form — Interactive Input

Renders a form that the user can fill and submit. The submission triggers a follow-up tool call or message. This is the foundation for human-in-the-loop approval flows.

```

1 struct FormView {
2     title: string

```

```

3     fields: [FormField]
4     submit_label: string
5     submit_action: string    // tool name or message prefix
6 }
7
8 struct FormField {
9     name: string
10    label: string
11    field_type: string        // "text", "select", "checkbox",
12                                "textarea"
13    value: string             // default value
14    options: [string]         // for "select" type
15    required: bool
16 }
17
18 @render("form")
19 tool request_approval(branch: string, mr_url: string) ->
20     FormView {
21         ""Request user approval before pushing to GitLab""
22
23         return FormView{
24             title: "Confirm Push to GitLab",
25             fields: [
26                 FormField{
27                     name: "confirm",
28                     label: "Push branch '" + branch + "' and create
29                             MR?",
30                     field_type: "select",
31                     value: "",
32                     options: ["Yes, push it", "No, cancel"],
33                     required: true,
34                 },
35                 FormField{
36                     name: "comment",
37                     label: "MR comment (optional)",
38                     field_type: "textarea",
39                     value: "",
40                     options: [],
41                     required: false,
42                 },
43             ],
44             submit_label: "Submit",
45             submit_action: "handle_approval",
46         }
47     }
48 }

```

When the user submits the form, the chat sends a message in the format: "[Form: handle_approval] confirm=Yes, push it; comment=Looks good", which the agent can interpret and act upon.

B.3.8 Component Summary

B.4 Streaming Protocol

Generative UI extends the existing SSE protocol with a new event type: `tool_render`.

Component	Use Case	Data Shape
table	Tabular results, lists	Headers + rows
status-card	Success/error summaries	Status + sections
code-block	SQL, JSON, code output	Language + code string
diff	Before/after comparisons	Two text blocks
key-value	Metadata, config display	Key-value pairs
progress	Pipeline/step tracking	Steps with status
form	Approvals, user input	Fields + submit action

Table B.1: Built-in generative UI components

B.4.1 Current Protocol

The existing protocol emits three event types:

```
event: tool_start
data: {"tool": "validate", "args": "{\"file\":
      \"/tmp/config.xlsx\"}"}

event: tool_end
data: {"tool": "validate", "ok": true}

data: {"delta": "The validation passed with..."}
```

B.4.2 Extended Protocol

When a tool has an `@render` decorator, the runtime emits `tool_render` between `tool_start` and `tool_end`:

```
event: tool_start
data: {"tool": "validate", "args": "{\"file\":
      \"/tmp/config.xlsx\"}"}

event: tool_render
data: {
  "tool": "validate",
  "component": "status-card",
  "props": {
    "status": "error",
    "title": "Validation Failed",
    "message": "3 errors, 1 warning",
    "sections": [...]
  }
}

event: tool_end
data: {"tool": "validate", "ok": false}
```

The `tool_render` event carries:

- `tool`: The tool name (for matching with the active tool card)
- `component`: The component name from the catalog
- `props`: The tool's return value, JSON-serialized, used as component props

The chat UI receives `tool_render`, looks up the component in its registry, and renders it inline—replacing or augmenting the tool card.

B.4.3 Dual Consumption

The tool result is consumed twice:

1. **By the UI:** The `tool_render` event renders the component for the user
2. **By the agent:** The result is still appended to the message history, so the agent can reason about it and decide the next step

This means the agent sees the structured data (to make decisions) and the user sees the rich component (for readability). Neither path is sacrificed.

B.5 Agent Behavior with Generative UI

Agents do not need any changes to use generative UI. The `@render` decorator is on the tool, not the agent. An agent with a mix of rendered and non-rendered tools works naturally:

```

1  agent Maltimize {
2      provider: azure_openai
3      system: "You help deploy Excel configs to PostgreSQL."
4      tools: [
5          read_excel,           // no @render -- text result
6          validate,            // @render("status-card")
7          generate_sql,        // @render("code-block")
8          dry_run,             // @render("status-card")
9          push_to_gitlab,       // no @render -- text result
10     ]
11     memory: conversation(max_turns: 20)
12     temperature: 0.3
13 }
```

When the agent calls `validate`, the user sees a status card. When it calls `push_to_gitlab`, the user sees the agent's text interpretation. Both are valid—generative UI is opt-in per tool.

B.6 Fallback Behavior

Generative UI degrades gracefully in two scenarios:

B.6.1 Non-Chat Contexts

When a tool with `@render` is called outside a chat workflow (e.g., from a form workflow or a plain function), the decorator is ignored. The tool returns its structured data normally. The `@render` decorator only takes effect when the tool is executed through an agent's streaming loop.

B.6.2 API Consumers

When the SSE stream is consumed by a programmatic client (not the built-in chat UI), the `tool_render` event provides the component name and props as JSON. The client can:

- Render its own implementation of the component

- Ignore `tool_render` events and use only `tool_end` for status
- Use the `props` data directly for custom processing

B.7 Complete Example

A full migration assistant with generative UI:

```

1  import "io"
2  import "http"
3  import "excel"
4  import "postgres"
5
6  provider azure_openai {
7      type: "azure-openai"
8      model: env("AZURE_OPENAI_DEPLOYMENT_NAME")
9  }
10
11  // --- Rendered tools ---
12
13  @render("table")
14  tool read_config(file_path: string) -> TableData {
15      """Read an Excel config file and list the tables found"""
16
17      wb = excel.open(file_path)?
18      index = wb.read_sheet("index"?
19      rows = []
20      for entry in index {
21          table = "" + entry["Table"]
22          sheet = wb.read_sheet(get_sheet(table)) or else []
23          rows = rows + [[table, conv.int_to_string(len(sheet)),
24                          get_type(table)]]
25      }
26      wb.close()
27      return TableData{
28          title: "Config Tables",
29          headers: ["Table", "Rows", "Type"],
30          rows: rows,
31          highlight: [],
32      }
33  }
34
35  @render("status-card")
36  tool dry_run(file_path: string) -> StatusCard {
37      """Dry-run the generated SQL against the local database"""
38
39      // ... generate and test SQL ...
40      if result["ok"] == true {
41          return StatusCard{
42              status: "success",
43              title: "Dry Run Passed",
44              message: "All SQL is valid",
45              sections: [],
46          }
47      }
48      return StatusCard{
49          status: "error",

```

```

49     title: "Dry Run Failed",
50     message: result["error"],
51     sections: [
52         CardSection{
53             label: "Root Cause",
54             content: analysis["root_cause"],
55             style: "error",
56         },
57         CardSection{
58             label: "Fix SQL",
59             content: "``sql\n" + analysis["fix"] + "\n``",
60             style: "code",
61         },
62     ],
63 }
64 }
65
66 // --- Agent ---
67
68 agent ConfigAssistant {
69     provider: azure_openai
70     system: "You deploy Excel configs to PostgreSQL via GitLab."
71     tools: [read_config, dry_run, push_to_gitlab]
72     memory: conversation(max_turns: 20)
73     temperature: 0.3
74 }
75
76 // --- Workflow ---
77
78 @webui(title: "Config Assistant", description: "Chat-based
79     config migration")
80 @webhook("/api/chat")
81 workflow Chat(message: string, session_id: string, file_path:
82     file) -> stream {
83     msg = message
84     if file_path != "" { msg = "${message}\n\n[Attached file:
85         ${file_path}]" }
86     return ConfigAssistant.stream(msg, session: session_id)
87 }
88
89 fn main() {
90     server = http.Server([Chat])
91     server.listen(9001)
92 }

```

The user uploads an Excel file and says “deploy this”. The agent calls `read_config`—the chat renders a table showing all tables with row counts. Then calls `dry_run`—the chat renders a status card showing success or failure with detailed sections. If it passes, the agent asks for confirmation before pushing.

All of this happens in a standard HAIRA program. No React, no Next.js, no bundler, no client-side framework.

B.8 Custom Components

The built-in catalog covers common patterns, but domain-specific use cases need custom components. HAIRA supports two levels of customization: composite components built from

built-in primitives, and fully custom components written as TypeScript Web Components.

B.8.1 Composite Components

The simplest way to extend the UI is to compose built-in components. A composite component is a Haira struct whose fields map to multiple built-in components, rendered together as a group:

```

1  @render("composite")
2  tool migration_report(file: string) -> MigrationReport {
3      """Generate a full migration report with tables,
4          validation, and SQL"""
5
6      return MigrationReport{
7          components: [
8              Component{
9                  type: "key-value",
10                 props: KeyValueList{
11                     title: "File Summary",
12                     items: [
13                         KVIItem{ key: "File", value: filename,
14                             style: "default" },
15                         KVIItem{ key: "Tables", value: "12",
16                             style: "default" },
17                     ],
18                 },
19             },
20             Component{
21                 type: "table",
22                 props: TableData{
23                     title: "Tables",
24                     headers: ["Name", "Rows", "Type"],
25                     rows: table_rows,
26                     highlight: [],
27                 },
28             },
29             Component{
30                 type: "status-card",
31                 props: StatusCard{
32                     status: "success",
33                     title: "Validation Passed",
34                     message: "All checks passed",
35                     sections: [],
36                 },
37             },
38         ],
39     }
40 }

```

The `composite` renderer lays out multiple built-in components vertically in a single tool render. This requires no custom JavaScript—only data.

B.8.2 Custom Web Components

For fully custom rendering, users write a TypeScript Web Component in a `components/` directory next to their HAIRA source:

```

1 project/
2   main.haira
3   components/
4     gantt-chart.ts
5     schema-diagram.ts

```

A custom component file follows a simple contract:

```

1 // components/gantt-chart.ts
2
3 interface GanttProps {
4   title: string;
5   tasks: Array<{
6     name: string;
7     start: string;
8     end: string;
9     progress: number;
10  }>;
11 }
12
13 export class HairaGanttChart extends HTMLElement {
14   connectedCallback() {
15     this.attachShadow({ mode: "open" });
16   }
17
18   // Called by the runtime with the tool's return value
19   setProps(props: GanttProps) {
20     const shadow = this.shadowRoot!;
21     shadow.innerHTML = `
22       <style>
23         :host { display: block; }
24         .chart {
25           padding: 1rem;
26           background: var(--haira-bg-card);
27           border: 1px solid var(--haira-border);
28           border-radius: var(--haira-radius);
29         }
30         .title {
31           font-weight: 600;
32           font-size: 0.85rem;
33           color: var(--haira-text);
34           margin-bottom: 0.75rem;
35         }
36         .bar-row {
37           display: flex;
38           align-items: center;
39           gap: 0.5rem;
40           margin: 0.35rem 0;
41         }
42         .bar-label {
43           font-size: 0.78rem;
44           color: var(--haira-text-dim);
45           min-width: 120px;
46         }
47         .bar-track {
48           flex: 1;
49           height: 20px;
50           background: var(--haira-bg-elevated);

```

```

51         border-radius: var(--haira-radius-sm);
52         overflow: hidden;
53     }
54     .bar-fill {
55         height: 100%;
56         background: var(--haira-gold);
57         border-radius: var(--haira-radius-sm);
58         transition: width 0.3s;
59     }
60     .bar-pct {
61         font-size: 0.7rem;
62         color: var(--haira-muted);
63         font-family: var(--haira-mono);
64         min-width: 35px;
65     }
66     </style>
67     <div class="chart">
68         <div
69             class="title">${this.esc(props.title)}</div>
70         ${props.tasks.map(t => '
71             <div class="bar-row">
72                 <span
73                     class="bar-label">${this.esc(t.name)}</span>
74                 <div class="bar-track">
75                     <div class="bar-fill"
76                         style="width:
77                             ${t.progress}%></div>
78                 </div>
79                 <span
80                     class="bar-pct">${t.progress}%</span>
81                 </div>
82             ').join("")}
83         </div>
84     ';
85 }
86
87 private esc(s: string): string {
88     return s.replace(/&/g, "&amp;")
89         .replace(/</g, "&lt;")
90         .replace(/>/g, "&gt;");
91 }
92
93 // Export the component class and its custom element tag name
94 export default {
95     tag: "haira-gantt-chart",
96     component: HairaGanttChart,
97 };

```

The component must:

1. Extend `HTMLElement`
2. Use Shadow DOM for style isolation
3. Implement a `setProps(props)` method that receives the tool's return value as a typed object

4. Export a default object with **tag** (the custom element name) and **component** (the class)

The component is referenced in **@render** by its file name (without extension):

```

1 struct GanttData {
2     title: string
3     tasks: [Task]
4 }
5
6 struct Task {
7     name: string
8     start: string
9     end: string
10    progress: int
11 }
12
13 @render("gantt-chart")
14 tool project_timeline(project_id: string) -> GanttData {
15     """Show the project timeline as a Gantt chart"""
16
17     return GanttData{
18         title: "Q1 Roadmap",
19         tasks: [
20             Task{ name: "Backend API", start: "2025-01-01",
21                 end: "2025-02-15", progress: 80 },
22             Task{ name: "Frontend", start: "2025-02-01",
23                 end: "2025-03-15", progress: 40 },
24         ],
25     }
26 }

```

B.8.3 Component Resolution

When the compiler encounters **@render("name")**, it resolves the component in this order:

1. **Built-in catalog:** Check if name matches a built-in component (table, status-card, composite, etc.)
2. **Local components:** Check if components/name.ts exists relative to the .haira source file
3. **Compile error:** If neither matches, emit a diagnostic with the list of available component names

Example diagnostic:

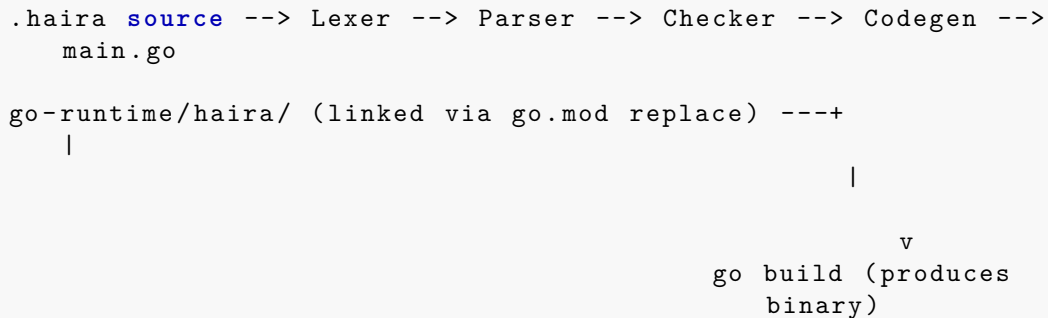
```

error: unknown render component "gantt-chart"
--> main.haira:42:9
  |
42 | @render("gantt-chart")
  |           ~~~~~
  |
  = available: table, status-card, code-block, diff,
                key-value, progress, form, composite
  = hint: create components/gantt-chart.ts to register
          a custom component

```


B.8.4 Build Process

The HAIRA compiler handles custom components as part of its normal build pipeline. The existing pipeline is:



The runtime's built-in UI is a pre-compiled JavaScript bundle (`ui/dist/haira-ui.js`) embedded in the Go binary via `//go:embed`. HTML templates contain a `<script>` tag that loads this bundle at `/_ui/assets/haira-ui.js`.

Custom components extend this pipeline with one additional step:

```

haira build main.haira

1. Parse, check, codegen as normal
2. Scan components/ directory for .ts files
3. Generate components/_register.ts (entry point):
    import c0 from "./gantt-chart.ts";
    import c1 from "./schema-diagram.ts";
    customElements.define(c0.tag, c0.component);
    customElements.define(c1.tag, c1.component);
4. Bundle components/_register.ts --> components.js
   (using bun build --minify --target browser)
5. Embed components.js as a Go string constant in
   the generated main.go:
    const customComponentsJS = `...minified JS...`
6. Register a second asset route:
    GET /_ui/assets/components.js
7. HTML templates include a second <script> tag
   after the built-in UI bundle

```

The generated `main.go` includes the custom bundle:

```
1 // Generated by haira build
2 package main
3
4 import "haira-go-runtime/haira"
5
6 // Custom components bundle (empty if no components/ dir)
7 const customComponentsJS = `(function(){class HairaGanttChart
8 extends HTMLElement{...}customElements.define("haira-gantt-
9 chart",HairaGanttChart);...})()`
10
11 func main() {
12     server := haira.NewServer(workflows)
13     server.SetCustomComponentsJS(customComponentsJS)
14     server.Listen(9001)
15 }
```

The server serves the custom bundle at `/_ui/assets/components.js` and the HTML template includes both scripts:

```
1 <script src="/_ui/assets/haira-ui.js"></script>
2 <script src="/_ui/assets/components.js"></script>
```

If no `components/` directory exists, the custom bundle is empty and the second `<script>` tag is omitted. Existing projects are unaffected.

Note

The compiler requires `bun` to be installed for TypeScript compilation of custom components. If `bun` is not found and custom components exist, the compiler emits a clear error with installation instructions. Projects without custom components do not require `bun`.

B.8.5 Referencing Components from Tools

The connection between a tool and its component is the `@render` decorator. The decorator name must match either a built-in component or the filename (without `.ts`) of a file in `components/`:

```
1 // Built-in: name matches catalog entry
2 @render("table")
3 tool list_items(...) -> TableData { ... }
4
5 // Custom: name matches components/gantt-chart.ts
6 @render("gantt-chart")
7 tool project_timeline(...) -> GanttData { ... }
8
9 // Custom: name matches components/schema-diagram.ts
10 @render("schema-diagram")
11 tool show_schema(...) -> SchemaView { ... }
```

The compiler validates this mapping at compile time. The runtime connects them at render time: when a `tool_render` SSE event arrives with `"component": "gantt-chart"`, the chat UI looks up the custom element registered under that name, creates an instance, and calls `setProps()` with the tool's result data.

The custom element tag name is defined by the component's `export default { tag: "..."} — the convention is haira-{name} (e.g., haira-gantt-chart for gantt-chart.ts), but the developer controls it. The runtime matches by the @render name, not the tag name.`

Note

Custom components run in the browser's Shadow DOM sandbox. They cannot access the parent page's DOM or interfere with other components. The HAIRA runtime provides theme CSS custom properties (`-haira-gold`, `-haira-bg`, etc.) that custom components can use for consistent styling.

B.8.6 Theme Integration

Custom components inherit HAIRA's theme through CSS custom properties. The runtime injects these variables into the document root, making them available inside Shadow DOM:

```

1  /* Inside a custom component's Shadow DOM styles: */
2  :host {
3      display: block;
4      font-family: var(--haira-font);
5      color: var(--haira-text);
6  }
7  .chart-bar {
8      background: var(--haira-gold);
9      border-radius: var(--haira-radius-sm);
10 }
11 .error { color: var(--haira-error); }
12 .success { color: var(--haira-success); }

```

Available CSS custom properties:

Variable	Purpose
-haira-bg	Page background
-haira-bg-card	Card/panel background
-haira-bg-elevated	Elevated surface background
-haira-border	Border color
-haira-gold	Brand accent (gold)
-haira-gold-light	Lighter gold variant
-haira-text	Primary text color
-haira-text-dim	Secondary text color
-haira-muted	Muted/disabled text
-haira-success	Success state (green)
-haira-error	Error state (red)
-haira-warn	Warning state (yellow)
-haira-info	Info state (blue)
-haira-font	System font stack
-haira-mono	Monospace font stack
-haira-radius	Standard border radius
-haira-radius-sm	Small border radius

Table B.2: CSS custom properties available to custom components

B.8.7 Interactive Components

Custom components can be interactive. When a user interacts with a component (clicks a button, selects a row, submits a form), the component dispatches a custom DOM event that the chat UI captures and sends as a follow-up message to the agent:

```

1  // Inside a custom component's setProps():
2  const btn = shadow.getElementById("approve-btn")!;
3  btn.addEventListener("click", () => {
4      this.dispatchEvent(new CustomEvent("haira-action", {
5          bubbles: true,
6          composed: true,      // crosses Shadow DOM boundary
7          detail: {
8              action: "approve",
9              data: { branch: "feature/config-123" },
10         },
11     }));

```

```
12 });
```

The chat UI listens for `haira-action` events and sends the action to the agent as a message: "[Action: approve] {branch: feature/config-123}". The agent can then interpret this and call the appropriate tool.

This pattern enables:

- Clickable table rows that trigger drill-down queries
- Approval/reject buttons on deployment cards
- Filter controls that re-run a tool with different parameters
- Form submissions within custom components

B.8.8 Example: Custom Schema Diagram

A complete custom component for visualizing database table relationships:

```
1 // components/schema-diagram.ts
2
3 interface SchemaProps {
4   title: string;
5   tables: Array<{
6     name: string;
7     columns: Array<{
8       name: string;
9       type: string;
10      pk: boolean;
11      fk: boolean;
12    }>;
13   }>;
14 }
15
16 export class HairaSchemaDiagram extends HTMLElement {
17   connectedCallback() {
18     this.attachShadow({ mode: "open" });
19   }
20
21   setProps(props: SchemaProps) {
22     const shadow = this.shadowRoot!;
23     shadow.innerHTML = `
24       <style>
25         :host { display: block; }
26         .diagram {
27           padding: 1rem;
28           background: var(--haira-bg-card);
29           border: 1px solid var(--haira-border);
30           border-radius: var(--haira-radius);
31           display: flex;
32           flex-wrap: wrap;
33           gap: 0.75rem;
34         }
35         .table-box {
36           border: 1px solid var(--haira-border);
37           border-radius: var(--haira-radius-sm);
38           min-width: 150px;
39         }

```

```

40      .table-name {
41        padding: 0.4rem 0.6rem;
42        background: var(--haira-gold);
43        color: #1a0e04;
44        font-weight: 700;
45        font-size: 0.8rem;
46        border-radius:
47          var(--haira-radius-sm)
48          var(--haira-radius-sm) 0 0;
49      }
50      .col {
51        padding: 0.25rem 0.6rem;
52        font-size: 0.75rem;
53        color: var(--haira-text-dim);
54        font-family: var(--haira-mono);
55        border-top: 1px solid var(--haira-border);
56      }
57      .col.pk { color: var(--haira-gold); }
58      .col.fk { color: var(--haira-info); }
59      .title {
60        font-weight: 600;
61        font-size: 0.85rem;
62        color: var(--haira-text);
63        width: 100%;
64        margin-bottom: 0.25rem;
65      }
66    </style>
67    <div class="diagram">
68      <div
69        class="title">${this.esc(props.title)}</div>
70      ${props.tables.map(t => '
71        <div class="table-box">
72          <div
73            class="table-name">${this.esc(t.name)}</div>
74            ${t.columns.map(c => '
75              <div class="col ${c.pk ? "pk" :
76                c.fk ? "fk" : ""}">
77                ${this.esc(c.name)}:
78                ${this.esc(c.type)}
79              </div>
80            ').join("")}
81          </div>
82          ${t.columns.map(c => '
83            <div class="col ${c.pk ? "pk" :
84              c.fk ? "fk" : ""}">
85              ${this.esc(c.name)}:
86              ${this.esc(c.type)}
87            </div>
88            ').join("")}
89          </div>
90        ');
91      }
92      private esc(s: string): string {
93        return s.replace(/&/g, "&")
94          .replace(/</g, "&lt;")
95          .replace(/>/g, "&gt;");
96      }
97    }
98    export default {
99      tag: "haira-schema-diagram",
100      component: HairaSchemaDiagram,
101    };

```

Used in HAIRA:

```

1  struct SchemaView {
2      title: string
3      tables: [TableView]
4  }
5
6  struct TableView {
7      name: string
8      columns: [ColumnView]
9  }
10
11 struct ColumnView {
12     name: string
13     type: string
14     pk: bool
15     fk: bool
16 }
17
18 @render("schema-diagram")
19 tool show_schema(table_names: string) -> SchemaView {
20     """Visualize the database schema for the given tables"""
21
22     // ... query schema, build view ...
23     return SchemaView{
24         title: "Database Schema",
25         tables: tables,
26     }
27 }

```

The result: the agent calls `show_schema`, and the chat renders an interactive diagram with color-coded primary and foreign keys—no React, no build system, just a `.ts` file next to the `.haira` source.

B.9 Design Principles

1. **Tools own their rendering.** The `@render` decorator lives on the tool, not the agent or workflow. This keeps rendering co-located with the data source.
2. **Structured data, not markup.** Tools return typed structs, not HTML or JSX. The runtime maps data to components. This keeps tools testable and platform-independent.
3. **Catalog first, custom when needed.** The built-in catalog covers common patterns with zero configuration. Custom components extend the catalog with a single `.ts` file—no framework, no bundler, no build pipeline.
4. **Additive, not breaking.** Every tool works without `@render`. Adding it improves the UI without changing the tool's logic or the agent's behavior.
5. **Dual consumption.** The UI and the agent both receive the tool result. The UI renders a component; the agent reasons about the data. Neither path is secondary.
6. **Progressive complexity.** Simple needs use built-in components (data only). Moderate needs use composites (multiple built-ins). Advanced needs use custom TypeScript Web Components (one `.ts` file). The learning curve matches the complexity of the use case.

7. **Interactivity through events.** Custom components communicate back to the agent through a single event contract (`haira-action`). No callback registration, no bidirectional binding—just events that become messages.

B.10 External Frontend API

The built-in chat UI is a convenience, not a requirement. HAIRA exposes a complete HTTP API that gives external frontends—React, Vue, Svelte, mobile apps, or any HTTP client—the same capabilities as the built-in UI. The built-in UI is itself just a consumer of this API.

B.10.1 Discovery: GET `/_api/`

Returns metadata about all registered workflows. This is the same data the built-in UI uses to render the index page and route to form/chat views.

```
GET /_api/

{
  "workflows": [
    {
      "name": "UploadConfig",
      "path": "/api/upload-config",
      "method": "POST",
      "ui_type": "form",
      "title": "Maltimize Config",
      "description": "Excel-to-PostgreSQL configuration migration",
      "params": [
        { "name": "file_path", "type": "file", "required": true },
        { "name": "debug", "type": "string", "required": false }
      ],
      "steps": [
        "Open Excel", "Extract table data", "Query schema",
        "Validate", "Generate SQL", "Dry-run SQL",
        "Push to GitLab", "Wait for CI/CD",
        "Wait for merge", "Verify deployment"
      ]
    },
    {
      "name": "Chat",
      "path": "/api/chat",
      "method": "POST",
      "ui_type": "chat",
      "title": "Maltimize Chat",
      "description": "Chat-based config migration assistant",
      "params": [
        { "name": "message", "type": "string", "required": true },
        { "name": "session_id", "type": "string", "required": true },
      ]
    }
  ]
}
```

```

        { "name": "file_path", "type": "file",
          "required": false }
      ],
      "steps": [],
      "tools": [
        {
          "name": "read_config_excel",
          "description": "Read an Excel configuration
            file...",
          "render": null
        },
        {
          "name": "validate_config",
          "description": "Validate Excel config
            data...",
          "render": "status-card"
        },
        {
          "name": "dry_run_migration",
          "description": "Run the generated SQL...",
          "render": "status-card"
        }
      ],
      "components": [
        {
          "name": "status-card",
          "type": "builtin"
        },
        {
          "name": "gantt-chart",
          "type": "custom",
          "tag": "haira-gantt-chart"
        }
      ]
    }
  ]
}

```

The `tools` array lists each tool available to the agent, including its `render` component (or `null` if the tool has no UI). The `components` array lists all generative UI components used by the workflow, distinguishing built-in from custom.

B.10.2 Workflow Metadata: GET `/_api/{path}`

Returns detailed metadata for a single workflow:

```

GET /_api/api/chat

{
  "name": "Chat",
  "path": "/api/chat",
  "method": "POST",
  "ui_type": "chat",
  "title": "Maltimize Chat",
  "description": "Chat-based config migration assistant",
  "params": [...],
  "tools": [...],

```



```
"components": [...]  
}
```

B.10.3 Streaming: POST /{path} with SSE

The same endpoint used by the built-in UI. An external client sends a request with `Accept: text/event-stream` to receive SSE events:

```
$ curl -N -H "Accept: text/event-stream" \  
  -d '{"message": "deploy this", "session_id": "abc123"}' \  
  http://localhost:9001/api/chat
```

The SSE stream emits these event types:

Event	When	Data
tool_start	Tool begins	{"tool": "name", "args": "{...}"}
tool_render	Tool has UI result	{"tool": "name", "component": "status-card", "props": {...}}
tool_end	Tool finishes	{"tool": "name", "ok": true}
(default)	Text delta	{"delta": "text chunk"}
step	Pipeline step event	{"name": "...", "status": "start end failed"}
[DONE]	Stream complete	—

Table B.3: SSE event types

An external frontend handles these events the same way the built-in UI does:

1. On `tool_start`: show a loading indicator for the tool
2. On `tool_render`: render the component using the `component` name and `props` data. For built-in components, the frontend implements its own version (e.g., a React `<StatusCard>`). For custom components, it uses the `props` JSON directly.
3. On `tool_end`: mark the tool as complete (success or failure)
4. On text deltas: append to the assistant message
5. On `[DONE]`: finalize the response

B.10.4 Form Submission: POST /{path} with JSON

For non-streaming workflows (form UI), the same endpoint accepts JSON and returns JSON:

```
$ curl -X POST http://localhost:9001/api/upload-config \  
  -F "file_path=@config.xlsx" \  
  -F "debug=true"  
  
# Response with step-by-step SSE:  
event: step  
data: {"name": "Open Excel", "status": "start"}  
  
event: step
```

```
data: {"name": "Open Excel", "status": "end", "duration_ms":
  120}

# ... more steps ...

event: result
data: {"status": "success", "message": "Configuration deployed"}
```

Without Accept: `text/event-stream`, the response is plain JSON:

```
$ curl -X POST http://localhost:9001/api/upload-config \
  -F "file_path=@config.xlsx"

{"status": "success", "message": "Configuration deployed to
  QUA"}
```

B.10.5 CORS Support

When an external frontend runs on a different origin (e.g., `localhost:3000` for a React dev server), the HAIRA server enables CORS automatically when the `HAIRA_CORS_ORIGIN` environment variable is set:

```
$ HAIRA_CORS_ORIGIN="http://localhost:3000" ./maltimize
```

The wildcard `*` is also supported for development:

```
$ HAIRA_CORS_ORIGIN="*" ./maltimize
```

B.10.6 External Frontend Example

A React frontend consuming the HAIRA API:

```
1 // React component consuming Haira's SSE API
2
3 function Chat() {
4   const [messages, setMessages] = useState([]);
5   const [tools, setTools] = useState(new Map());
6
7   async function send(text: string, file?: File) {
8     // Build request
9     const body = new FormData();
10    body.append("message", text);
11    body.append("session_id", sessionId);
12    if (file) body.append("file_path", file);
13
14    // Open SSE connection
15    const response = await fetch("/api/chat", {
16      method: "POST",
17      body,
18      headers: { Accept: "text/event-stream" },
19    });
20
21    const reader = response.body.getReader();
22    const decoder = new TextDecoder();
```

```

23
24     while (true) {
25         const { done, value } = await reader.read();
26         if (done) break;
27
28         const lines = decoder.decode(value).split("\n");
29         for (const line of lines) {
30             if (line.startsWith("event: tool_start")) {
31                 // Show loading state for the tool
32             }
33             if (line.startsWith("event: tool_render")) {
34                 const data = JSON.parse(/* next data line
35                                         */);
36                 // data.component = "status-card"
37                 // data.props = { status: "error", ... }
38                 // Render <StatusCard {...data.props} />
39             }
40             if (line.startsWith("data: ")) {
41                 const data = JSON.parse(line.slice(6));
42                 if (data.delta) {
43                     // Append text to assistant message
44                 }
45             }
46         }
47     }
48
49     return (
50         <div>
51             {messages.map(msg =>
52                 msg.component
53                 ? <ComponentRenderer
54                     name={msg.component}
55                     props={msg.props} />
56                 : <MessageBubble text={msg.text} />
57             )}
58         </div>
59     );
60 }
61
62 // Map Haira component names to React components
63 function ComponentRenderer({ name, props }) {
64     const components = {
65         "status-card": StatusCard,
66         "table": DataTable,
67         "code-block": CodeBlock,
68         "gantt-chart": GanttChart, // custom component
69     };
70     const Component = components[name];
71     return Component ? <Component {...props} /> : null;
72 }

```

The external frontend renders its own components for each `tool_render` event. The `component` name and `props` JSON provide all the information needed—the frontend maps component names to its own implementations (React, Vue, Svelte, or native mobile components).

B.10.7 API Parity Guarantee

The built-in UI has no privileged access. Every feature available in the built-in chat and form UIs is available through the HTTP API:

Feature	Built-in UI	External API
Workflow discovery	Index page	GET <code>/_api/</code>
Workflow metadata	<code><script id="haira-meta"></code>	GET <code>/_api/{path}</code>
Form submission	<code><haira-form></code> submit	POST <code>/{path}</code> (JSON)
Chat streaming	<code><haira-chat></code> SSE	POST <code>/{path}</code> (SSE)
Tool execution status	<code><haira-tool-card></code>	<code>tool_start/tool_end</code> events
Generative UI render	<code><haira-*></code> components	<code>tool_render</code> event + props
File upload	Drag & drop / attach	<code>multipart/form-data</code>
Step pipeline	<code><haira-pipeline></code>	<code>step</code> events

Table B.4: API parity between built-in UI and external frontends

Note

The built-in UI can be disabled entirely with the `HAIRA_DISABLE_UI` environment variable. The API endpoints remain active. This is useful when deploying HAIRA as a pure backend behind a separate frontend application.

B.11 Grammar Extension

The `@render` decorator uses the existing decorator syntax. No new grammar is required:

```

1  tool_decl      = { decorator } "tool" identifier "(" [ params ]
                    ")" [ ">" type ]
2                  "{" triple_string { statement } "}" ;
3
4  decorator      = "@" identifier [ "(" decorator_args ")" ] ;

```

The compiler validates that the component name in `@render("name")` matches a known component from the catalog. Unknown component names produce a compile-time error.

Appendix C

Reserved Keywords

The following identifiers are reserved as keywords and cannot be used as identifiers:

<code>agent</code>	<code>and</code>	<code>as</code>	<code>break</code>	<code>catch</code>
<code>chan</code>	<code>const</code>	<code>continue</code>	<code>defer</code>	<code>else</code>
<code>enum</code>	<code>export</code>	<code>false</code>	<code>fn</code>	<code>for</code>
<code>from</code>	<code>if</code>	<code>impl</code>	<code>import</code>	<code>in</code>
<code>match</code>	<code>nil</code>	<code>not</code>	<code>or</code>	<code>provider</code>
<code>pub</code>	<code>return</code>	<code>select</code>	<code>spawn</code>	<code>struct</code>
<code>tool</code>	<code>trait</code>	<code>true</code>	<code>try</code>	<code>type</code>
<code>unsafe</code>	<code>where</code>	<code>while</code>	<code>workflow</code>	

Appendix D

Operator Precedence

Operators are listed from highest to lowest precedence:

Precedence	Operators	Associativity
1 (highest)	() [] .	Left
2	! - ~ (unary) not	Right
3	* / %	Left
4	+ -	Left
5	« »	Left
6	& (bitwise AND)	Left
7	^ (bitwise XOR)	Left
8	(bitwise OR)	Left
9	=	None
10	> (pipe)	Left
11	< > <= >=	Left
12	== !=	Left
13	and &&	Left
14	or	Left
15 (lowest)	= += -= *= /= &= = ^= <= >=	Right

Appendix E

Standard Library Reference

Quick reference for standard library modules. See Chapter 10 for full documentation.

Module	Description
<code>io</code>	Input/output operations
<code>string</code>	String manipulation
<code>math</code>	Mathematical functions
<code>array</code>	Array operations
<code>map</code>	Map/dictionary operations
<code>json</code>	JSON encoding/decoding
<code>http</code>	HTTP client/server
<code>fs</code>	File system operations
<code>os</code>	Operating system interface
<code>time</code>	Time and duration
<code>regex</code>	Regular expressions
<code>encoding</code>	Base64/hex encoding
Tool Packs (importable tools for agents)	
<code>tools/http</code>	HTTP tools (get, post, put, delete)
<code>tools/slack</code>	Slack integration (send, read)
<code>tools/github</code>	GitHub API (create_issue, list_prs)
<code>tools/postgres</code>	PostgreSQL (query, insert)
<code>tools/email</code>	Email sending
<code>tools/fs</code>	File system tools for agents